Prognostic Modelling

pea700 · MSc Physics of the Earth and Atmosphere · Universities of Bonn and Cologne

Stefan Kollet

Invalid Date

Table of contents

Preface 5

Syllabus 6
Learning goals . . . 6
Outline . . . 6

I Foundations 8

1 What is prognostic modelling? 9
1.1 Models of the Earth system . . . 9
1.1.1 A first mathematical model . . . 10
1.1.2 Prognostic and diagnostic equations . . . 11
1.2 From physical law to computer code . . . 11
1.2.1 The central idea: replace derivatives by differences . . . 12
1.2.2 A first analysis . . . 14
1.3 Sources of error and uncertainty . . . 15
1.4 Floating point arithmetic and round-off . . . 17
1.4.1 How a computer stores real numbers . . . 17
1.4.2 Loss of significance . . . 18
1.4.3 Conditioning . . . 19
1.5 Concepts that return in every unit . . . 21
1.6 Verification and validation . . . 21
1.7 Working practice . . . 22
1.8 Summary . . . 22
1.9 Further reading . . . 23

2 Governing equations and their classification 24
2.1 Vector calculus recap . . . 24
2.1.1 Derivatives . . . 24
2.1.2 Vectors . . . 24
2.1.3 The gradient . . . 25
2.1.4 The divergence . . . 26
2.1.5 Curl and Laplacian . . . 27
2.2 Conservation laws . . . 30

2.3 Classification of differential equations . . . 33
2.3.1 Ordinary and partial differential equations . . . 33
2.3.2 Systems . . . 33
2.3.3 Order . . . 33
2.3.4 Linear and nonlinear . . . 34
2.3.5 Elliptic, parabolic, hyperbolic . . . 35
2.4 Prototype equations . . . 36
2.5 The equations of this lecture . . . 40
2.6 Initial and boundary conditions . . . 43
2.7 Well-posed problems . . . 44
2.8 Summary . . . 45
2.9 Further reading . . . 45

3 The numerical toolbox I: derivatives, integrals, roots 47
3.1 Taylor series and finite differences . . . 48
3.1.1 The Taylor series . . . 48
3.1.2 Forward and backward difference . . . 48
3.1.3 Central difference . . . 49
3.1.4 Second derivative . . . 50
3.1.5 Measuring the order of convergence . . . 51
3.1.6 Why the step cannot be made arbitrarily small . . . 53
3.1.7 Stencils of any order . . . 56
3.2 Interpolation . . . 59
3.2.1 Linear interpolation . . . 59
3.2.2 Polynomials of high degree and the Runge phenomenon . . . 61
3.3 Quadrature . . . 64
3.3.1 Midpoint and trapezoidal rule . . . 64
3.3.2 Simpson's rule . . . 66
3.3.3 Where quadrature enters prognostic models . . . 68
3.4 Root finding . . . 68
3.4.1 Bisection . . . 69
3.4.2 Newton–Raphson . . . 70
3.4.3 Newton's method for systems . . . 73
3.5 Summary . . . 74
3.6 Further reading . . . 75

4 The numerical toolbox II: matrices and waves 76
4.1 From a differential equation to a matrix . . . 76
4.1.1 A model problem . . . 76
4.1.2 Sparsity . . . 77
4.1.3 Symmetry and positive definiteness . . . 77
4.1.4 Eigenvalues and eigenvectors . . . 78

4.2 Norms and the condition number . . . 79
4.2.1 Norms . . . 79
4.2.2 The condition number . . . 80
4.2.3 The condition number of the model matrix . . . 81
4.3 Fourier analysis . . . 81
4.3.1 Waves on a grid . . . 81
4.3.2 The discrete Fourier transform . . . 82
4.3.3 The shortest wave and aliasing . . . 83
4.3.4 Why waves are the natural test for a numerical method . . . 84
4.3.5 The second difference and the model matrix . . . 86
4.4 Summary . . . 86
4.5 Further reading . . . 86

II Time integration of ODEs **87**

5 One-step methods **88**
5.1 The initial value problem . . . 88
5.1.1 Direction fields . . . 89
5.1.2 An exact solution . . . 90
5.2 The forward Euler method . . . 91
5.2.1 First view: following the tangent . . . 91
5.2.2 Second view: replacing the derivative . . . 92
5.2.3 Third view: integrating the tendency . . . 92
5.2.4 Implementation . . . 93
5.3 The backward Euler method . . . 94
5.4 Errors . . . 95
5.4.1 Local and global error . . . 95
5.4.2 Local truncation error of the forward Euler method . . . 96
5.4.3 From the local to the global error . . . 96
5.5 Methods of higher order . . . 98
5.5.1 Heun's method . . . 98
5.5.2 The order of two-stage methods . . . 98
5.5.3 Runge–Kutta methods . . . 99
5.5.4 Verification . . . 100
5.5.5 Accuracy for a given amount of work . . . 102
5.6 Systems of equations . . . 103
5.7 Adaptive time steps . . . 105
5.8 Summary . . . 107
5.9 Further reading . . . 107

References **108**

Preface

This script accompanies the lecture *Prognostic Modelling* (pea700). It develops numerical methods for the equations that govern the atmosphere, the land surface and the solid Earth, from ordinary differential equations to coupled nonlinear partial differential equations.

The script, the slides and the exercises are generated from one source. All code is written in Python and is shown next to the results it produces.

Syllabus

Module	pea700 Prognostic Modelling (Prognostische Modellierung)
Programme	MSc Physics of the Earth and Atmosphere
Credits	6 LP, 180 h
Format	Lecture (2 SWS) and exercises (2 SWS), 14 weeks
Assessment	Project
Prerequisite for assessment	Successful completion of the exercises (50 % of total points)
Lecturer	Stefan Kollet

Learning goals

Students

- know common numerical methods for geophysical and meteorological prognostic models and their properties,
- can judge the accuracy, stability, cost and limits of a method and interpret its results,
- can implement, verify and document numerical models in Python following good programming practice.

Outline

Unit	Part	Topic
1	Foundations	What is prognostic modelling?
2	Foundations	Governing equations and their classification
3	Foundations	The numerical toolbox I: derivatives, integrals, roots

Unit	Part	Topic
4	Foundations	The numerical toolbox II: matrices and waves
5	Time integration of ODEs	One-step methods
6	Time integration of ODEs	Stability, consistency, convergence, stiffness
7	Time integration of ODEs	Oscillations and multistep methods
8	Time integration of ODEs	Chaos, predictability, stochastic ODEs
9	Elliptic problems	Finite differences and finite volumes for elliptic problems
10	Elliptic problems	Direct solvers
11	Elliptic problems	Iterative solvers
12	Parabolic problems	Diffusion
13	Hyperbolic problems	Linear advection
14	Hyperbolic problems	Conservative and monotone advection
15	Hyperbolic problems	Waves and hyperbolic systems
16	Hyperbolic problems	Boundaries, domains and geometry
17	Nonlinear and coupled problems	Nonlinear PDEs
18	Nonlinear and coupled problems	Operator splitting, coupling and parametrization
19	Other discretizations	Galerkin and finite element methods
20	Other discretizations	Spectral methods
21	Outlook	Data assimilation and inverse problems
22	Outlook	High-performance computing and machine learning

Part I

Foundations

1 What is prognostic modelling?

Learning goals

After this unit you can

- explain what a prognostic model is and how prognostic and diagnostic equations differ,
- name the steps that lead from a physical law to a simulation result,
- list the sources of error in a numerical model and say which of them you control,
- explain round-off error and recognise loss of significance in a computation,
- distinguish verification from validation.

1.1 Models of the Earth system

In this lecture we deal with prognostic modelling of the physics of the Earth and the atmosphere. Before we write down any equation, we clarify what models are for.

Think first: what are models good for?

- **Prediction.** A model simulates the past and present state of a system and predicts its future state. Weather and climate models predict the future state of the atmosphere and the ocean. This is what we call prognostic modelling.
- **Filling gaps.** Measurements of temperature, rain or water levels are never continuous in space and time, not even from satellites. Models interpolate between locations and between measurement times. Nobody can measure the future, so looking ahead always needs a model that extrapolates.
- **Seeing the unobservable.** Models give access to variables for which no sensor exists, or which cannot be measured with the required accuracy or coverage, for example the water stored in the deep subsurface.
- **Virtual experiments.** Models allow experiments that are impossible, too expensive or too dangerous in reality: doubling atmospheric CO_2, removing a mountain range, moving an earthquake source.
- **Understanding.** Models help to understand complex systems and their feedbacks, because every process can be switched on and off.

Think first: what types of models exist, and what are their strengths?

Type	Strength	Weakness
Analog (laboratory experiment)	It is real	Needs a laboratory and often much money; valid only in the range of the experiment; scaling to the real system is difficult
Statistical, including machine learning	Works without knowing the governing laws; fast once trained	Needs much data; based on assumptions that are hard to check; unreliable outside the range of the training data
Mathematical (physics-based)	Based on conservation laws, therefore able to extrapolate	Based on simplifying assumptions; parameters are often unknown at the scale of the model; needs applied mathematics and computing

This lecture treats mathematical models. The three types are increasingly combined, for example when a machine-learning model is trained on the output of a physics-based model (Chapter 22).

1.1.1 A first mathematical model

A very familiar physical model is Newton's second law, $ma = F$. Written as a differential equation it reads

$$m\frac{\mathrm{d}^2u}{\mathrm{d}t^2} = F\left(t, u, \frac{\mathrm{d}u}{\mathrm{d}t}\right), \tag{1.1}$$

where $u(t)$ is the position of a particle at time t and F is the force acting on it. Many problems in science and engineering have this form: an equation that contains one or more derivatives of an unknown function. To model the motion of the particle we have to find the function $u(t)$ that satisfies Equation 1.1.

Throughout the lecture u denotes the unknown. Depending on the problem it is a position, a temperature, a pressure, a water depth or a vector of all of these.

1.1.2 Prognostic and diagnostic equations

A **prognostic equation** contains a time derivative of the unknown. It states how fast the state changes, and therefore allows us to step from the state now to the state a moment later:

$$\frac{\partial u}{\partial t} = \mathcal{F}(u, t). \tag{1.2}$$

A **diagnostic equation** contains no time derivative. It relates variables at the same instant. The ideal gas law $p = \rho RT$ and hydrostatic balance $\partial p/\partial z = -\rho g$ are diagnostic equations of atmospheric models.

A prognostic model consists of prognostic equations for the variables that carry the memory of the system, the **state**, together with diagnostic equations for everything that follows from the state. The mathematical task is an **initial value problem**: given the state at $t = 0$, find the state at later times.

Model	Prognostic variables (examples)	Equations
Numerical weather prediction, climate	wind, temperature, pressure or density, humidity	conservation of momentum, energy and mass in a rotating, stratified fluid
Ocean circulation	currents, temperature, salinity, sea surface height	the same, for sea water
Groundwater and soil hydrology	pressure head or water content	conservation of mass with Darcy's law
Seismic wave propagation	particle velocity, stress	conservation of momentum with a stress–strain relation

All these equations are derived in the same way, from the conservation of mass, momentum and energy (Chapter 2), and they are solved with the same small set of numerical methods. That is why meteorologists and geophysicists can share this lecture.

1.2 From physical law to computer code

Almost no prognostic model of a real system can be solved with pencil and paper. We have to solve it numerically, which means that a simulation result is separated from reality by a chain of steps:

1. **Physical system.** The atmosphere, an aquifer, the crust.

2. **Conceptual model.** Which processes matter, which are neglected, where the boundaries of the domain are.
3. **Mathematical model.** Differential equations with initial and boundary conditions and parameters.
4. **Discretization.** The continuous equations are replaced by algebraic equations for a finite number of unknowns.
5. **Solution algorithm.** The algebraic equations are solved, often only approximately.
6. **Implementation.** The algorithm is written as a program.
7. **Simulation and evaluation.** The program is run and its result is compared with observations.

Every step introduces errors. The lecture is mainly about steps 4 to 6, but you will only be able to judge a result if you keep the whole chain in mind.

1.2.1 The central idea: replace derivatives by differences

A computer can add, subtract, multiply and divide. It cannot differentiate. The simplest way to translate a differential equation into arithmetic is the **finite difference method**, which replaces a derivative by a difference quotient, for example

$$\frac{\mathrm{d}u}{\mathrm{d}t} \approx \frac{u(t+\Delta t) - u(t)}{\Delta t}, \tag{1.3}$$

where Δt is the time step. The derivative is the limit of the right hand side for $\Delta t \to 0$. On a computer Δt stays finite, and this is the origin of the discretization error.

As an example take the decay equation

$$\frac{\mathrm{d}u}{\mathrm{d}t} = -ku, \qquad u(0) = u_0, \tag{1.4}$$

which describes radioactive decay, the cooling of a body or the emptying of a linear reservoir. Its exact solution is $u(t) = u_0 e^{-kt}$. We insert Equation 1.3, write u_n for the approximation of $u(n\Delta t)$ and solve for the new value:

$$u_{n+1} = u_n - k\,\Delta t\, u_n. \tag{1.5}$$

This is a complete prognostic model. Starting from u_0 it produces the state at all later times using only multiplication and subtraction.

```python
import numpy as np
import matplotlib.pyplot as plt

k, u0, t_end = 1.0, 1.0, 4.0

fig, ax = plt.subplots(figsize=(6, 3.5))
t_exact = np.linspace(0, t_end, 200)
ax.plot(t_exact, u0 * np.exp(-k * t_exact), "k", label="exact")

for dt in [1.0, 0.5, 0.1]:
    n_steps = int(round(t_end / dt))
    u = np.empty(n_steps + 1)
    u[0] = u0
    for n in range(n_steps):
        u[n + 1] = u[n] - k * dt * u[n]
    ax.plot(dt * np.arange(n_steps + 1), u, "o-", ms=3, label=rf"$\Delta t$ = {dt}")
    error = abs(u[-1] - u0 * np.exp(-k * t_end))
    print(f"dt = {dt:4.1f}: error at t = {t_end:.0f} is {error:.4f}")

ax.set_xlabel("$t$")
ax.set_ylabel("$u$")
ax.legend()
plt.show()
```

```
dt =  1.0: error at t = 4 is 0.0183
dt =  0.5: error at t = 4 is 0.0144
dt =  0.1: error at t = 4 is 0.0035
```

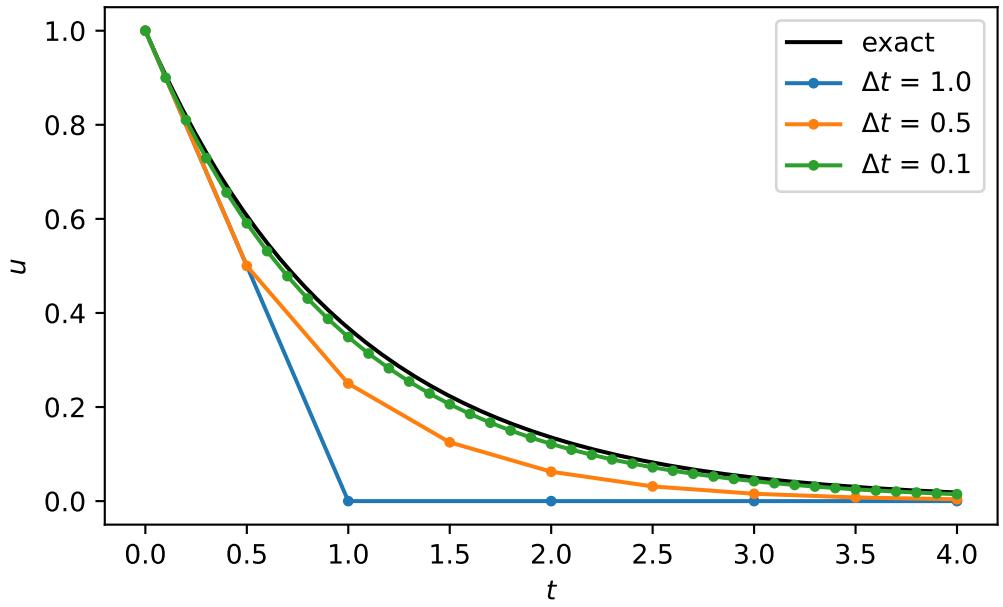


Figure 1.1: Numerical solution of the decay equation with the forward difference for three time steps, compared with the exact solution.

Figure 1.1 shows the behaviour we hope for: the smaller the time step, the closer the numerical solution is to the exact one. It also raises the questions that occupy us for the rest of the semester.

- How fast does the error decrease when Δt decreases? (Accuracy)
- With $\Delta t = 1$ the numerical solution drops to zero in one step. What happens for $\Delta t = 3$? (Stability)
- Is there a better formula than Equation 1.5 for the same cost? (Efficiency)

1.2.2 A first analysis

For this simple model the first two questions can be answered on paper. Equation 1.5 multiplies the solution by the same factor in every step, $u_{n+1} = (1 - k\Delta t)\, u_n$. Applying it n times,

$$u_1 = (1 - k\Delta t)\, u_0, \quad u_2 = (1 - k\Delta t)^2\, u_0, \quad \ldots, \quad u_n = (1 - k\Delta t)^n\, u_0. \tag{1.6}$$

The exact solution is multiplied in every step by $e^{-k\Delta t}$, since $u(t + \Delta t) = u_0 e^{-k(t+\Delta t)} = e^{-k\Delta t}\, u(t)$. The Taylor series of this factor is

$$e^{-k\Delta t} = 1 - k\Delta t + \frac{(k\Delta t)^2}{2} - \dots$$

The numerical factor $1 - k\Delta t$ consists of its first two terms.

Accuracy. In one step the numerical and the exact factor differ by $(k\Delta t)^2/2$. To reach a fixed time t we need $n = t/\Delta t$ steps, so the errors add up to something proportional to $n\,(k\Delta t)^2 \propto \Delta t$. The error at a fixed time is proportional to the time step: halving Δt halves the error.

Stability. The behaviour of $u_n = (1 - k\Delta t)^n u_0$ depends only on the size of the factor.

Time step	Factor $1 - k\Delta t$	Numerical solution
$k\Delta t < 1$	between 0 and 1	decays monotonically, as the exact solution
$k\Delta t = 1$	0	zero after one step
$1 < k\Delta t < 2$	between -1 and 0	decays, but changes sign in every step
$k\Delta t > 2$	below -1	oscillates and grows without bound

For $k\Delta t > 2$ the numerical solution has nothing in common with the exact one. The scheme is unstable. The time step of this scheme is therefore limited by the fastest process in the model, not by the accuracy we want. This observation, generalised to systems and to partial differential equations, is one of the main themes of the lecture (Chapter 6).

1.3 Sources of error and uncertainty

Think first: which errors occur when a numerical model is developed and applied?

Collect your own list before reading on, and sort it by the step of the modelling chain at which the error arises.

The following table sorts the errors by their origin (after Wick 2020).

Origin	Error	Description
Conceptual and mathematical model	Model error	Processes are missing or simplified, for example a linear equation replaces a nonlinear one. Found only by comparison with observations and by careful reasoning.
	Homogenization error	A model at full resolution is replaced by an averaged one; processes smaller than the grid are represented by parameters (parametrization, Chapter 18).
	Regularization error	An ill-posed problem is modified so that it becomes well-posed.
Data	Data error	Initial conditions, boundary conditions and parameters come from measurements and are uncertain and incomplete.
	Reference error	The observations used for evaluation are themselves inaccurate.
Discretization	Discretization error	Continuous functions are represented by a finite number of values.
	Interpolation error	Values between grid points are obtained from simpler functions.
Solution algorithm	Iteration error	Iterative algorithms are stopped after a finite number of steps.
Computer	Round-off error	A computer represents only a finite set of numbers.
Implementation	Bugs	Some make the program abort, which is the harmless case. Others produce results that look plausible and are wrong.

Two remarks help to keep this list in proportion.

First, the errors are of very different size, and the total error is dominated by the largest contribution. If the parameters of a groundwater model are known only within a factor of ten, it is pointless to reduce the discretization error to 0.01 %. Deciding which error dominates is a central task of numerical modelling.

Second, errors in the data are of two kinds. **Epistemic** uncertainty comes from lack of knowledge and can in principle be reduced by more measurements. **Aleatoric** uncertainty comes from variability that is random for all practical purposes. Both are treated by running the model many times (Chapter 8).

Of all these errors, the discretization, iteration and round-off errors and the bugs are the ones that numerical methods control. The rest of this unit treats round-off; the discretization error is the subject of most of the following units.

1.4 Floating point arithmetic and round-off

1.4.1 How a computer stores real numbers

A computer stores a real number as a **floating point number**

$$x = \pm\, m \cdot 2^{e},$$

with a significand m and an exponent e that each occupy a fixed number of bits. Python and NumPy use by default the *double precision* format of the IEEE 754 standard with 64 bits, of which 53 hold the significand. This has three consequences (Goldberg 1991).

1. **Finite precision.** A double carries about 16 significant decimal digits. The distance between 1 and the next larger floating point number is the **machine epsilon**, $\varepsilon = 2^{-52} \approx 2.2 \times 10^{-16}$.
2. **Finite range.** The largest double is about 1.8×10^{308}. Larger results overflow to `inf`.
3. **Most decimal fractions are not representable.** In base 2 the number 0.1 has infinitely many digits, as 1/3 has in base 10, and is rounded when it is stored.

```
import numpy as np

print("machine epsilon:        ", np.finfo(float).eps)
print("1 + eps/2 == 1:         ", 1.0 + np.finfo(float).eps / 2 == 1.0)
print("0.1 + 0.2 == 0.3:       ", 0.1 + 0.2 == 0.3)
print("0.1 + 0.2 =             ", repr(0.1 + 0.2))
print("largest double:         ", np.finfo(float).max)
print("single precision epsilon:", np.finfo(np.float32).eps)
```

```
machine epsilon:          2.220446049250313e-16
1 + eps/2 == 1:           True
0.1 + 0.2 == 0.3:         False
0.1 + 0.2 =               0.30000000000000004
largest double:           1.7976931348623157e+308
single precision epsilon: 1.1920929e-07
```

Every arithmetic operation therefore returns the exact result rounded to the nearest floating point number, with a relative error of at most $\varepsilon/2$. Two rules follow immediately.

- Never test floating point numbers for equality. Test whether their difference is small, for example with `np.isclose`.
- Single precision has only about 7 significant digits. Weather and climate models nevertheless increasingly use it, because it halves memory and data transfer. Whether this is safe has to be shown for each algorithm.

1.4.2 Loss of significance

A relative error of 10^{-16} sounds harmless, and for a single operation it is. It becomes dangerous when two nearly equal numbers are subtracted. The leading digits cancel, and the result consists mostly of the rounding errors of the operands. Consider

$$f(x) = \frac{1 - \cos x}{x^2} \;\to\; \frac{1}{2} \quad \text{for } x \to 0.$$

```python
def f_naive(x):
    return (1 - np.cos(x)) / x**2

def f_stable(x):
    return 2 * np.sin(x / 2) ** 2 / x**2

for x in [1e-2, 1e-4, 1e-6, 1e-8]:
    print(f"x = {x:.0e}:  naive {f_naive(x):.10f}   stable {f_stable(x):.10f}")
```

```
x = 1e-02:  naive 0.4999958333   stable 0.4999958333
x = 1e-04:  naive 0.4999999970   stable 0.4999999996
x = 1e-06:  naive 0.5000444503   stable 0.5000000000
x = 1e-08:  naive 0.0000000000   stable 0.5000000000
```

For $x = 10^{-8}$ the first formula returns 0 instead of 0.5: $\cos x$ differs from 1 by 5×10^{-17}, which is less than the machine epsilon, so it is stored as exactly 1. The second formula uses the identity $1 - \cos x = 2\sin^2(x/2)$, is mathematically identical and avoids the subtraction.

This has a direct consequence for our central idea. The difference quotient Equation 1.3 subtracts two numbers that become more and more equal as the step decreases. Making the step ever smaller therefore does not make a finite difference ever more accurate. We quantify this in Chapter 3.

1.4.3 Conditioning

Loss of significance is an example of a more general notion. A problem is **well-conditioned** if small relative changes of the input cause small relative changes of the result, and **ill-conditioned** if they cause large ones. The amplification factor is the **condition number** κ.

For the evaluation of a function $f(x)$ it follows from the Taylor series. A small change δx of the input changes the result by $\delta f \approx f'(x)\,\delta x$. We divide by f and expand with x to obtain relative changes on both sides:

$$\frac{\delta f}{f} \approx \frac{f'(x)\,\delta x}{f(x)} = \underbrace{\frac{x\,f'(x)}{f(x)}}_{\pm\kappa}\,\frac{\delta x}{x}, \qquad \kappa = \left|\frac{x\,f'(x)}{f(x)}\right|.$$

The problem. We apply this to the example above, $f(x) = (1-\cos x)/x^2$. For small x the Taylor series of the cosine gives

$$f(x) = \frac{1}{2} - \frac{x^2}{24} + \ldots, \qquad f'(x) \approx -\frac{x}{12},$$

and therefore

$$\kappa_f = \left|\frac{x\,f'(x)}{f(x)}\right| \approx \frac{x^2/12}{1/2} = \frac{x^2}{6}.$$

For $x = 10^{-8}$ this is about 10^{-17}. The problem is extremely well-conditioned: a small relative error in x has almost no effect on f. The failure of `f_naive` therefore cannot be blamed on the problem.

The algorithm. `f_naive` computes f in steps, and each step is a problem of its own with its own condition number. First it computes $c = \cos x$. The result is rounded, so c carries a relative error of up to ε. Then it computes $h(c) = 1 - c$. With the same formula, now for the input c,

$$\kappa_h = \left|\frac{c\,h'(c)}{h(c)}\right| = \frac{|c|}{|1-c|}.$$

For small x the Taylor series $\cos x = 1 - x^2/2 + x^4/24 - \dots$ gives $c \approx 1$ and $1 - c = x^2/2 - x^4/24 + \dots \approx x^2/2$, hence

$$\kappa_h \approx \frac{1}{x^2/2} = \frac{2}{x^2}.$$

For $x = 10^{-8}$ this is 2×10^{16}. The rounding error $\varepsilon \approx 10^{-16}$ of c is amplified to a relative error of order one: no correct digit remains. `f_stable` computes $\sin(x/2)$, squares it and divides by x^2. These steps have condition numbers of about 1 and 2, so the rounding errors stay at the level of ε.

```
eps = np.finfo(float).eps
print("     x   kappa problem   kappa step 1-cos x   predicted error   actual error")
for x in [1e-2, 1e-4, 1e-6, 1e-8]:
    kappa_f = x**2 / 6                                    # the problem f
    c = np.cos(x)
    kappa_h = abs(c) / abs(1 - c) if c != 1 else np.inf    # the step 1 - c in
↪   f_naive
    predicted = kappa_h * eps                             # bound for the relative
↪   error of f_naive
    actual = abs(f_naive(x) - f_stable(x)) / f_stable(x)   # f_stable serves as the
↪   exact value
    print(f"{x:6.0e}   {kappa_f:13.1e}   {kappa_h:18.1e}   {predicted:15.1e}
    ↪  {actual:12.1e}")
```

```
     x   kappa problem   kappa step 1-cos x   predicted error   actual error
 1e-02         1.7e-05              2.0e+04           4.4e-12        2.9e-13
 1e-04         1.7e-09              2.0e+08           4.4e-08        5.2e-09
 1e-06         1.7e-13              2.0e+12           4.4e-04        8.9e-05
 1e-08         1.7e-17                  inf               inf        1.0e+00
```

The last two columns compare the error predicted by the condition number of the step $1 - c$, $\kappa_h\,\varepsilon$, with the actual relative error of `f_naive`, for which the result of `f_stable` serves as the exact value. The prediction is an upper bound, since the rounding error of $\cos x$ is usually smaller than ε; it lies within a factor of about 10 of the actual error. Above all, it explains why the error grows by a factor of 10^4 each time x shrinks by a factor of 100. For $x = 10^{-8}$ the stored value of $\cos x$ is exactly 1: the step $1 - c$ has lost all information, which the infinite condition number expresses, and the relative error is 100 %.

Conditioning and stability. Conditioning is a property of the problem. It must be distinguished from the **stability** of an algorithm. An algorithm is unstable if it amplifies errors more than the problem itself does, typically because it contains an ill-conditioned intermediate step that the problem does not need. Here the problem is well-conditioned, `f_naive` is unstable

because of the step $1 - \cos x$, and `f_stable` is stable. For an ill-conditioned problem, in contrast, no algorithm helps; the problem itself has to be reformulated.

We meet the condition number again for linear systems of equations, where the same definition leads to the condition number of a matrix (Chapter 4, Chapter 11), and ill-conditioning in its most famous geophysical form, the limited predictability of weather (Chapter 8).

1.5 Concepts that return in every unit

A few concepts apply to all numerical methods of this lecture. We introduce them informally here and make them precise in Chapter 6.

Approximation The exact solution u is not available. We compute an approximation $\tilde{u}$ and need to know how far it is from u.

Consistency A method is consistent if the discrete equations approach the differential equation when the grid spacing and the time step go to zero.

Stability A method is stable if errors, for example round-off errors, do not grow without bound during the computation. Unstable methods typically show oscillations that grow in time. It is therefore important to know from the physics whether oscillations are to be expected.

Convergence A method converges if the numerical solution approaches the exact solution when the grid spacing and the time step go to zero. The rate at which it does so is the **order** of the method.

Efficiency The accuracy reached for a given amount of computing time, memory and energy. A method of higher order is often, but not always, more efficient.

Well-posedness A property of the mathematical problem: a solution exists, it is unique, and it depends continuously on the data (Chapter 2). A numerical method cannot repair a problem that is not well-posed.

1.6 Verification and validation

Two questions must be asked of every model (Oberkampf and Roy 2010).

Verification: are we solving the equations right? This is a mathematical question. It is answered by comparing the numerical solution with exact solutions and by checking that the error decreases at the expected rate when the grid is refined. Verification concerns discretization, algorithm and code, and it can be completed.

Validation: are we solving the right equations? This is a physical question. It is answered by comparing the simulation with observations. Validation concerns the conceptual and mathematical model and the data, and it is never complete, because a model can only be shown to agree with the observations available so far.

The order matters. A model that has not been verified cannot be validated, because agreement with observations may result from errors that compensate each other. In this lecture you verify every program you write.

1.7 Working practice

Numerical modelling is to a large part work on paper. Classifying the problem, deriving the discrete equations, and writing down the grid, the indices and the boundary conditions come first. Programming, debugging and plotting are the smaller part, and they go quickly when the paper work is complete.

The following habits save most of the time that is usually lost.

- **Simplify.** Start with the lowest resolution, the simplest boundary and initial conditions and constant coefficients. Add one complication at a time.
- **Know the answer in advance.** Test first on a problem with a known solution. For a linear system with five unknowns you can check every number by hand.
- **Write functions and test them.** A function that does one thing can be tested once and then trusted: on a problem with a known answer, and again after every change. In the exercises you collect your functions in your own module `toolbox.py` and reuse them in later sheets and in the project.
- **Use version control.** With `git` every working state of a program can be recovered, and every result can be attributed to the version of the code that produced it.
- **Make results reproducible.** A figure should be produced by a script from the raw output, without manual steps. All figures in this script are produced by the code shown next to them.

1.8 Summary

- A prognostic model predicts the future state of a system from its present state with equations that contain time derivatives. Diagnostic equations relate variables at the same time.
- A simulation result is separated from reality by a chain of steps: conceptual model, mathematical model, discretization, algorithm, implementation. Each contributes errors, and the largest contribution dominates.
- The basic idea of discretization is to replace derivatives by difference quotients. The resulting discretization error decreases with the step.
- Floating point numbers have about 16 significant digits. Subtracting nearly equal numbers destroys most of them.
- Verification checks the solution of the equations, validation checks the equations. Verification comes first.

1.9 Further reading

Durran (2010), chapter 1, for the view from atmospheric science; Goldberg (1991) for floating point arithmetic; Oberkampf and Roy (2010) for verification and validation.

2 Governing equations and their classification

Learning goals

After this unit you can

- compute and interpret gradient, divergence, curl and Laplacian,
- derive a conservation law from a balance over a control volume,
- classify a differential equation by order, linearity and type,
- recognise the prototype equations for diffusion, advection and waves in the equations of atmospheric, hydrological and seismic models,
- choose initial and boundary conditions that make a problem well-posed.

Numerical modelling starts on paper with the classification of the problem. The type of an equation decides which numerical methods work and which fail. This unit provides the language: the operators of vector calculus, the conservation laws built from them, the classification of the resulting equations, and the equations of this lecture with their types.

2.1 Vector calculus recap

2.1.1 Derivatives

If the unknown depends on a single variable, for example $u(t)$, its rate of change is the ordinary derivative $\mathrm{d}u/\mathrm{d}t$. If it depends on several variables, for example $u(x,t)$, the partial derivatives $\partial u/\partial t$ and $\partial u/\partial x$ give the rate of change with respect to one variable while the others are held fixed. Derivatives of order n are written $\mathrm{d}^n u/\mathrm{d}t^n$ and $\partial^n u/\partial x^n$. We also use the short forms u_t, u_x, u_{xx}.

2.1.2 Vectors

A vector in the plane is written $\vec{v} = v_1\vec{i} + v_2\vec{j}$, in space $\vec{v} = v_1\vec{i} + v_2\vec{j} + v_3\vec{k}$, where $\vec{i}, \vec{j}, \vec{k}$ are the unit vectors along the coordinate axes. Its magnitude is

$$\|\vec{v}\| = \sqrt{v_1^2 + v_2^2 + v_3^2},$$

and the unit vector in the direction of $\vec{v}$ is $\vec{v}/\|\vec{v}\|$.

2.1.3 The gradient

The gradient of a scalar field $f(x, y, z)$ is the vector

$$\operatorname{grad} f = \nabla f = \frac{\partial f}{\partial x}\vec{i} + \frac{\partial f}{\partial y}\vec{j} + \frac{\partial f}{\partial z}\vec{k}.$$

It has two geometric properties, shown in Figure 2.1.

- ∇f points in the direction in which f increases fastest, and is perpendicular to the contour of f through the point.
- $\|\nabla f\|$ is the rate of change in that direction. It is large where the contours are close together.

In physics, gradients drive fluxes: heat flows down the temperature gradient, groundwater flows down the gradient of hydraulic head, and the pressure gradient accelerates the wind.

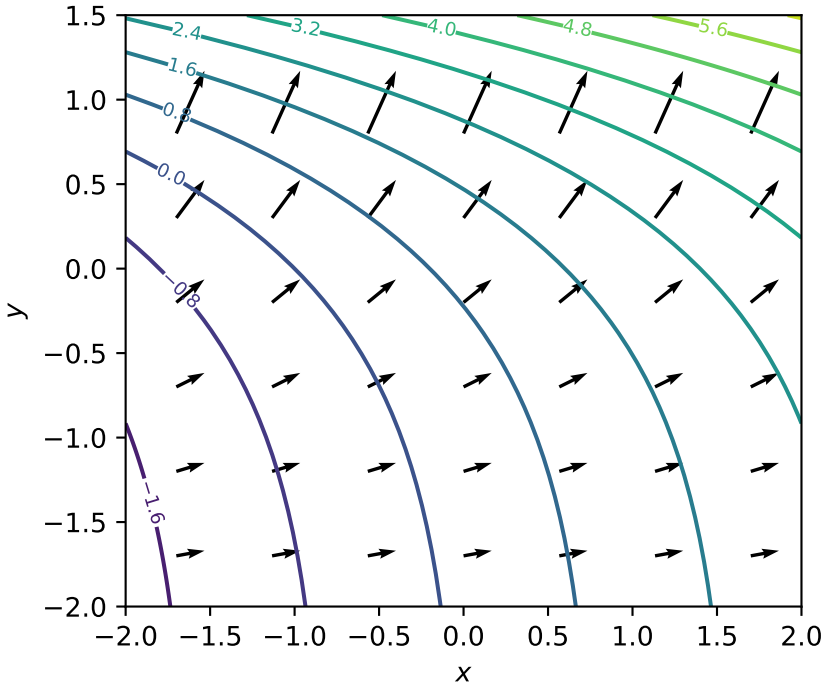


Figure 2.1: Contours of $f(x, y) = x + e^y$ and its gradient vectors. The vectors are perpendicular to the contours and longer where the contours are closer.

Exercise: gradient of $f(x, y) = x + e^y$ at the point $(1, 1)$

$\nabla f = \vec{i} + e^y \vec{j}$, so $\nabla f(1, 1) = \vec{i} + e\,\vec{j}$. The maximum rate of change at this point is $\|\nabla f\| = \sqrt{1 + e^2} \approx 2.90$.

2.1.4 The divergence

The divergence of a vector field $\vec{F}$ measures the outflow per unit volume at a point. Geometrically, it is the flux out of a small closed surface S around the point, divided by the enclosed volume V, in the limit of vanishing volume:

$$\operatorname{div} \vec{F} = \nabla \cdot \vec{F} = \lim_{V \to 0} \frac{1}{V} \oint_S \vec{F} \cdot \mathrm{d}\vec{A}. \tag{2.1}$$

Here $\mathrm{d}\vec{A}$ is the surface element with the direction of the outward normal. In Cartesian coordinates the divergence is

$$\nabla \cdot \vec{F} = \frac{\partial F_1}{\partial x} + \frac{\partial F_2}{\partial y} + \frac{\partial F_3}{\partial z}. \tag{2.2}$$

Why do both definitions agree? Consider a small box with one corner at (x_0, y_0, z_0) and edge lengths $\Delta x, \Delta y, \Delta z$ (Figure 2.2). On the face at $x = x_0$ the outward normal is $-\vec{i}$. If $\vec{F}$ is approximately constant over this small face, the flux through it is

$$\int_{S_1} \vec{F} \cdot \mathrm{d}\vec{A} \approx -F_1(x_0, y_0, z_0)\, \Delta y\, \Delta z.$$

On the opposite face at $x = x_0 + \Delta x$ the outward normal is $+\vec{i}$, and the flux is $F_1(x_0 + \Delta x, y_0, z_0)\, \Delta y\, \Delta z$. The sum of the two is

$$\frac{F_1(x_0 + \Delta x, y_0, z_0) - F_1(x_0, y_0, z_0)}{\Delta x}\, \Delta x\, \Delta y\, \Delta z \approx \frac{\partial F_1}{\partial x}\, \Delta x\, \Delta y\, \Delta z.$$

The other two pairs of faces contribute in the same way. Dividing the total flux by the volume $\Delta x\, \Delta y\, \Delta z$ gives Equation 2.2.

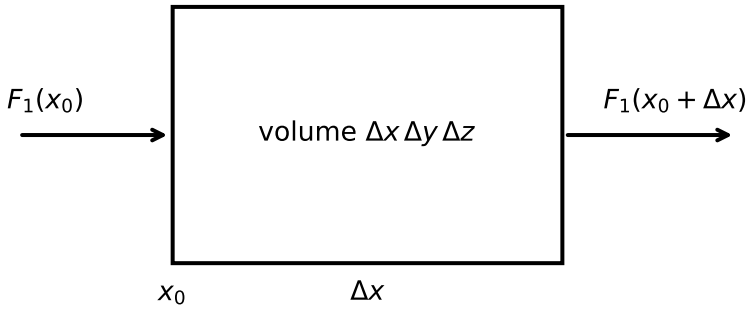


Figure 2.2: Fluxes in the x direction through a control volume. The divergence is the net outflow per unit volume.

This derivation is more than a proof. Summing fluxes over the faces of a small box is exactly what the finite volume method does (Chapter 9).

Two results that we use constantly:

Product rule. For a scalar field g and a vector field $\vec{F}$,

$$\nabla \cdot (g\vec{F}) = g\,\nabla \cdot \vec{F} + \vec{F} \cdot \nabla g. \tag{2.3}$$

Divergence theorem (Gauss). For a volume V with closed surface S,

$$\int_V \nabla \cdot \vec{F}\,\mathrm{d}V = \oint_S \vec{F} \cdot \mathrm{d}\vec{A}. \tag{2.4}$$

What flows out through the surface equals the sum of all sources inside.

2.1.5 Curl and Laplacian

The **curl** $\nabla \times \vec{F}$ measures the local rotation of a vector field. For a horizontal wind field $\vec{v} = u\vec{i} + v\vec{j}$ its vertical component is the relative vorticity

$$\zeta = \frac{\partial v}{\partial x} - \frac{\partial u}{\partial y},$$

a central quantity of dynamic meteorology. A small paddle wheel placed in the flow and carried along with it spins with the angular velocity $\zeta/2$: anticlockwise for $\zeta > 0$, clockwise for $\zeta < 0$. Figure 2.3 shows two examples. In solid-body rotation, $u = -y$, $v = x$, we get $\zeta = 1 - (-1) = 2$.

In the shear flow $u = y$, $v = 0$, we get $\zeta = 0 - 1 = -1$. The streamlines of the shear flow are straight lines, yet the flow has vorticity: the faster flow on the upper side of the wheel pushes it clockwise. Curl measures local rotation, not curved streamlines.

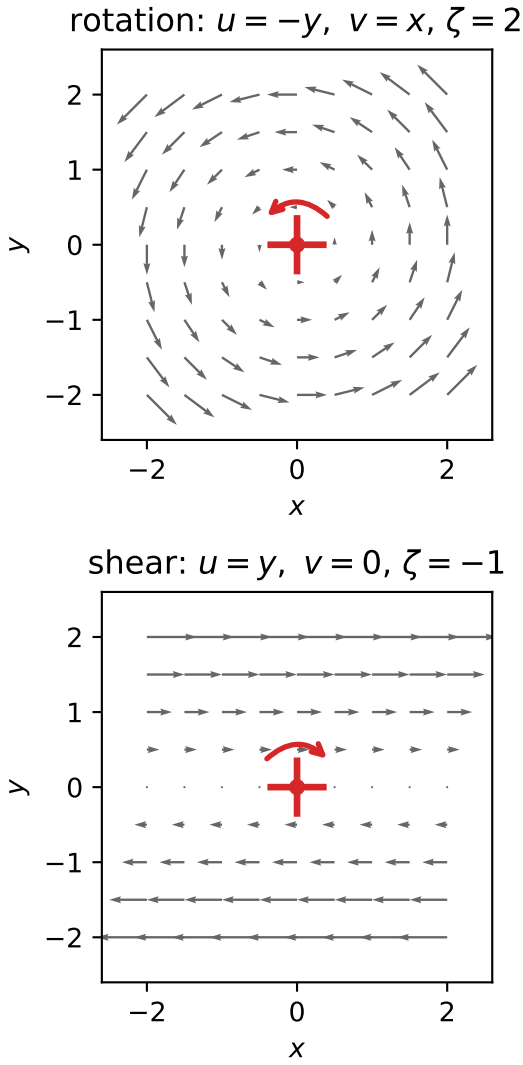


Figure 2.3: Two velocity fields with vorticity, each with a paddle wheel (red) and its sense of rotation. Top: solid-body rotation, $\zeta = 2$, anticlockwise. Bottom: shear flow with straight streamlines, $\zeta = -1$, clockwise.

The **Laplacian** is the divergence of the gradient:

$$\nabla^2 f = \nabla \cdot (\nabla f) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} + \frac{\partial^2 f}{\partial z^2}.$$

It compares the value of f at a point with the average over its neighbourhood. To see this, start in one dimension and take the two neighbours of x at distance h. Their Taylor expansions are

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2} f''(x) + \frac{h^3}{6} f'''(x) + O(h^4),$$
$$f(x-h) = f(x) - hf'(x) + \frac{h^2}{2} f''(x) - \frac{h^3}{6} f'''(x) + O(h^4).$$

Adding the two lines, the odd-order terms cancel:

$$f(x+h) + f(x-h) = 2f(x) + h^2 f''(x) + O(h^4).$$

Dividing by 2 and subtracting $f(x)$ gives the difference between the average of the neighbours and the value at the point:

$$\underbrace{\frac{f(x+h) + f(x-h)}{2}}_{\text{neighbour average}} - f(x) = \frac{h^2}{2} f''(x) + O(h^4).$$

In two dimensions we do the same in the x and the y direction, with the four neighbours $(x \pm h, y)$ and $(x, y \pm h)$. Each pair gives its own second derivative, exactly as in one dimension:

$$f(x+h, y) + f(x-h, y) = 2f(x, y) + h^2 f_{xx} + O(h^4),$$
$$f(x, y+h) + f(x, y-h) = 2f(x, y) + h^2 f_{yy} + O(h^4).$$

We call the average over the four neighbours $\bar{f}$. Adding the two lines and dividing by 4 gives

$$\bar{f} = f(x, y) + \frac{h^2}{4} (f_{xx} + f_{yy}) + O(h^4),$$

and hence

$$\bar{f} - f(x, y) = \frac{h^2}{4} \nabla^2 f + O(h^4).$$

In three dimensions, with six neighbours, the factor is $h^2/6$. So, up to a positive factor, the Laplacian is the amount by which the neighbourhood average exceeds the value at the point:

- $\nabla^2 f < 0$: f is larger than its surroundings, as at a local maximum (a peak);
- $\nabla^2 f > 0$: f is smaller than its surroundings, as at a local minimum (a trough);
- $\nabla^2 f = 0$: f equals the average of its surroundings. Such functions are called harmonic; they have no interior maxima or minima.

This is why the Laplacian appears in every equation that describes smoothing. In the diffusion equation $\partial u/\partial t = D\nabla^2 u$ with $D > 0$, u decreases where it lies above the average of its neighbourhood and increases where it lies below. Peaks are eroded, troughs are filled, and the field approaches its local average. Rearranged, the formula above is also the finite difference approximation

$$\nabla^2 f \approx \frac{f(x+h,y) + f(x-h,y) + f(x,y+h) + f(x,y-h) - 4f(x,y)}{h^2}$$

that we will use for elliptic and parabolic problems.

2.2 Conservation laws

Nearly all equations of this lecture state that a quantity is conserved: mass, momentum or energy. They are all derived with the same argument.

Let u be the density of the conserved quantity (amount per volume) and $\vec{F}$ its flux (amount per area and time). The amount inside a fixed volume V changes only by flux through the surface and by sources s (amount per volume and time) inside:

$$\frac{\mathrm{d}}{\mathrm{d}t}\int_V u \,\mathrm{d}V = -\oint_S \vec{F}\cdot \mathrm{d}\vec{A} + \int_V s \,\mathrm{d}V. \tag{2.5}$$

The minus sign appears because $\mathrm{d}\vec{A}$ points outward. We turn this balance into a differential equation in three steps.

1. The volume is fixed in space, so the time derivative can be taken inside the integral: $\frac{\mathrm{d}}{\mathrm{d}t}\int_V u \,\mathrm{d}V = \int_V \frac{\partial u}{\partial t}\,\mathrm{d}V$.
2. The divergence theorem Equation 2.4 converts the surface integral into a volume integral: $\oint_S \vec{F}\cdot\mathrm{d}\vec{A} = \int_V \nabla\cdot\vec{F}\,\mathrm{d}V$.
3. All terms are now integrals over the same volume and can be collected:

$$\int_V \left(\frac{\partial u}{\partial t} + \nabla\cdot\vec{F} - s\right)\mathrm{d}V = 0.$$

This holds for every volume V, however small and wherever it is placed. That is only possible if the integrand itself vanishes everywhere, which gives the differential form

$$\frac{\partial u}{\partial t} + \nabla \cdot \vec{F} = s. \tag{2.6}$$

Equation 2.6 is a prognostic equation for u, but it is not yet closed, because it contains the unknown flux. A **flux law** (constitutive relation) expresses $\vec{F}$ in terms of u. Two flux laws cover most of this lecture.

Advective flux. The quantity is carried along by a velocity field $\vec{v}$:

$$\vec{F} = \vec{v}\, u.$$

Diffusive flux. The quantity flows down its own gradient:

$$\vec{F} = -D\, \nabla u.$$

The second law has many names, depending on the quantity.

Law	Flux	Driven by	Coefficient
Fourier	heat	temperature gradient	thermal conductivity
Fick	dissolved substance	concentration gradient	diffusion coefficient
Darcy	groundwater	gradient of hydraulic head	hydraulic conductivity

We insert the total flux $\vec{F} = \vec{v}\, u - D\, \nabla u$ into Equation 2.6 and keep track of every sign. The divergence is linear, so it acts on the two parts separately:

$$\nabla \cdot \vec{F} = \nabla \cdot (\vec{v}\, u) + \nabla \cdot (-D\, \nabla u) = \nabla \cdot (\vec{v}\, u) - \nabla \cdot (D\, \nabla u).$$

Inserted into Equation 2.6:

$$\frac{\partial u}{\partial t} + \nabla \cdot (\vec{v}\, u) - \nabla \cdot (D\, \nabla u) = s.$$

Moving the diffusion term to the right-hand side changes its sign once more, which gives the **advection–diffusion equation**

$$\frac{\partial u}{\partial t} + \nabla \cdot (\vec{v}\, u) = \nabla \cdot (D\, \nabla u) + s. \tag{2.7}$$

The diffusion term now carries a plus sign. Two minus signs have cancelled: the one from the outward normal in Equation 2.5 (flux out of the volume reduces u) and the one from the flux law (the flux points down the gradient).

Keep track of the signs

Lost or doubled minus signs are among the most frequent and most time-consuming errors in numerical modelling, in course projects as well as in research codes. A code with a sign error usually runs; it just gives wrong results, and the error is often found only after days of debugging. The signs depend on conventions that differ between textbooks, disciplines and codes:

- **Direction of the normal**: outward (as here) or inward.
- **Flux law**: $\vec{F} = -D\nabla u$, or a flux defined positive in the direction of decreasing u.
- **Vertical coordinate**: z pointing up (height) or down (depth, common in hydrology and seismology). Gravity terms change sign with it.
- **Operator**: many solvers are written for $-\nabla^2 u = f$, because the matrix of $-\nabla^2$ is positive definite (Chapter 4); others for $\nabla^2 u = f$.
- **Source term**: $s > 0$ as a source or as a sink (e.g. extraction from a well, or evaporation).

Three habits prevent most of these errors:

1. **Write the convention down** at the top of the derivation and in the code (which direction is positive, what the flux means).
2. **Check the physics of the result.** In $\partial u/\partial t = D\nabla^2 u$, at a maximum $\nabla^2 u < 0$ (see the Laplacian above), so u must decrease there. With the wrong sign the maximum grows; this is anti-diffusion, and it is unstable.
3. **Test the code on a case with a known answer**: a Gaussian must spread and flatten, the total amount $\int u \, \mathrm{d}V$ must stay constant without sources, and water must flow from high to low hydraulic head.

Flux form and advective form

With the product rule Equation 2.3 the advection term becomes $\nabla \cdot (\vec{v}u) = \vec{v} \cdot \nabla u + u \, \nabla \cdot \vec{v}$. For a divergence-free (incompressible) flow the last term vanishes and the equation can be written in **advective form**, $\partial u/\partial t + \vec{v} \cdot \nabla u = \ldots$

The two forms are mathematically equivalent. Their discretizations are not. A discretization of the **flux form** conserves the total amount of u exactly, because what leaves one grid cell enters its neighbour. A discretization of the advective form in general does not (Chapter 14).

2.3 Classification of differential equations

2.3.1 Ordinary and partial differential equations

If the unknown depends on one independent variable, the equation contains only ordinary derivatives and is an **ordinary differential equation** (ODE). An example is the equation for the charge $Q(t)$ on a capacitor in a circuit with inductance L, resistance R and capacitance C:

$$L\frac{\mathrm{d}^2Q}{\mathrm{d}t^2} + R\frac{\mathrm{d}Q}{\mathrm{d}t} + \frac{1}{C}Q = E(t).$$

If the unknown depends on several independent variables, the equation contains partial derivatives and is a **partial differential equation** (PDE). The advection–diffusion equation Equation 2.7 is a PDE.

2.3.2 Systems

Several unknowns coupled by several equations form a system. The predator–prey equations

$$\frac{\mathrm{d}x}{\mathrm{d}t} = ax - \alpha xy, \qquad \frac{\mathrm{d}y}{\mathrm{d}t} = -cy + \gamma xy$$

for the number of prey x and predators y are a system of two ODEs. The models of the atmosphere and the ocean are systems of PDEs, one for each prognostic variable.

2.3.3 Order

The order of a differential equation is the order of the highest derivative in it. It determines how many initial and boundary conditions are needed.

Every ODE of order n can be rewritten as a system of n first-order ODEs. For Newton's law Equation 1.1 we introduce the velocity $v = \mathrm{d}u/\mathrm{d}t$ as a second unknown:

$$\frac{\mathrm{d}u}{\mathrm{d}t} = v, \qquad \frac{\mathrm{d}v}{\mathrm{d}t} = \frac{F}{m}.$$

This is important in practice: numerical methods and software for ODEs are written for first-order systems. PDEs of second order in time can be treated in the same way; for seismic waves we will see that the wave equation is equivalent to a system of two first-order equations.

2.3.4 Linear and nonlinear

An equation is **linear** if the unknown and its derivatives appear only to the first power and are not multiplied with each other. The coefficients may depend on the independent variables. The general linear ODE of order n is

$$a_0(t)\,y^{(n)} + a_1(t)\,y^{(n-1)} + \cdots + a_n(t)\,y = g(t).$$

It is **homogeneous** if $g(t) = 0$.

Equation	Order	Linear?
$y' - 2xy = e^x$	1	yes
$y'' + 4y' + 4y = 0$	2	yes, homogeneous
$y' = x/y$	1	no
$y'' + \sin y = x^2$	2	no
$y''' + 2e^t y'' + yy' = t^4$	3	no
$u_t + c(x)\,u_x = 0$	1	yes
$u_t + u\,u_x = 0$	1	no

The last two rows deserve attention. Advection by a *given* velocity field $c(x)$ is linear, even though the coefficient varies. Advection of the velocity by itself, as in the momentum equations of atmosphere and ocean, is nonlinear.

The pendulum equation

$$\frac{\mathrm{d}^2\theta}{\mathrm{d}t^2} + \frac{g}{L}\sin\theta = 0$$

is nonlinear. For small angles $\sin\theta \approx \theta$, and it becomes the linear equation of the harmonic oscillator. This **linearization** is a standard tool: the stability of numerical methods is analysed for linear equations, and the results are then applied to the nonlinear problem with care.

Linearity has a great practical advantage. Solutions of a linear homogeneous equation can be added (superposition), so that the solution can be analysed wave by wave (Chapter 4). Discretizing a linear PDE gives a linear system of algebraic equations; discretizing a nonlinear PDE gives a nonlinear system that has to be solved iteratively (Chapter 17).

2.3.5 Elliptic, parabolic, hyperbolic

The most important classification of PDEs concerns their type. Linear second-order PDEs in two independent variables have the general form

$$a\,u_{xx} + 2b\,u_{xy} + c\,u_{yy} + \text{lower-order terms} = f, \tag{2.8}$$

where the second variable y may be a space coordinate or time. The type is determined by the coefficients of the second derivatives alone:

Type	Condition	Prototype	Eigenvalues of A
Elliptic	$b^2 - ac < 0$	Laplace, Poisson: $u_{xx} + u_{yy} = f$	same sign
Parabolic	$b^2 - ac = 0$	Diffusion: $u_t = Du_{xx}$	one is zero
Hyperbolic	$b^2 - ac > 0$	Wave: $u_{tt} = c^2 u_{xx}$	opposite signs

The names come from the quadratic form $q(x, y) = ax^2 + 2bxy + cy^2$, whose contour $q = 1$ is an ellipse or a hyperbola under the first and third condition. In matrix notation,

$$q(x,y) = \begin{pmatrix} x & y \end{pmatrix} A \begin{pmatrix} x \\ y \end{pmatrix}, \qquad A = \begin{pmatrix} a & b \\ b & c \end{pmatrix},$$

and the condition can be read off the signs of the eigenvalues of A. To see why, we compute the eigenvalues. They are the roots of the characteristic polynomial

$$\det(A - \lambda I) = \begin{vmatrix} a-\lambda & b \\ b & c-\lambda \end{vmatrix} = (a-\lambda)(c-\lambda) - b^2 = \lambda^2 - (a+c)\,\lambda + (ac - b^2).$$

A quadratic polynomial with roots λ_1 and λ_2 can be written as $(\lambda - \lambda_1)(\lambda - \lambda_2) = \lambda^2 - (\lambda_1 + \lambda_2)\lambda + \lambda_1\lambda_2$. Comparing the coefficients gives

$$\lambda_1 + \lambda_2 = a + c, \qquad \lambda_1\lambda_2 = ac - b^2 = \det A.$$

The eigenvalues are real, because A is symmetric. The discriminant of the characteristic polynomial confirms this: $(a+c)^2 - 4(ac - b^2) = (a-c)^2 + 4b^2 \geq 0$. The sign of the product $\lambda_1\lambda_2$ therefore decides the type, and it is the opposite of the sign of $b^2 - ac$:

- $b^2 - ac < 0$: $\lambda_1\lambda_2 > 0$, both eigenvalues have the same sign (elliptic);
- $b^2 - ac = 0$: $\lambda_1\lambda_2 = 0$, one eigenvalue is zero (parabolic);
- $b^2 - ac > 0$: $\lambda_1\lambda_2 < 0$, the eigenvalues have opposite signs (hyperbolic).

The geometric meaning follows from the eigenvectors. Because A is symmetric, its eigenvectors are orthogonal, and in coordinates (ξ, η) along the eigenvectors the mixed term disappears:

$$q = \lambda_1 \xi^2 + \lambda_2 \eta^2.$$

For $\lambda_1, \lambda_2 > 0$ the contour $q = 1$ is an ellipse with semi-axes $1/\sqrt{\lambda_1}$ and $1/\sqrt{\lambda_2}$. For eigenvalues of opposite sign it is a hyperbola. If $\lambda_2 = 0$, $q = \lambda_1 \xi^2$ does not depend on η at all, and $q = 1$ degenerates into two parallel lines, the limiting case between ellipse and hyperbola. The same rotation of coordinates applied to the PDE removes the mixed derivative and gives its **canonical form** $\lambda_1 u_{\xi\xi} + \lambda_2 u_{\eta\eta} + \text{lower-order terms} = f$. Elliptic equations then look like the Poisson equation, hyperbolic ones like the wave equation, and parabolic ones contain only one second derivative, like the diffusion equation.

As an example, take $u_{xx} + 4u_{xy} + u_{yy} = 0$, that is $a = c = 1$ and $b = 2$. The condition gives $b^2 - ac = 4 - 1 = 3 > 0$: hyperbolic. The eigenvalues confirm this: $\lambda_1 + \lambda_2 = 2$ and $\lambda_1 \lambda_2 = 1 - 4 = -3$, so $\lambda_1 = 3$ and $\lambda_2 = -1$, of opposite sign. The mixed derivative hides the type: both pure second derivatives have the same sign, yet the equation is a wave equation in rotated coordinates.

The eigenvalue formulation carries over to more than two variables, where the condition $b^2 - ac$ no longer exists (Wick 2020). With n independent variables, A is a symmetric $n \times n$ matrix. The equation is elliptic if all eigenvalues have the same sign, parabolic if one eigenvalue is zero and the others have the same sign, and hyperbolic if one eigenvalue has the opposite sign of the others. The heat equation $u_t = D(u_{xx} + u_{yy})$ in the variables (x, y, t), for example, has the eigenvalues D, D and 0 (up to an overall sign): parabolic. We meet quadratic forms again when we solve linear systems by minimisation (Chapter 11).

Check: classify the diffusion equation with this rule

Write $u_t - Du_{xx} = 0$ with the variables (x, t) in the form Equation 2.8. The only second derivative is u_{xx} with $a = -D$, and $b = c = 0$. Hence $b^2 - ac = 0$: parabolic. For the wave equation $u_{tt} - c_0^2 u_{xx} = 0$ we have $a = -c_0^2$, $b = 0$, $c = 1$ and $b^2 - ac = c_0^2 > 0$: hyperbolic.

What the three types mean physically is shown by their prototypes in the next section.

2.4 Prototype equations

Three special cases of conservation laws in one space dimension serve as prototypes, one for each type of the classification. Each has its own physical behaviour, and each requires its own numerical treatment.

Diffusion equation (heat equation), from Equation 2.7 with $\vec{v} = 0$ and constant D; parabolic:

$$\frac{\partial u}{\partial t} = D\frac{\partial^2 u}{\partial x^2}. \tag{2.9}$$

Advection equation, with constant velocity c and $D = 0$; of first order and hyperbolic:

$$\frac{\partial u}{\partial t} + c\frac{\partial u}{\partial x} = 0. \tag{2.10}$$

Wave equation, with wave speed c; hyperbolic:

$$\frac{\partial^2 u}{\partial t^2} = c^2\frac{\partial^2 u}{\partial x^2}. \tag{2.11}$$

On an unbounded domain all three have exact solutions for the initial condition $u(x, 0) = f(x)$.

Advection. The solution is $u(x, t) = f(x - ct)$: the initial shape moves with velocity c and does not change. Information travels along the lines $x - ct = \text{const}$, the **characteristics** of the equation, with the finite speed c; first-order equations with this property are called hyperbolic, like the wave equation. To verify the solution, set $\xi = x - ct$ and apply the chain rule:

$$\frac{\partial u}{\partial t} = f'(\xi)\frac{\partial \xi}{\partial t} = -c\, f'(\xi), \qquad \frac{\partial u}{\partial x} = f'(\xi)\frac{\partial \xi}{\partial x} = f'(\xi) \quad \Longrightarrow \quad \frac{\partial u}{\partial t} + c\frac{\partial u}{\partial x} = 0.$$

This verifies the solution but does not tell us where it comes from. The box derives it.

Derivation: the method of characteristics

We look at u along a curve $x(t)$ in the (x, t) plane, that is at the function $U(t) = u(x(t), t)$. Its rate of change follows from the chain rule for a function of two variables, both of which depend on t:

$$\frac{\mathrm{d}U}{\mathrm{d}t} = \frac{\partial u}{\partial x}\frac{\mathrm{d}x}{\mathrm{d}t} + \frac{\partial u}{\partial t}\frac{\mathrm{d}t}{\mathrm{d}t} = \frac{\partial u}{\partial t} + \frac{\mathrm{d}x}{\mathrm{d}t}\frac{\partial u}{\partial x}.$$

The right-hand side looks like the left-hand side of Equation 2.10. We choose the curve such that the two agree, that is with

$$\frac{\mathrm{d}x}{\mathrm{d}t} = c.$$

Integrating from $t = 0$, where the curve starts at the point x_0, gives the straight line $x(t) = x_0 + ct$. Along this line the PDE becomes an ODE:

$$\frac{\mathrm{d}U}{\mathrm{d}t} = \frac{\partial u}{\partial t} + c\,\frac{\partial u}{\partial x} = 0.$$

So U does not change, and u keeps the value it had at the starting point:

$$u(x_0 + ct, t) = U(t) = U(0) = u(x_0, 0) = f(x_0).$$

Finally we express x_0 by the point (x, t) we are interested in. The characteristic through (x, t) started at $x_0 = x - ct$, hence

$$u(x, t) = f(x - ct).$$

The derivation shows more than the verification:

- **Uniqueness.** Every point (x, t) lies on exactly one characteristic, and the value there is fixed by the initial condition. There is no other solution.
- **Domain of dependence.** $u(x, t)$ depends only on the initial value at the single point $x - ct$, not on f anywhere else. Information travels along the characteristics with speed c.
- **Generalisation.** The same steps work for a variable velocity $c(x, t)$: the characteristics are the solutions of the ODE $\mathrm{d}x/\mathrm{d}t = c(x, t)$, are no longer straight, and u is still constant along them. We return to this view of advection in the semi-Lagrangian methods (Chapter 14).

Wave equation. Differentiating once more gives $\partial^2 u/\partial t^2 = c^2 f''(\xi)$ and $\partial^2 u/\partial x^2 = f''(\xi)$, so $f(x - ct)$ also solves the wave equation. The sign of c drops out in the second derivative, so $f(x + ct)$ is a solution as well. For zero initial velocity the solution is $u(x, t) = \frac{1}{2}[f(x - ct) + f(x + ct)]$: the initial shape splits into two halves that travel in opposite directions. At $t = 0$ the two halves add up to $f(x)$, and their time derivatives, $\mp\frac{1}{2}cf'(x)$, cancel.

Diffusion. For the Gaussian initial condition $f(x) = e^{-x^2/w^2}$ the solution remains a Gaussian that widens and flattens,

$$u(x, t) = \frac{w}{\sqrt{w^2 + 4Dt}} \exp\left(-\frac{x^2}{w^2 + 4Dt}\right).$$

The squared width grows linearly in time, $w^2 + 4Dt$, and the amplitude decreases such that the integral of u, the total amount, stays constant.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-10, 10, 801)
w = 1.0                          # width of the initial Gaussian
f = lambda x: np.exp(-(x / w) ** 2)
D, c = 1.0, 1.0

def diffusion(x, t):
    return w / np.sqrt(w**2 + 4 * D * t) * np.exp(-x**2 / (w**2 + 4 * D * t))
```

```
solutions = {
    "Diffusion": diffusion,
    "Advection": lambda x, t: f(x - c * t),
    "Wave": lambda x, t: 0.5 * (f(x - c * t) + f(x + c * t)),
}

fig, axes = plt.subplots(1, 3, figsize=(9, 2.8), sharey=True)
for ax, (name, u) in zip(axes, solutions.items()):
    ax.plot(x, f(x), "k", label="$t = 0$")
    for t in [2, 4, 6]:
        ax.plot(x, u(x, t), label=f"$t = {t}$")
    ax.set_title(name)
    ax.set_xlabel("$x$")
axes[0].set_ylabel("$u$")
axes[0].legend(fontsize=8)
plt.tight_layout()
plt.show()
```

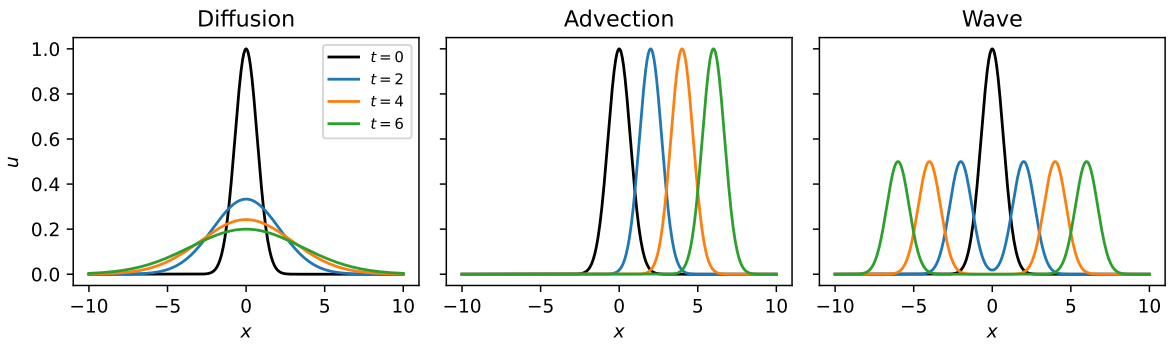


Figure 2.4: Exact solutions of the three prototype equations for the same Gaussian initial condition (black) at three later times.

Figure 2.4 contains the most important physical distinction of this lecture. Diffusion smooths and has no direction. Advection and waves transport information with a finite speed and preserve the shape. A good numerical method must reproduce this behaviour: it must not move a diffusing profile, and it must not smear or distort a travelling one. The second requirement turns out to be the harder one (Chapter 13).

A fourth prototype, the elliptic one, has no time derivative. In steady state, $\partial u/\partial t = 0$, the diffusion equation with a source becomes the **Poisson equation**

$$-\nabla^2 u = f, \tag{2.12}$$

and for $f = 0$ the **Laplace equation**. It describes equilibria: steady temperature and groundwater fields, the gravitational potential, the pressure in an incompressible fluid.

Figure 2.4 and the Poisson equation show why the classification matters: the three types behave differently.

	Elliptic	Parabolic	Hyperbolic
Describes	equilibrium	smoothing, relaxation to equilibrium	propagation
A disturbance is felt	everywhere at once	everywhere at once, but damped	only where the wave has arrived; finite speed
Solution is	smoother than the data	smoother than the initial condition	as rough as the initial condition
Needs	boundary conditions on the whole boundary	initial condition and boundary conditions	initial conditions; boundary conditions only where information enters
Numerical task	solve one large linear system	time stepping, often implicit	time stepping with attention to the propagation speed

2.5 The equations of this lecture

The equations of real models are combinations of the prototypes. We list the ones that appear in later units, so that you recognise the building blocks.

Shallow water equations. A layer of fluid of depth h and velocity u over a flat bottom, in one dimension and without rotation:

$$\begin{aligned} \frac{\partial h}{\partial t} + \frac{\partial (hu)}{\partial x} &= 0, \\ \frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + g\frac{\partial h}{\partial x} &= 0. \end{aligned} \tag{2.13}$$

The first equation is conservation of mass in flux form, the second is conservation of momentum in advective form. The system is nonlinear.

To see what it describes we **linearize** it about a state of rest with depth H. We write $h = H + \eta$ and assume that the surface elevation η and the velocity u are small, so that products of small quantities can be neglected: $hu \approx Hu$ and $u\,\partial u/\partial x \approx 0$. What remains is

$$\frac{\partial \eta}{\partial t} + H\frac{\partial u}{\partial x} = 0, \qquad \frac{\partial u}{\partial t} + g\frac{\partial \eta}{\partial x} = 0. \tag{2.14}$$

These are two first-order equations for two unknowns. We combine them into one second-order equation for η alone by eliminating u. The two equations are coupled through derivatives of u: the first contains $\partial u/\partial x$, the second $\partial u/\partial t$. Differentiating the first equation with respect to t and the second with respect to x produces the same mixed derivative of u in both. Since H and g are constants, they stay in front of the derivatives:

$$\frac{\partial^2 \eta}{\partial t^2} + H\frac{\partial^2 u}{\partial t\,\partial x} = 0, \qquad \frac{\partial^2 u}{\partial x\,\partial t} + g\frac{\partial^2 \eta}{\partial x^2} = 0.$$

For smooth functions the order of differentiation does not matter (Schwarz's theorem), so $\partial^2 u/\partial t\,\partial x = \partial^2 u/\partial x\,\partial t$. The second equation gives this mixed derivative in terms of η:

$$\frac{\partial^2 u}{\partial x\,\partial t} = -g\frac{\partial^2 \eta}{\partial x^2}.$$

Inserting it into the first equation,

$$\frac{\partial^2 \eta}{\partial t^2} + H\left(-g\frac{\partial^2 \eta}{\partial x^2}\right) = 0,$$

and moving the second term to the right-hand side gives

$$\frac{\partial^2 \eta}{\partial t^2} = gH\frac{\partial^2 \eta}{\partial x^2},$$

the wave equation Equation 2.11 with $c = \sqrt{gH}$. The shallow water equations describe **gravity waves**; they are a hyperbolic system. Note how the signs work out: the minus sign from solving the second equation for the mixed derivative and the one from moving the term to the right-hand side combine to $+gH$. Since $gH > 0$ the result is a wave equation; with $-gH$ it would be elliptic, and there would be no waves. Eliminating η instead, by differentiating the first equation with respect to x and the second with respect to t, gives the same wave equation for u. For an ocean of depth 4 km, $c = \sqrt{9.81 \cdot 4000}$ m/s ≈ 200 m/s, the speed of a tsunami. With rotation added the system contains much of the dynamics of the atmosphere and the ocean, which makes it the standard test bed for the numerical methods of weather and climate models (Chapter 15).

What makes a system of first-order equations hyperbolic? A first-order system

$$\frac{\partial \vec{q}}{\partial t} + B\frac{\partial \vec{q}}{\partial x} = 0$$

is hyperbolic if the matrix B has real eigenvalues and a full set of eigenvectors. It can then be decomposed into advection equations, one for each eigenvalue, and the eigenvalues are the speeds.

For the linearized shallow water equations Equation 2.14, $\vec{q} = (\eta, u)^T$ and

$$B = \begin{pmatrix} 0 & H \\ g & 0 \end{pmatrix}, \qquad \det(B - \lambda I) = \lambda^2 - gH = 0 \quad \Longrightarrow \quad \lambda = \pm\sqrt{gH}.$$

The two eigenvalues are the speeds of the gravity waves travelling to the right and to the left. For the full equations Equation 2.13 the same calculation gives $u \pm \sqrt{gh}$: the waves are in addition carried by the flow.

Primitive equations. The equations of atmospheric and ocean models are the conservation laws for momentum, mass and energy of a stratified fluid on the rotating sphere, with the hydrostatic approximation in the vertical. Each is an advection equation for one prognostic variable with source terms: pressure gradient and Coriolis force for momentum, heating for temperature, phase changes for humidity. Their dynamics are dominated by advection and waves, that is, by hyperbolic behaviour; diffusion terms represent the mixing by turbulence.

Groundwater flow. Conservation of mass with Darcy's law gives a diffusion equation for the hydraulic head h,

$$S_s \frac{\partial h}{\partial t} = \nabla \cdot (K \nabla h) + q, \tag{2.15}$$

with specific storage S_s, hydraulic conductivity K and sources q. It is parabolic, and in steady state elliptic. K varies in space over many orders of magnitude, which is the main numerical difficulty (Chapter 9).

Richards equation. In the unsaturated soil, water content θ and hydraulic conductivity K depend on the pressure head ψ:

$$\frac{\partial \theta(\psi)}{\partial t} = \nabla \cdot [K(\psi) \nabla(\psi + z)]\,. \tag{2.16}$$

The equation is a parabolic diffusion equation whose coefficients depend strongly on the solution. It is the prototype of a nonlinear problem in this lecture (Chapter 17).

Seismic waves. Conservation of momentum with Hooke's law gives, for a shear wave propagating in the x direction with particle velocity v and shear stress σ,

$$\rho \frac{\partial v}{\partial t} = \frac{\partial \sigma}{\partial x}, \qquad \frac{\partial \sigma}{\partial t} = \mu \frac{\partial v}{\partial x}, \tag{2.17}$$

with density ρ and shear modulus μ. For constant ρ and μ we differentiate the first equation with respect to t and insert the second:

$$\rho \frac{\partial^2 v}{\partial t^2} = \frac{\partial}{\partial x} \frac{\partial \sigma}{\partial t} = \mu \frac{\partial^2 v}{\partial x^2} \quad \Longrightarrow \quad \frac{\partial^2 v}{\partial t^2} = \frac{\mu}{\rho} \frac{\partial^2 v}{\partial x^2}.$$

This is the wave equation Equation 2.11 with the shear wave speed $c = \sqrt{\mu/\rho}$, and Equation 2.17 is a hyperbolic system. The structure is the same as in Equation 2.14: two first-order equations for two variables that drive each other.

Real models mix the types. The advection–diffusion equation Equation 2.7 is formally parabolic, but behaves like a hyperbolic equation when advection dominates. The same holds for the Navier–Stokes equations at high Reynolds number. Numerical models therefore often treat the terms of one equation with different methods (Chapter 18).

2.6 Initial and boundary conditions

A differential equation has many solutions. Initial and boundary conditions select one.

Initial conditions prescribe the state at $t = 0$. Their number equals the order of the time derivative: one for the diffusion equation, $u(x, 0) = u_0(x)$, and two for the wave equation, $u(x, 0)$ and $u_t(x, 0)$.

Boundary conditions prescribe the behaviour on the boundary $\partial\Omega$ of the domain Ω. For second-order equations three types are used, where $\partial u/\partial n$ is the derivative in the direction of the outward normal.

Type	Condition on $\partial\Omega$	Physical meaning	Example
Dirichlet	$u = g$	the value is prescribed	fixed temperature; water level of a lake bounding an aquifer
Neumann	$\partial u/\partial n = g$	the flux is prescribed	insulated wall or impermeable layer ($g = 0$); prescribed heat flux
Robin	$\partial u/\partial n + \alpha u = g$	the flux depends on the value	heat exchange with the surrounding air

A condition with $g = 0$ is called homogeneous. On each part of the boundary exactly one condition is prescribed.

A fourth kind is common in atmospheric modelling: **periodic** boundary conditions, $u(0,t) = u(L,t)$. On a latitude circle they are exact. Limited-area models face the opposite situation: their lateral boundaries are artificial, and waves must be able to leave the domain without reflection. Such open boundaries are a subject of their own (Chapter 16).

2.7 Well-posed problems

A problem is **well-posed** in the sense of Hadamard[1] if

1. a solution exists,
2. the solution is unique,
3. the solution depends continuously on the data, that is on initial conditions, boundary conditions and parameters.

If one of the conditions is violated, a numerical solution is meaningless or impossible. Three examples show that this is of practical relevance.

Too few conditions. The Poisson equation with Neumann conditions on the whole boundary prescribes only fluxes. If u is a solution, so is u plus any constant, which violates condition 2. Moreover a solution exists only if the prescribed boundary fluxes balance the sources, which concerns condition 1. In the discrete problem this shows up as a singular matrix.

The wrong direction of time. The diffusion equation smooths. Solving it backward in time, in order to find the initial state that led to an observed state, must therefore roughen: small-scale errors in the observed state are amplified without bound, which violates condition 3. Problems of this kind, where causes are inferred from effects, are the **inverse problems** of geophysics and the **data assimilation** problem of meteorology. They need additional information to become well-posed (Chapter 21).

The wrong kind of conditions. Hadamard's own example shows that the conditions must match the type of the equation. Take the Laplace equation in the upper half plane $y > 0$ and prescribe, as for the wave equation, the value and the normal derivative on the line $y = 0$:

$$u_{xx} + u_{yy} = 0, \qquad u(x,0) = 0, \qquad u_y(x,0) = \frac{1}{n}\sin(nx).$$

We look for a solution of the form $u = \sin(nx)\,Y(y)$. Inserting it, $u_{xx} = -n^2 \sin(nx)\,Y$ and $u_{yy} = \sin(nx)\,Y''$, so the PDE becomes the ODE $Y'' = n^2 Y$ with the general solution $Y = A\cosh(ny) + B\sinh(ny)$. The first condition gives $Y(0) = A = 0$. The second gives $Y'(0) = nB = 1/n$, so $B = 1/n^2$ and

[1] Jacques Hadamard (1865–1963), French mathematician, who formulated these conditions in 1902 and also proved the prime number theorem.

$$u(x,y) = \frac{1}{n^2}\sin(nx)\sinh(ny).$$

Now let $n \to \infty$. The data tend to zero, $|u_y(x,0)| \le 1/n$, and for zero data the solution is $u = 0$. But at any fixed height $y > 0$ the solution grows without bound, since $\sinh(ny) \approx \frac{1}{2}e^{ny}$: at $y = 1$ and $n = 20$, data of size 0.05 produce a solution of size $\frac{1}{400}\sinh(20) \approx 6 \cdot 10^5$. Condition 3 is violated.

The same data are harmless for the wave equation $u_{tt} = u_{xx}$ with t in place of y. There the ODE is $Y'' = -n^2 Y$, the hyperbolic functions become trigonometric ones, and the solution $u = \frac{1}{n^2}\sin(nx)\sin(nt)$ stays below $1/n^2$. The only difference is the sign in front of n^2, and that sign is the type of the equation. An elliptic equation needs one condition on the whole boundary, not two conditions on part of it (see the table in Section 2.4). In geophysics this example appears as the **downward continuation** of gravity and magnetic fields: computing the field at depth from measurements at the surface amplifies the short wavelengths exponentially.

Condition 3 for the mathematical problem has a counterpart for the numerical method: stability (Chapter 6). And well-posed does not mean predictable for all times. The equations of the atmosphere are well-posed, yet small errors in the initial state grow so fast that the forecast loses its value after about two weeks (Chapter 8).

2.8 Summary

- Gradient, divergence, curl and Laplacian are the building blocks of all equations in this lecture. The divergence is the net outflow per volume.
- A conservation law states that the amount in a volume changes only by the flux through its surface and by sources: $\partial u/\partial t + \nabla \cdot \vec{F} = s$. Advective and diffusive fluxes close the equation.
- Classify before you compute: ODE or PDE, order, linear or nonlinear, elliptic, parabolic or hyperbolic. The type decides the numerical method.
- The prototypes show what the types mean: diffusion smooths; advection and waves propagate with finite speed; the Poisson equation describes equilibria.
- The equations of atmospheric, hydrological and seismic models are combinations of the prototypes, and each can be classified the same way.
- A well-posed problem has the right number of initial and boundary conditions, and its solution depends continuously on the data.

2.9 Further reading

Boyce and DiPrima (2009), chapter 1, for the classification of ODEs; Wick (2020) for the classification of PDEs and well-posedness; Durran (2010), chapter 1, for the equations of

geophysical fluid dynamics.

3 The numerical toolbox I: derivatives, integrals, roots

Learning goals

After this unit you can

- derive finite difference formulas and their truncation error from the Taylor series,
- measure the order of convergence of a numerical method,
- explain why a finite difference cannot be made arbitrarily accurate, and estimate the best step,
- interpolate and integrate numerically and derive the error of the basic rules,
- find roots with bisection and Newton's method and explain their rates of convergence.

Numerical models are built from a small number of tools. This unit introduces the tools that come from calculus: approximating derivatives, interpolating, integrating and finding roots. All of them rest on the Taylor series. The next unit adds the tools from linear algebra and Fourier analysis.

About the code in this script

From this unit on, each computation is shown in two tabs.

- **Step by step** is written the way you would write it yourself after the first exercise sheets: explicit loops, one operation per line, a comment on each line. Read this version first. New Python is explained below the code where it first appears.
- **Compact** does the same in the style that experienced Python programmers use: whole NumPy arrays instead of loops, and functions from libraries such as SciPy. It is shorter and faster but hides the individual steps.

Both versions give the same results. In the PDF the two versions follow each other.

3.1 Taylor series and finite differences

3.1.1 The Taylor series

The Taylor series expresses the value of a smooth function at a neighbouring point by its value and its derivatives at x:

$$u(x+h) = u(x) + h\,u'(x) + \frac{h^2}{2}u''(x) + \frac{h^3}{6}u'''(x) + \frac{h^4}{24}u''''(x) + \dots \tag{3.1}$$

The general term is $\frac{h^k}{k!}u^{(k)}(x)$, with $2! = 2$, $3! = 6$, $4! = 24$. If the series is stopped after the term with h^n, the rest can be written as a single term,

$$u(x+h) = \sum_{k=0}^{n} \frac{h^k}{k!}u^{(k)}(x) + \frac{h^{n+1}}{(n+1)!}u^{(n+1)}(\xi)$$

with some ξ between x and $x+h$ (Lagrange form of the remainder). This is what makes the dots precise: the neglected part behaves like the first neglected term, provided $u^{(n+1)}$ is bounded. "Smooth" means that the derivatives that appear exist and are bounded.

Replacing h by $-h$ changes the sign of the odd terms, because $(-h)^k = -h^k$ for odd k:

$$u(x-h) = u(x) - h\,u'(x) + \frac{h^2}{2}u''(x) - \frac{h^3}{6}u'''(x) + \frac{h^4}{24}u''''(x) - \dots \tag{3.2}$$

Every finite difference formula is a combination of such series, chosen so that unwanted terms cancel.

Notation. We write $e = O(h^p)$, "e is of order h^p", if $|e| \le Ch^p$ for all sufficiently small h, with a constant C that does not depend on h. A method whose error is $O(h^p)$ is **of order** p. Halving h then divides the error by about 2^p.

3.1.2 Forward and backward difference

We solve Equation 3.1 for $u'(x)$. Subtracting $u(x)$ on both sides gives

$$u(x+h) - u(x) = h\,u'(x) + \frac{h^2}{2}u''(x) + \frac{h^3}{6}u'''(x) + \dots,$$

and dividing by h

$$\frac{u(x+h)-u(x)}{h} = u'(x) + \frac{h}{2}u''(x) + \frac{h^2}{6}u'''(x) + \dots .$$

Bringing everything except $u'(x)$ to the other side,

$$u'(x) = \underbrace{\frac{u(x+h)-u(x)}{h}}_{\text{forward difference}} \underbrace{- \frac{h}{2}u''(x) - \frac{h^2}{6}u'''(x) - \dots}_{\text{truncation error}} \tag{3.3}$$

The terms that are dropped when only the difference quotient is kept form the **truncation error**. Its leading term is $-\frac{h}{2}u''(x)$, proportional to h. The forward difference is of first order: its error is $O(h)$.

The same steps applied to Equation 3.2 give

$$u(x) - u(x-h) = h\,u'(x) - \frac{h^2}{2}u''(x) + \frac{h^3}{6}u'''(x) - \dots ,$$

and after division by h the **backward difference**

$$u'(x) = \frac{u(x)-u(x-h)}{h} + \frac{h}{2}u''(x) - \frac{h^2}{6}u'''(x) + \dots , \tag{3.4}$$

which is also of first order, with the opposite sign of the leading error.

3.1.3 Central difference

The errors of the two one-sided formulas have opposite signs, which suggests combining them. We subtract Equation 3.2 from Equation 3.1 term by term:

Term	in $u(x+h)$	in $u(x-h)$	difference
u	u	u	0
u'	$h\,u'$	$-h\,u'$	$2h\,u'$
u''	$\frac{h^2}{2}u''$	$\frac{h^2}{2}u''$	0
u'''	$\frac{h^3}{6}u'''$	$-\frac{h^3}{6}u'''$	$\frac{h^3}{3}u'''$
u''''	$\frac{h^4}{24}u''''$	$\frac{h^4}{24}u''''$	0

All even terms cancel:

$$u(x+h) - u(x-h) = 2h\,u'(x) + \frac{h^3}{3}u'''(x) + \dots$$

Dividing by $2h$ gives $\frac{u(x+h)-u(x-h)}{2h} = u'(x) + \frac{h^2}{6}u'''(x) + \ldots$, and solving for $u'(x)$ the **central difference**

$$u'(x) = \frac{u(x+h) - u(x-h)}{2h} - \frac{h^2}{6}u'''(x) - \ldots \tag{3.5}$$

Its error is $O(h^2)$: second order. Halving h halves the error of the one-sided differences but quarters the error of the central difference, at the same cost of two function values.

3.1.4 Second derivative

Adding the two series instead cancels the odd terms (last column of the table, with the signs of $u(x-h)$ reversed):

$$u(x+h) + u(x-h) = 2u(x) + h^2u''(x) + \frac{h^4}{12}u''''(x) + \ldots,$$

where $\frac{h^4}{12} = 2\cdot\frac{h^4}{24}$. Subtracting $2u(x)$ gives $u(x+h)-2u(x)+u(x-h) = h^2u''(x)+\frac{h^4}{12}u''''(x)+\ldots$, and dividing by h^2 and solving for $u''(x)$ gives the standard approximation of the second derivative,

$$u''(x) = \frac{u(x+h) - 2u(x) + u(x-h)}{h^2} - \frac{h^2}{12}u''''(x) - \ldots, \tag{3.6}$$

which is also of second order. This formula is the core of every finite difference model of diffusion.

There is a second way to obtain it, which becomes important when the diffusion coefficient varies in space. The second derivative is the derivative of the first derivative. We approximate the first derivative half-way between the grid points,

$$u'\Big(x+\tfrac{h}{2}\Big) \approx \frac{u(x+h)-u(x)}{h}, \qquad u'\Big(x-\tfrac{h}{2}\Big) \approx \frac{u(x)-u(x-h)}{h}.$$

Seen from the point $x+\frac{h}{2}$, the first quotient uses the two neighbours at distance $\frac{h}{2}$ on either side: it is a central difference with step $h/2$ and therefore of second order. The same holds for the second quotient at $x-\frac{h}{2}$. We then take the central difference of these two values, again with step $h/2$:

$$u''(x) \approx \frac{u'\Big(x+\frac{h}{2}\Big) - u'\Big(x-\frac{h}{2}\Big)}{h} = \frac{1}{h}\left[\frac{u(x+h)-u(x)}{h} - \frac{u(x)-u(x-h)}{h}\right] = \frac{u(x+h)-2u(x)+u(x-h)}{h^2}.$$

In this form the formula is a difference of two fluxes, as in the derivation of the divergence in Chapter 2.

3.1.5 Measuring the order of convergence

If the error behaves as $e(h) \approx Ch^p$, then taking the logarithm gives

$$\log e = \log C + p \log h.$$

In a plot with logarithmic axes the error is a straight line with slope p. To compute p from two step sizes h_1 and h_2 we divide the two errors. The unknown constant C drops out,

$$\frac{e(h_1)}{e(h_2)} = \frac{Ch_1^p}{Ch_2^p} = \left(\frac{h_1}{h_2}\right)^p,$$

and taking the logarithm of both sides, $\log\left(e(h_1)/e(h_2)\right) = p \log(h_1/h_2)$, gives the **observed order**

$$p = \frac{\log\left(e(h_1)/e(h_2)\right)}{\log\left(h_1/h_2\right)}. \tag{3.7}$$

If the step is halved, $h_1/h_2 = 2$ and $p = \log_2\left(e(h_1)/e(h_2)\right)$: an error that falls by a factor of 4 means second order.

Comparing the observed order with the theoretical one is the most important test of a numerical code. A bug almost always reduces the order. We test the forward and central difference with $u = \sin$, whose derivative cos is known, at $x = 1$.

Step by step

```
import numpy as np                    # NumPy: np.sin, np.cos, np.log, ...

x = 1.0                               # point where we approximate u'(x)
exact = np.cos(x)                     # exact derivative of sin at x

h_list = [0.1, 0.05, 0.025, 0.0125] # step sizes, each half the previous one
error_forward = []                    # empty lists, filled in the loop
error_central = []

for h in h_list:                      # h takes the values 0.1, 0.05, ... in turn
    forward = (np.sin(x + h) - np.sin(x)) / h
    central = (np.sin(x + h) - np.sin(x - h)) / (2 * h)
```

```
    error_forward.append(abs(forward - exact))   # add the error to the list
    error_central.append(abs(central - exact))

print("     h    error forward  order  error central  order")
for i in range(1, len(h_list)):     # i = 1, 2, 3; compare step i with step i-1
    log_ratio_h = np.log(h_list[i - 1] / h_list[i])
    p_forward = np.log(error_forward[i - 1] / error_forward[i]) / log_ratio_h
    p_central = np.log(error_central[i - 1] / error_central[i]) / log_ratio_h
    print(f"{h_list[i]:7.4f} {error_forward[i]:12.2e} {p_forward:6.2f}"
          f" {error_central[i]:14.2e} {p_central:6.2f}")
```

```
     h    error forward  order  error central  order
 0.0500      2.13e-02   1.01       2.25e-04   2.00
 0.0250      1.06e-02   1.01       5.63e-05   2.00
 0.0125      5.27e-03   1.00       1.41e-05   2.00
```

Compact

```
import numpy as np

x = 1.0
h = np.array([0.1, 0.05, 0.025, 0.0125])      # all step sizes at once
error_forward = abs((np.sin(x + h) - np.sin(x)) / h - np.cos(x))
error_central = abs((np.sin(x + h) - np.sin(x - h)) / (2 * h) - np.cos(x))

# e[:-1] is the array without its last, e[1:] without its first element
def observed_order(e, h):
    return np.log(e[:-1] / e[1:]) / np.log(h[:-1] / h[1:])

print("forward, observed order:", observed_order(error_forward, h).round(3))
print("central, observed order:", observed_order(error_central, h).round(3))
```

```
forward, observed order: [1.014 1.007 1.004]
central, observed order: [1.999 2.    2.   ]
```

The observed orders are 1 and 2, as derived. Moreover the forward error is close to $\frac{h}{2}|u''(1)| = \frac{h}{2}\sin 1 \approx 0.42\,h$, the leading term of Equation 3.3.

New Python in the step-by-step version.

- `[0.1, 0.05, ...]` is a **list**, `[]` an empty list. `error_forward.append(value)` adds a value at the end of the list.

- `for h in h_list:` executes the indented lines once for each element of the list, with `h` equal to that element. It replaces the counter of a `while` loop.
- `range(1, len(h_list))` produces the whole numbers $1, 2, \ldots$ up to `len(h_list) - 1`; `len` is the number of elements. `h_list[i]` is element number `i`, counted from **0**: `h_list[0]` is 0.1.
- In `f"{h:7.4f}"` the part after the colon sets the format: 7 characters wide with 4 digits after the decimal point. `e` instead of `f` gives the exponential notation, `1.23e-04`.

3.1.6 Why the step cannot be made arbitrarily small

According to Equation 3.3 the error vanishes for $h \to 0$. On a computer it does not, and we can estimate why.

The computer does not hold $u(x)$ and $u(x+h)$ but rounded values $u(x)(1+\delta_1)$ and $u(x+h)(1+\delta_2)$ with $|\delta_{1,2}| \le \varepsilon$, the machine epsilon (Chapter 1). The computed numerator is

$$u(x+h)(1+\delta_2) - u(x)(1+\delta_1) = \big[u(x+h) - u(x)\big] + u(x+h)\,\delta_2 - u(x)\,\delta_1,$$

so it differs from the exact numerator by

$$|u(x+h)\,\delta_2 - u(x)\,\delta_1| \le \varepsilon\,\big(|u(x+h)| + |u(x)|\big) \approx 2\varepsilon|u|,$$

where we used $u(x+h) \approx u(x)$ for small h and wrote $|u|$ for $|u(x)|$. The division by h is itself accurate to ε, but it turns this absolute error of the numerator into an error of $2\varepsilon|u|/h$ in the difference quotient: the smaller h, the larger the error. Together with the leading truncation error from Equation 3.3 the total error of the forward difference is bounded by

$$e(h) = \underbrace{\frac{h}{2}|u''|}_{\text{truncation}} + \underbrace{\frac{2\varepsilon|u|}{h}}_{\text{round-off}}. \tag{3.8}$$

The first term decreases with h, the second increases. The minimum is where the derivative with respect to h vanishes:

$$\frac{\mathrm{d}e}{\mathrm{d}h} = \frac{|u''|}{2} - \frac{2\varepsilon|u|}{h^2} = 0 \quad\Longrightarrow\quad h^2 = \frac{4\varepsilon|u|}{|u''|} \quad\Longrightarrow\quad h_{\text{opt}} = 2\sqrt{\frac{\varepsilon|u|}{|u''|}}.$$

Inserting h_{opt} into Equation 3.8, both terms become equal,

$$\frac{h_{\text{opt}}}{2}|u''| = \sqrt{\frac{\varepsilon|u|}{|u''|}}\,|u''| = \sqrt{\varepsilon|u||u''|}, \qquad \frac{2\varepsilon|u|}{h_{\text{opt}}} = \varepsilon|u|\sqrt{\frac{|u''|}{\varepsilon|u|}} = \sqrt{\varepsilon|u||u''|},$$

and the smallest error is their sum,

$$e(h_{\text{opt}}) = 2\sqrt{\varepsilon\,|u|\,|u''|}.$$

If u and u'' are of order one, the best step is of the order $\sqrt{\varepsilon} \approx 10^{-8}$, and the smallest error that can be reached is also about 10^{-8}: half of the 16 digits are lost.

For the central difference the numerator has the same round-off error $2\varepsilon|u|$, but it is divided by $2h$, which gives $\varepsilon|u|/h$. With the truncation error $\frac{h^2}{6}|u'''|$ from Equation 3.5,

$$e(h) = \frac{h^2}{6}|u'''| + \frac{\varepsilon|u|}{h}, \qquad \frac{\mathrm{d}e}{\mathrm{d}h} = \frac{h}{3}|u'''| - \frac{\varepsilon|u|}{h^2} = 0 \quad \Longrightarrow \quad h_{\text{opt}} = \left(\frac{3\varepsilon|u|}{|u'''|}\right)^{1/3}.$$

At h_{opt} we have $\varepsilon|u| = \frac{1}{3}h_{\text{opt}}^3|u'''|$, so the round-off term is $\frac{1}{3}h_{\text{opt}}^2|u'''|$, twice the truncation term, and

$$e(h_{\text{opt}}) = \frac{h_{\text{opt}}^2}{6}|u'''| + \frac{h_{\text{opt}}^2}{3}|u'''| = \frac{h_{\text{opt}}^2}{2}|u'''|.$$

For u and u''' of order one, $h_{\text{opt}} \sim \varepsilon^{1/3} \approx 10^{-5}$ and $e(h_{\text{opt}}) \sim \varepsilon^{2/3} \approx 10^{-11}$.

Step by step

```
import matplotlib.pyplot as plt     # the plotting library

x = 1.0
exact = np.cos(x)
h_values = []                       # step sizes
error_forward = []                  # errors of the two formulas
error_central = []
slope_1 = []                        # reference lines h/2 and h^2/6
slope_2 = []

for k in range(60):                 # k = 0, 1, ..., 59
    h = 10.0 ** (-15 + 0.25 * k)    # h = 10^-15, 10^-14.75, ..., 10^-0.25
    forward = (np.sin(x + h) - np.sin(x)) / h
    central = (np.sin(x + h) - np.sin(x - h)) / (2 * h)
    h_values.append(h)
    error_forward.append(abs(forward - exact))
    error_central.append(abs(central - exact))
    slope_1.append(h / 2)
    slope_2.append(h**2 / 6)
```

```
fig, ax = plt.subplots(figsize=(6, 3.8))     # a figure with one set of axes
ax.loglog(h_values, error_forward, ".", label="forward difference")
ax.loglog(h_values, error_central, ".", label="central difference")
ax.loglog(h_values, slope_1, "k--", lw=0.8, label="slope 1 and 2")
ax.loglog(h_values, slope_2, "k--", lw=0.8)
ax.set_ylim(1e-13, 1)                # show errors between 1e-13 and 1
ax.set_xlabel("$h$")
ax.set_ylabel("error")
ax.legend()                          # the box with the labels
plt.show()
```

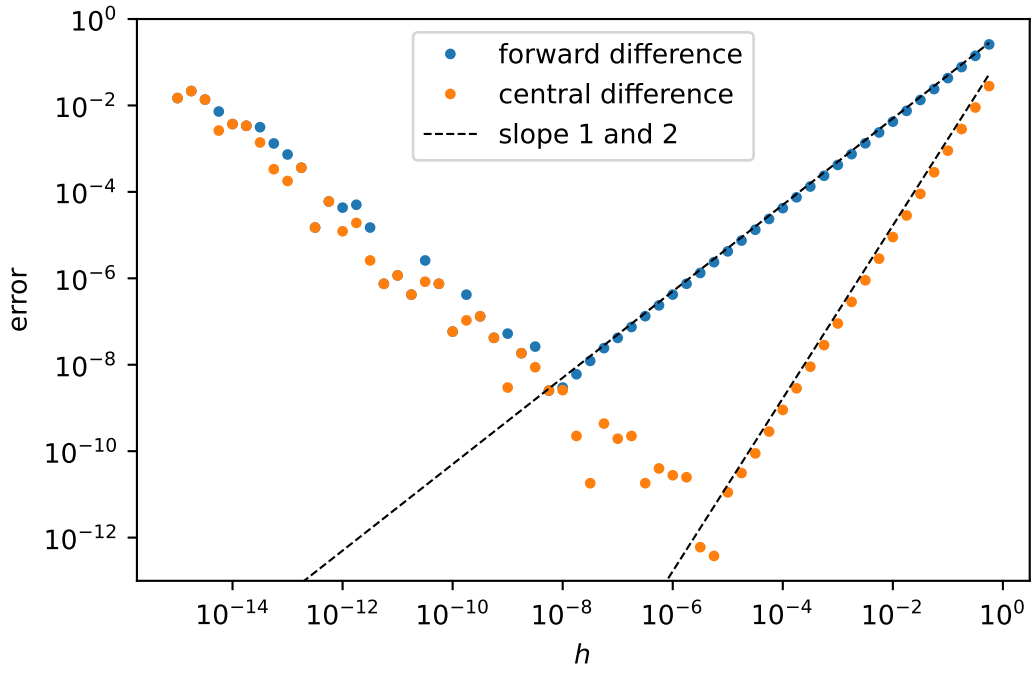


Figure 3.1: Error of the forward and central difference for the derivative of $\sin x$ at $x = 1$. The error decreases with the theoretical slope until round-off takes over.

Compact

```
import matplotlib.pyplot as plt

h = 10.0 ** np.arange(-15, 0, 0.25)          # all 60 step sizes as one array
forward = abs((np.sin(x + h) - np.sin(x)) / h - np.cos(x))
central = abs((np.sin(x + h) - np.sin(x - h)) / (2 * h) - np.cos(x))
```

```
fig, ax = plt.subplots(figsize=(6, 3.8))
ax.loglog(h, forward, ".", label="forward difference")
ax.loglog(h, central, ".", label="central difference")
ax.loglog(h, h / 2, "k--", lw=0.8, label="slope 1 and 2")
ax.loglog(h, h**2 / 6, "k--", lw=0.8)
ax.set(ylim=(1e-13, 1), xlabel="$h$", ylabel="error")
ax.legend()
plt.show()
```

The exponents are computed from the integer counter `k`, as in the time loop of Exercise 1, instead of adding 0.25 repeatedly. `ax.loglog` plots with logarithmic axes; `"."` draws dots, `"k--"` a black (k) dashed line, `lw` is the line width.

Figure 3.1 confirms the estimates: the error of the forward difference stops decreasing near $h = 10^{-8}$, at about 10^{-8}, that of the central difference near $h = 10^{-5}$, at about 10^{-11}. Single points can lie lower, where the rounding errors happen to cancel; the estimates are bounds for the typical error. Left of the optimum the error grows like $1/h$ and scatters, because the rounding errors δ_1, δ_2 vary irregularly from one h to the next. The figure has two lessons. First, a higher order gives a smaller error for the same step and is less affected by round-off. Second, in a convergence test the step must be neither so large that higher-order terms disturb the slope nor so small that round-off dominates.

3.1.7 Stencils of any order

The points used by a difference formula form its **stencil**. Formulas for any stencil and any derivative follow from one recipe, the method of undetermined coefficients. We look for weights w_j such that

$$\sum_j w_j \, u(x + s_j h) \approx u^{(m)}(x),$$

where the offsets s_j define the stencil.

Worked example. We want the first derivative at a boundary point x, where only the points x, $x + h$ and $x + 2h$ are available: offsets $s = (0, 1, 2)$. We write the Taylor series of the three values,

$$\begin{aligned} u(x) &= u(x), \\ u(x+h) &= u(x) + h\,u'(x) + \tfrac{1}{2}h^2 u''(x) + \tfrac{1}{6}h^3 u'''(x) + \dots, \\ u(x+2h) &= u(x) + 2h\,u'(x) + \tfrac{(2h)^2}{2}u''(x) + \tfrac{(2h)^3}{6}u'''(x) + \dots \\ &= u(x) + 2h\,u'(x) + 2h^2 u''(x) + \tfrac{8}{6}h^3 u'''(x) + \dots, \end{aligned}$$

and collect, for each derivative of u, the factor with which it appears:

	$u(x)$	$h\,u'(x)$	$h^2u''(x)$	$h^3u'''(x)$
$u(x)$	1	0	0	0
$u(x+h)$	1	1	$\frac{1}{2}$	$\frac{1}{6}$
$u(x+2h)$	1	2	2	$\frac{8}{6}$

The combination $w_0u(x) + w_1u(x+h) + w_2u(x+2h)$ therefore contains u with the factor $w_0+w_1+w_2$, hu' with the factor w_1+2w_2, and so on: each column, multiplied by the weights and summed. With three weights we can impose three conditions. The sum shall contain $u'(x)$ exactly once and neither u nor u'':

$$\begin{aligned} w_0 + w_1 + w_2 &= 0 \quad (\text{no } u) \\ w_1 + 2w_2 &= \tfrac{1}{h} \quad (hu' \text{ with factor } \tfrac{1}{h}, \text{ that is } u' \text{ once}) \\ \tfrac{1}{2}w_1 + 2w_2 &= 0 \quad (\text{no } u'') \end{aligned}$$

Subtracting the third equation from the second gives $\frac{1}{2}w_1 = \frac{1}{h}$, hence $w_1 = \frac{2}{h}$. The third equation then gives $2w_2 = -\frac{1}{2}w_1 = -\frac{1}{h}$, hence $w_2 = -\frac{1}{2h}$, and the first $w_0 = -w_1 - w_2 = -\frac{3}{2h}$.

The leading error comes from the last column, which was not controlled:

$$\left(\tfrac{1}{6}w_1 + \tfrac{8}{6}w_2\right) h^3u''' = \left(\tfrac{1}{3h} - \tfrac{2}{3h}\right) h^3u''' = -\tfrac{1}{3}h^2u'''.$$

The weighted sum is thus $u'(x) - \frac{1}{3}h^2u'''(x) + \ldots$. Solving for $u'(x)$ gives the one-sided formula of second order,

$$u'(x) = \frac{-3u(x) + 4u(x+h) - u(x+2h)}{2h} + \frac{h^2}{3}u'''(x) + \ldots \tag{3.9}$$

It is needed at boundaries, where no neighbour exists on one side (Chapter 9). Its error constant, $\frac{1}{3}$, is twice that of the central difference: one-sided formulas are less accurate than central ones of the same order.

The general recipe. For offsets $s_0, \ldots, s_{n-1}$ the Taylor series of each value is $u(x+s_jh) = \sum_k \frac{(s_jh)^k}{k!}u^{(k)}(x)$. The weighted sum is

$$\sum_j w_j\, u(x+s_jh) = \sum_k \left(\sum_j w_j\, s_j^k\right) \frac{h^k}{k!}u^{(k)}(x).$$

It equals $u^{(m)}(x)$ up to terms with $k \geq n$ if the bracket is $m!/h^m$ for $k = m$ and zero for all other $k = 0, \ldots, n-1$. These are n linear equations for the n weights,

$$\sum_{j=0}^{n-1} s_j^k \, w_j = \begin{cases} m!/h^m, & k = m, \\ 0, & k \neq m, \end{cases} \qquad k = 0, \ldots, n-1. \tag{3.10}$$

For $s = (0, 1, 2)$ and $m = 1$ these are exactly the three equations of the worked example. The weights scale with $1/h^m$, so it suffices to compute them for $h = 1$ and divide by h^m. A more efficient algorithm for the same task is due to Fornberg (1988).

Step by step

```
from math import factorial          # factorial(3) = 3 * 2 * 1 = 6

def stencil_weights(offsets, m):
    # weights for the m-th derivative on the given offsets, for h = 1
    n = len(offsets)                # number of stencil points
    A = np.zeros((n, n))            # n x n matrix filled with zeros
    b = np.zeros(n)                 # right hand side, n zeros
    for k in range(n):              # row k: condition for the k-th derivative
        for j in range(n):          # column j: stencil point j
            A[k, j] = offsets[j] ** k
        if k == m:
            b[k] = factorial(m)
    w = np.linalg.solve(A, b)       # solve the linear system A w = b
    return w

print("central, 1st derivative:  ", stencil_weights([-1, 0, 1], 1))
print("central, 2nd derivative:  ", stencil_weights([-1, 0, 1], 2))
print("one-sided, 1st derivative:", stencil_weights([0, 1, 2], 1))
w = stencil_weights([-2, -1, 0, 1, 2], 1)
print("5 points, 1st derivative: ", w.round(4))
```

```
central, 1st derivative:   [-0.5  0.   0.5]
central, 2nd derivative:   [ 1. -2.  1.]
one-sided, 1st derivative: [-1.5  2.  -0.5]
5 points, 1st derivative:  [ 0.0833 -0.6667  0.      0.6667 -0.0833]
```

Compact

```python
from math import factorial

def stencil_weights(offsets, m):
    s = np.asarray(offsets, dtype=float)
    A = np.vander(s, increasing=True).T          # A[k, j] = s[j]**k
    b = np.zeros(s.size)
    b[m] = factorial(m)
    return np.linalg.solve(A, b)

cases = [([-1, 0, 1], 1), ([-1, 0, 1], 2), ([0, 1, 2], 1), ([-2, -1, 0, 1, 2], 1)]
for offsets, m in cases:
    print(f"{offsets}, derivative {m}:", stencil_weights(offsets, m).round(4))
```

```
[-1, 0, 1], derivative 1: [-0.5  0.   0.5]
[-1, 0, 1], derivative 2: [ 1. -2.  1.]
[0, 1, 2], derivative 1: [-1.5  2.  -0.5]
[-2, -1, 0, 1, 2], derivative 1: [ 0.0833 -0.6667  0.      0.6667 -0.0833]
```

The first three results reproduce Equation 3.5, Equation 3.6 and Equation 3.9 (weights $-\frac{1}{2}, 0, \frac{1}{2}$; $1, -2, 1$; $-\frac{3}{2}, 2, -\frac{1}{2}$). The fourth, $\frac{1}{12}(1, -8, 0, 8, -1)$, cancels one more pair of terms and has order four. Wider stencils give higher order, at the price of more work per point and of difficulties near boundaries.

New Python. `def name(arguments):` defines a **function**: the indented lines are executed when the function is called, and `return` hands back the result. `np.zeros((n, n))` creates an $n \times n$ array of zeros, and `A[k, j]` is its entry in row `k` and column `j`, both counted from 0. The two `for` loops are **nested**: for each `k` the inner loop runs through all `j`. `np.linalg.solve(A, b)` solves the linear system $A\vec{w} = \vec{b}$; how this is done is the subject of the next unit. Note that Python computes `0 ** 0` as 1, as the first row of Equation 3.10 requires.

3.2 Interpolation

Interpolation constructs a function that passes through given data points (x_j, u_j). Models need it all the time: to obtain a value between grid points, to transfer fields between different grids, to find the value at the departure point of an air parcel (Chapter 14).

3.2.1 Linear interpolation

Between two neighbours x_j and $x_{j+1} = x_j + h$ the straight line through the two data points is

$$p(x) = u_j + \frac{u_{j+1} - u_j}{h}(x - x_j). \tag{3.11}$$

At $x = x_j$ it gives u_j, at $x = x_{j+1}$ it gives $u_j + (u_{j+1} - u_j) = u_{j+1}$. The special case at the midpoint, $x - x_j = h/2$, is the average $\frac{1}{2}(u_j + u_{j+1})$, which is used whenever a value is needed half-way between grid points.

How large is the error? For a polynomial p of degree n through $n+1$ points $x_0, \ldots, x_n$ the interpolation error is

$$u(x) - p(x) = \frac{u^{(n+1)}(\xi)}{(n+1)!} \prod_{j=0}^{n} (x - x_j) \tag{3.12}$$

for some ξ in the interval that contains x and the data points.

Derivation of the interpolation error

At a data point both sides are zero. Fix an x that is not a data point and write $w(t) = \prod_{j=0}^{n}(t - x_j)$, a polynomial of degree $n+1$ with leading term t^{n+1}. The function

$$g(t) = u(t) - p(t) - \frac{u(x) - p(x)}{w(x)}\, w(t)$$

is zero at the $n+1$ data points, because there $u = p$ and $w = 0$, and also at $t = x$, where the last term equals $u(x) - p(x)$. It has $n+2$ zeros. Between two zeros of a smooth function its derivative has a zero (Rolle's theorem). Hence g' has at least $n+1$ zeros, g'' at least n, and $g^{(n+1)}$ at least one, at some point ξ. Now $p^{(n+1)} = 0$, because p has degree n, and $w^{(n+1)} = (n+1)!$, because only the leading term t^{n+1} survives $n+1$ differentiations. Therefore

$$0 = g^{(n+1)}(\xi) = u^{(n+1)}(\xi) - \frac{u(x) - p(x)}{w(x)}\,(n+1)!,$$

which, solved for $u(x) - p(x)$, is Equation 3.12.

For the straight line, $n = 1$, the error is $\frac{1}{2}u''(\xi)\,(x - x_j)(x - x_{j+1})$. Writing $t = x - x_j$, the product is $t\,(t - h)$ for $0 \le t \le h$. Its derivative $2t - h$ vanishes at $t = h/2$, the midpoint, where the product equals $\frac{h}{2} \cdot \left(-\frac{h}{2}\right) = -\frac{h^2}{4}$. Hence

$$|u(x) - p(x)| \le \frac{1}{2} \cdot \frac{h^2}{4} \max |u''| = \frac{h^2}{8} \max |u''|.$$

Linear interpolation is of second order, and it is exact where the function has no curvature.

3.2.2 Polynomials of high degree and the Runge phenomenon

Polynomial interpolation passes a single polynomial of degree n through $n+1$ points. It can be written down directly in the Lagrange form

$$p(x) = \sum_{j=0}^{n} u_j \, L_j(x), \qquad L_j(x) = \prod_{i \neq j} \frac{x - x_i}{x_j - x_i}.$$

Each L_j is a product of n linear factors, a polynomial of degree n. At $x = x_j$ every factor equals one, so $L_j(x_j) = 1$. At any other data point x_i the factor $(x - x_i)$ is zero, so $L_j(x_i) = 0$. In the sum for $p(x_i)$ therefore only the term $j = i$ survives, and $p(x_i) = u_i$. For $n = 1$ the Lagrange form is $p(x) = u_0 \frac{x-x_1}{x_0-x_1} + u_1 \frac{x-x_0}{x_1-x_0}$, which is Equation 3.11 written differently.

From Equation 3.12 one would expect the accuracy to improve with n, because of the factor $1/(n+1)!$. On equidistant points it does not. The product $\prod(x - x_j)$ is much larger near the ends of the interval than in the middle, and for many functions the derivatives $u^{(n+1)}$ grow faster than the factorial. The polynomial oscillates near the ends, and the oscillations grow with n. This is the **Runge phenomenon**.

There are two remedies. **Piecewise interpolation** uses polynomials of low degree between neighbouring points. **Cubic splines** are piecewise cubic polynomials that in addition make the first and second derivative continuous at the data points. Alternatively, the data points can be placed more densely near the ends, for example at the Chebyshev points $x_j = \cos(j\pi/n)$; this keeps the product $\prod(x - x_j)$ small everywhere. Models rarely have this freedom: the grid is given.

Step by step

```
from scipy.interpolate import CubicSpline   # cubic splines from SciPy

def runge(x):
    return 1 / (1 + 25 * x**2)

def lagrange(x_data, u_data, x):
    # value at x of the polynomial through the points (x_data, u_data)
    n_points = len(x_data)
    p = 0.0
    for j in range(n_points):
        L = 1.0                     # L_j(x), built up factor by factor
        for i in range(n_points):
            if i != j:              # != means "not equal"
                L = L * (x - x_data[i]) / (x_data[j] - x_data[i])
        p = p + u_data[j] * L
    return p
```

```
x_data = []                          # 11 equidistant data points in [-1, 1]
u_data = []
for j in range(11):
    x_data.append(-1 + 0.2 * j)
    u_data.append(runge(-1 + 0.2 * j))

spline = CubicSpline(x_data, u_data)        # the spline through the data

x_fine = []                          # 401 points for drawing the curves
u_exact = []
u_polynomial = []
u_spline = []
for k in range(401):
    x = -1 + 0.005 * k
    x_fine.append(x)
    u_exact.append(runge(x))
    u_polynomial.append(lagrange(x_data, u_data, x))
    u_spline.append(spline(x))

fig, ax = plt.subplots(figsize=(6, 3.5))
ax.plot(x_fine, u_exact, "k", label="function")
ax.plot(x_fine, u_polynomial, label="polynomial, degree 10")
ax.plot(x_fine, u_spline, "--", label="cubic spline")
ax.plot(x_data, u_data, "ko", ms=4)         # data points as black circles
ax.set_xlabel("$x$")
ax.legend()
plt.show()
```

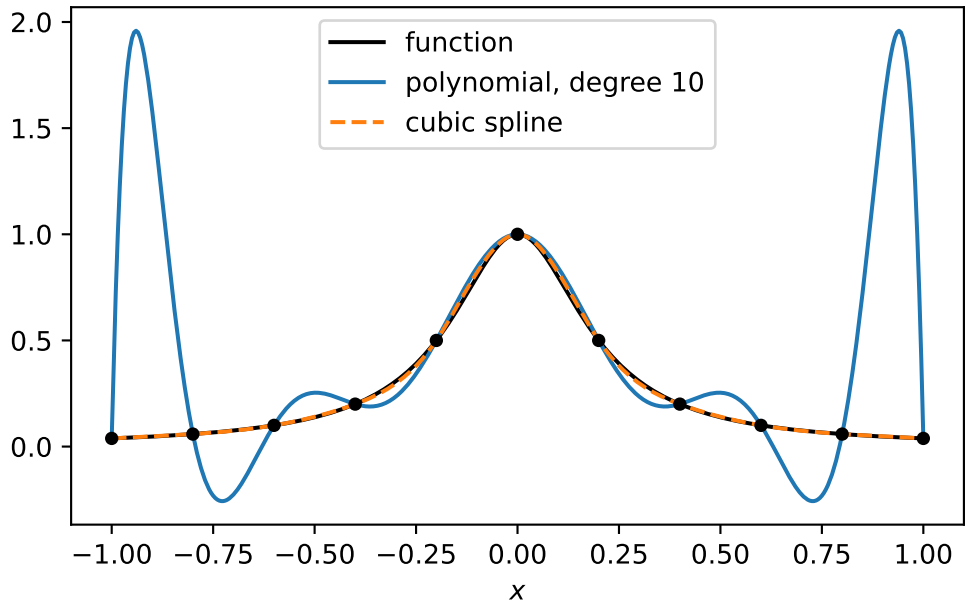


Figure 3.2: Interpolation of $1/(1+25x^2)$ through 11 equidistant points. The polynomial of degree 10 oscillates near the ends; the cubic spline does not.

Compact

```
from scipy.interpolate import BarycentricInterpolator, CubicSpline

runge = lambda x: 1 / (1 + 25 * x**2)       # a function in one line
x_data = np.linspace(-1, 1, 11)             # 11 equidistant points
x_fine = np.linspace(-1, 1, 401)
polynomial = BarycentricInterpolator(x_data, runge(x_data))
spline = CubicSpline(x_data, runge(x_data))

fig, ax = plt.subplots(figsize=(6, 3.5))
ax.plot(x_fine, runge(x_fine), "k", label="function")
ax.plot(x_fine, polynomial(x_fine), label="polynomial, degree 10")
ax.plot(x_fine, spline(x_fine), "--", label="cubic spline")
ax.plot(x_data, runge(x_data), "ko", ms=4)
ax.set_xlabel("$x$")
ax.legend()
plt.show()
```

The step-by-step function `lagrange` is a direct transcription of the Lagrange form. The cubic spline requires solving a linear system for the second derivatives at the data points; we take it

from the library SciPy. `CubicSpline(x_data, u_data)` returns an object that can be called like a function. The compact version uses `BarycentricInterpolator`, a numerically more stable way to evaluate the same polynomial.

The lesson reaches beyond interpolation. Accuracy is improved reliably by refining the grid with a method of fixed, moderate order. Raising the order on a fixed grid works only under special conditions (Chapter 20).

3.3 Quadrature

Quadrature approximates an integral by a weighted sum of function values. The interval $[a, b]$ is divided into N subintervals of width $h = (b - a)/N$, and on each the integrand is replaced by a polynomial.

3.3.1 Midpoint and trapezoidal rule

Consider one subinterval of width h with midpoint m. We expand the integrand around the midpoint, $f(m+s) = f(m) + sf'(m) + \frac{1}{2}s^2 f''(m) + \frac{1}{6}s^3 f'''(m) + \ldots$, and integrate term by term from $s = -h/2$ to $s = h/2$. The terms with odd powers of s are odd functions and integrate to zero over the symmetric interval. For the others,

$$\int_{-h/2}^{h/2} 1 \,\mathrm{d}s = h, \qquad \int_{-h/2}^{h/2} s^2 \,\mathrm{d}s = \left[\frac{s^3}{3}\right]_{-h/2}^{h/2} = \frac{2}{3}\left(\frac{h}{2}\right)^3 = \frac{h^3}{12}.$$

Hence

$$\int_{m-h/2}^{m+h/2} f(x)\,\mathrm{d}x = h\,f(m) + \frac{f''(m)}{2}\cdot\frac{h^3}{12} + O(h^5) = \underbrace{h\,f(m)}_{\text{midpoint rule}} + \frac{h^3}{24}f''(m) + O(h^5). \quad (3.13)$$

The **trapezoidal rule** uses the two end points instead. From the same expansion at $s = \pm h/2$, $f(m \pm \frac{h}{2}) = f(m) \pm \frac{h}{2}f'(m) + \frac{h^2}{8}f''(m) \pm \ldots$, so that in the sum of the two the f' terms cancel:

$$f\Big(m - \tfrac{h}{2}\Big) + f\Big(m + \tfrac{h}{2}\Big) = 2f(m) + \frac{h^2}{4}f''(m) + O(h^4),$$

and

$$\frac{h}{2}\left[f\Big(m - \tfrac{h}{2}\Big) + f\Big(m + \tfrac{h}{2}\Big)\right] = h\,f(m) + \frac{h^3}{8}f''(m) + O(h^5).$$

We replace $hf(m)$ with Equation 3.13, $hf(m) = \int f\,\mathrm{d}x - \frac{h^3}{24}f''(m)$:

$$\frac{h}{2}\,[\,\ldots\,] = \int f\,\mathrm{d}x - \frac{h^3}{24}f''(m) + \frac{h^3}{8}f''(m) = \int f\,\mathrm{d}x + \frac{h^3}{12}f''(m),$$

since $\frac{1}{8} - \frac{1}{24} = \frac{3-1}{24} = \frac{1}{12}$. Solved for the integral,

$$\int_{m-h/2}^{m+h/2} f(x)\,\mathrm{d}x = \underbrace{\frac{h}{2}\left[f\Big(m-\tfrac{h}{2}\Big) + f\Big(m+\tfrac{h}{2}\Big)\right]}_{\text{trapezoidal rule}} - \frac{h^3}{12}f''(m) + O(h^5). \tag{3.14}$$

Composite rules. On the whole interval we sum over the N subintervals with midpoints $m_i = a + (i+\frac{1}{2})h$, $i = 0, \ldots, N-1$, and end points $x_i = a + ih$, $i = 0, \ldots, N$. The composite midpoint rule is $h\sum_i f(m_i)$. In the composite trapezoidal rule every interior point $x_1, \ldots, x_{N-1}$ is the end point of two subintervals and appears twice with the weight $h/2$:

$$T_N = h\left[\tfrac{1}{2}f(x_0) + f(x_1) + \cdots + f(x_{N-1}) + \tfrac{1}{2}f(x_N)\right].$$

The local errors add up. For the midpoint rule

$$\sum_{i=0}^{N-1} \frac{h^3}{24}f''(m_i) = \frac{h^2}{24}\sum_{i=0}^{N-1} h\,f''(m_i) \approx \frac{h^2}{24}\int_a^b f''(x)\,\mathrm{d}x = \frac{h^2}{24}\,[f'(b) - f'(a)],$$

because $\sum_i hf''(m_i)$ is itself the midpoint rule for $\int f''$. The same argument for the trapezoidal rule gives $-\frac{h^2}{12}\,[f'(b) - f'(a)]$. Each subinterval contributes $O(h^3)$, but there are $N = (b-a)/h$ of them, so the composite error is $O(h^2)$: both composite rules are of **second order**. The error of the trapezoidal rule is twice that of the midpoint rule and has the opposite sign.

The composite error depends only on the derivative of f at the two end points. If f is periodic and the interval is one period, $f'(b) = f'(a)$ and the h^2 term vanishes. A more complete analysis (the Euler–Maclaurin formula) shows that the same holds for all higher terms, which contain $f'''(b) - f'''(a)$, $f^{(5)}(b) - f^{(5)}(a)$, and so on. For smooth periodic functions the trapezoidal rule is therefore extremely accurate.

3.3.2 Simpson's rule

Opposite signs again invite a combination. Taking two thirds of Equation 3.13 and one third of Equation 3.14,

$$\tfrac{2}{3}\left(\int f - \tfrac{h^3}{24}f''\right) + \tfrac{1}{3}\left(\int f + \tfrac{h^3}{12}f''\right) = \int f + \left(-\tfrac{2}{72} + \tfrac{1}{36}\right)h^3 f'' = \int f,$$

the h^3 terms cancel. In function values, the combination is

$$\tfrac{2}{3}h\,f(m) + \tfrac{1}{3}\cdot\tfrac{h}{2}\left[f\left(m-\tfrac{h}{2}\right) + f\left(m+\tfrac{h}{2}\right)\right] = \frac{h}{6}\left[f\left(m-\tfrac{h}{2}\right) + 4f(m) + f\left(m+\tfrac{h}{2}\right)\right],$$

which is **Simpson's rule**:

$$\int_{m-h/2}^{m+h/2} f(x)\,\mathrm{d}x \approx \frac{h}{6}\left[f\left(m-\tfrac{h}{2}\right) + 4f(m) + f\left(m+\tfrac{h}{2}\right)\right].$$

The next terms of both rules are odd powers of s, which vanish, and terms with h^5. The local error of Simpson's rule is therefore $O(h^5)$, and its composite form is of **fourth order**. Many textbooks write Simpson's rule with the distance $\hat{h} = h/2$ between neighbouring points, $\frac{\hat{h}}{3}[f_0 + 4f_1 + f_2]$; this is the same formula.

Rule	On one subinterval	Error of the composite rule
Midpoint	$h\,f(m)$	$O(h^2)$
Trapezoidal	$\frac{h}{2}\left[f(m-\frac{h}{2}) + f(m+\frac{h}{2})\right]$	$O(h^2)$
Simpson	$\frac{h}{6}\left[f(m-\frac{h}{2}) + 4f(m) + f(m+\frac{h}{2})\right]$	$O(h^4)$

We test the three rules on $\int_0^1 e^x\,\mathrm{d}x = e - 1$ with $N = 4, 8, 16, 32$. Since N doubles, h halves, and the observed order is $\log_2$ of the ratio of successive errors.

Step by step

```
def midpoint(f, a, b, n):
    h = (b - a) / n                 # width of a subinterval
    total = 0.0
    for i in range(n):              # subintervals i = 0, ..., n-1
        m = a + (i + 0.5) * h       # midpoint of subinterval i
        total = total + f(m)
    return h * total
```

```
def trapezoidal(f, a, b, n):
    h = (b - a) / n
    total = 0.5 * f(a) + 0.5 * f(b) # the two end points count half
    for i in range(1, n):           # interior points i = 1, ..., n-1
        total = total + f(a + i * h)
    return h * total

def simpson(f, a, b, n):
    return (2 * midpoint(f, a, b, n) + trapezoidal(f, a, b, n)) / 3

exact = np.e - 1                    # integral of exp(x) from 0 to 1
n_list = [4, 8, 16, 32]
error_m = []
error_t = []
error_s = []
for n in n_list:
    error_m.append(abs(midpoint(np.exp, 0, 1, n) - exact))
    error_t.append(abs(trapezoidal(np.exp, 0, 1, n) - exact))
    error_s.append(abs(simpson(np.exp, 0, 1, n) - exact))

print(" N   midpoint  order   trapezoid  order   Simpson   order")
for i in range(1, len(n_list)):
    p_m = np.log2(error_m[i - 1] / error_m[i])  # h halved: order = log2(ratio)
    p_t = np.log2(error_t[i - 1] / error_t[i])
    p_s = np.log2(error_s[i - 1] / error_s[i])
    print(f"{n_list[i]:2d} {error_m[i]:9.2e} {p_m:5.2f}",
          f"{error_t[i]:9.2e} {p_t:5.2f}  {error_s[i]:9.2e} {p_s:5.2f}")
ratio = error_t[-1] / error_m[-1]
print("error trapezoidal / error midpoint for N = 32:", round(ratio, 3))
```

```
 N   midpoint  order   trapezoid  order   Simpson   order
 8  1.12e-03  2.00  2.24e-03  2.00   1.46e-07  4.00
16  2.80e-04  2.00  5.59e-04  2.00   9.10e-09  4.00
32  6.99e-05  2.00  1.40e-04  2.00   5.69e-10  4.00
error trapezoidal / error midpoint for N = 32: 2.0
```

Compact

```
def midpoint(f, a, b, n):
    h = (b - a) / n
    return h * np.sum(f(a + h * (np.arange(n) + 0.5)))   # all midpoints at once

def trapezoidal(f, a, b, n):
    x = np.linspace(a, b, n + 1)                         # all n+1 end points
```

```
    return (b - a) / n * (0.5 * f(x[0]) + np.sum(f(x[1:-1])) + 0.5 * f(x[-1]))

def simpson(f, a, b, n):
    return (2 * midpoint(f, a, b, n) + trapezoidal(f, a, b, n)) / 3

n = np.array([4, 8, 16, 32])
for rule in [midpoint, trapezoidal, simpson]:
    errors = np.array([abs(rule(np.exp, 0, 1, k) - np.e + 1) for k in n])
    print(f"{rule.__name__:12s} error for N = 32: {errors[-1]:.2e},"
          f" observed order: {np.log2(errors[:-1] / errors[1:]).round(2)}")
```

```
midpoint     error for N = 32: 6.99e-05, observed order: [2. 2. 2.]
trapezoidal  error for N = 32: 1.40e-04, observed order: [2. 2. 2.]
simpson      error for N = 32: 5.69e-10, observed order: [4. 4. 4.]
```

The output confirms the orders, and also that the trapezoidal error is twice the midpoint error. A function can be handed to another function like a number: `midpoint(np.exp, 0, 1, n)` integrates `np.exp`, and inside `midpoint` it is called as `f(m)`. `np.log2` is the logarithm to base 2, and `error_t[-1]` is the last element of the list.

3.3.3 Where quadrature enters prognostic models

- **Time stepping.** Integrating $\mathrm{d}u/\mathrm{d}t = f(t,u)$ over one time step gives $u(t+\Delta t) = u(t) + \int_t^{t+\Delta t} f\,\mathrm{d}t$. Every time stepping scheme is a quadrature rule for this integral. Evaluating f only at the start of the step, $\int \approx \Delta t\, f(t,u(t))$ (the left rectangle rule, of first order), gives the forward Euler scheme Equation 1.5; the trapezoidal rule gives a scheme of second order (Chapter 5).
- **Finite volumes.** The fluxes through the faces of a grid cell and the sources inside it are integrals (Chapter 9).
- **Finite elements.** The entries of the system matrix are integrals over elements, evaluated with Gauss quadrature, which chooses the positions of the points as well as the weights (Chapter 19).

3.4 Root finding

Many steps of a model require the solution of a nonlinear equation $f(x) = 0$: every step of an implicit time stepping scheme for a nonlinear problem, the saturation temperature of an air parcel, the water table in a soil column.

As an example we take

$$\tan\lambda + 2\lambda = 0.$$

Equations of this kind determine the eigenvalues of diffusion problems with Robin boundary conditions. The tangent has poles at $\lambda = \pi/2, 3\pi/2, \ldots$, where it jumps from $+\infty$ to $-\infty$; a root finder would take such a sign change for a root. We therefore multiply by $\cos\lambda$ and solve the equivalent smooth equation

$$f(\lambda) = \sin\lambda + 2\lambda\cos\lambda = 0.$$

Its roots are those of the original equation, because at the poles, $\cos\lambda = 0$, the new function equals $\sin\lambda = \pm 1 \neq 0$. Reformulating a problem before solving it numerically is often the most effective step.

The first positive root lies between $\pi/2$ and π: there $f(\pi/2) = \sin(\pi/2) = 1 > 0$ and $f(\pi) = 2\pi\cos\pi = -2\pi < 0$. Newton's method below also needs the derivative, which by the product rule is

$$f'(\lambda) = \cos\lambda + 2\cos\lambda - 2\lambda\sin\lambda = 3\cos\lambda - 2\lambda\sin\lambda.$$

3.4.1 Bisection

If f is continuous and $f(a)$ and $f(b)$ have opposite signs, there is a root in $[a, b]$ (intermediate value theorem). Bisection evaluates f at the midpoint $c = (a + b)/2$. If $f(a)$ and $f(c)$ have opposite signs, the root lies in $[a, c]$, otherwise in $[c, b]$, and the method continues with that half.

After k iterations the interval that contains the root has the length $(b - a)/2^k$. Its midpoint, our estimate, is at most half of this length away from the root:

$$e_k \leq \frac{b - a}{2^{k+1}}.$$

To reach a tolerance τ we need $(b - a)/2^{k+1} \leq \tau$, that is $2^{k+1} \geq (b - a)/\tau$, or after taking the logarithm to base 2,

$$k \geq \log_2 \frac{b - a}{\tau} - 1.$$

For $b - a = \pi/2$ and $\tau = 10^{-12}$ this is $k \geq 40.5 - 1$, that is 40 iterations. Each iteration gains one binary digit, and $\log_2 10 \approx 3.3$ iterations gain one decimal digit. This is **linear convergence**: the error bound is multiplied by a fixed factor, here $\frac{1}{2}$, in every iteration. The method cannot fail, but it is slow, and it does not extend to systems of equations.

3.4.2 Newton–Raphson

Newton's method uses the Taylor series Equation 3.1. At the current estimate x_k we keep the first two terms,

$$f(x_k + \delta) \approx f(x_k) + \delta\, f'(x_k),$$

which is the tangent to f at x_k, and choose δ such that the right hand side is zero: $\delta = -f(x_k)/f'(x_k)$. The next estimate is $x_{k+1} = x_k + \delta$:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}. \tag{3.15}$$

Geometrically, x_{k+1} is the point where the tangent at x_k crosses zero.

How fast does it converge? Let r be the root and $e_k = x_k - r$ the error. We expand f around x_k and evaluate at the root, now keeping the next term in the Lagrange form. The step from x_k to r is $r - x_k = -e_k$:

$$0 = f(r) = f(x_k) - e_k\, f'(x_k) + \frac{e_k^2}{2} f''(\xi),$$

with ξ between x_k and r. Dividing by $f'(x_k)$,

$$0 = \frac{f(x_k)}{f'(x_k)} - e_k + \frac{f''(\xi)}{2f'(x_k)}\, e_k^2,$$

and rearranging,

$$e_k - \frac{f(x_k)}{f'(x_k)} = \frac{f''(\xi)}{2f'(x_k)}\, e_k^2.$$

The left hand side is $x_k - r - f(x_k)/f'(x_k) = x_{k+1} - r = e_{k+1}$ by Equation 3.15. Hence

$$e_{k+1} = \frac{f''(\xi)}{2f'(x_k)}\, e_k^2 \approx \frac{f''(r)}{2f'(r)}\, e_k^2. \tag{3.16}$$

The new error is proportional to the *square* of the old one: **quadratic convergence**. An error of 10^{-3} becomes 10^{-6} and then 10^{-12}; the number of correct digits doubles in every iteration. The argument requires $f'(r) \neq 0$, a simple root. At a double root, where f touches zero without crossing, $f'(r) = 0$ and Newton's method converges only linearly.

The code finds the first root with both methods. It then repeats five Newton steps from $\lambda_0 = 2$ and prints the error and the ratio e_{k+1}/e_k^2, which according to Equation 3.16 should approach $f''(r)/(2f'(r))$, with $f''(\lambda) = -5\sin\lambda - 2\lambda\cos\lambda$.

Step by step

```
def f(lam):
    return np.sin(lam) + 2 * lam * np.cos(lam)

def dfdlam(lam):                    # derivative of f
    return 3 * np.cos(lam) - 2 * lam * np.sin(lam)

def bisection(f, a, b, tol):
    fa = f(a)
    k = 0                           # counts the iterations
    while (b - a) / 2 > tol:        # midpoint not yet within tol of the root
        c = (a + b) / 2             # midpoint
        fc = f(c)
        k = k + 1
        if fa * fc <= 0:            # different signs: the root is in [a, c]
            b = c
        else:                       # same sign: the root is in [c, b]
            a = c
            fa = fc
    return (a + b) / 2, k           # return two values: the root and k

def newton(f, dfdx, x, tol):
    k = 0
    step = 2 * tol                  # any value > tol, so that the loop starts
    while abs(step) > tol and k < 50:   # stop after 50 iterations at most
        step = -f(x) / dfdx(x)      # Newton step
        x = x + step
        k = k + 1
    return x, k

root_b, k_b = bisection(f, np.pi / 2, np.pi, 1e-12)   # receive both values
root_n, k_n = newton(f, dfdlam, 2.0, 1e-12)
print(f"bisection: root = {root_b:.12f} after {k_b} iterations")
print(f"Newton:    root = {root_n:.12f} after {k_n} iterations")

r = root_n                          # the root, as reference
d2f = -5 * np.sin(r) - 2 * r * np.cos(r)   # second derivative f''(r)
C = d2f / (2 * dfdlam(r))
lam = 2.0
error_old = abs(lam - r)
for k in range(1, 5):
    lam = lam - f(lam) / dfdlam(lam)
```

```
    error = abs(lam - r)
    ratio = error / error_old**2
    print(f"iteration {k}: error = {error:.1e},  error / old error^2 = {ratio:.3f}")
    error_old = error
print(f"predicted ratio |f''(r) / (2 f'(r))| = {abs(C):.3f}")
```

```
bisection: root = 1.836597203152 after 40 iterations
Newton:    root = 1.836597203152 after 5 iterations
iteration 1: error = 8.8e-03,  error / old error^2 = 0.330
iteration 2: error = 3.4e-05,  error / old error^2 = 0.438
iteration 3: error = 5.2e-10,  error / old error^2 = 0.445
iteration 4: error = 2.2e-16,  error / old error^2 = 836.237
predicted ratio |f''(r) / (2 f'(r))| = 0.445
```

Compact

```
from scipy.optimize import root_scalar

f = lambda lam: np.sin(lam) + 2 * lam * np.cos(lam)
dfdlam = lambda lam: 3 * np.cos(lam) - 2 * lam * np.sin(lam)

b = root_scalar(f, bracket=[np.pi / 2, np.pi], method="bisect", xtol=1e-12)
n = root_scalar(f, fprime=dfdlam, x0=2.0, method="newton", xtol=1e-12)
print(f"bisection: root = {b.root:.12f} after {b.iterations} iterations")
print(f"Newton:    root = {n.root:.12f} after {n.iterations} iterations")
```

```
bisection: root = 1.836597203152 after 41 iterations
Newton:    root = 1.836597203152 after 5 iterations
```

The errors of the Newton iteration show the squaring predicted by Equation 3.16: the exponent roughly doubles from one iteration to the next, and the ratio e_{k+1}/e_k^2 approaches the predicted value. Once the error reaches the round-off level of about 10^{-16} it cannot decrease further, and the ratio loses its meaning, as in the fourth iteration. SciPy's bisection counts one iteration more than ours because it uses a slightly different stopping test. `while` conditions can be combined with `and`: the loop continues only while both are true. The second condition protects against an endless loop if the iteration does not converge. A function can `return` several values separated by commas; they are received as `root_b, k_b = ...`.

The speed has a price, which is also visible in Equation 3.16. Newton's method needs the derivative. And it needs a starting value that is close enough: the argument assumed that e_k is small, and where $f'(x_k) \approx 0$ the tangent is nearly horizontal and the next estimate is thrown

far away. In practice the two methods are combined: bisection until the interval is small, then Newton. `scipy.optimize.root_scalar` offers such safeguarded methods, for example `method="brentq"`.

3.4.3 Newton's method for systems

Newton's method is important for a further reason: it generalises to systems. For N equations $\vec{F}(\vec{x}) = 0$ in N unknowns, the Taylor series of each component F_i to first order is

$$F_i(\vec{x}_k + \delta\vec{x}) \approx F_i(\vec{x}_k) + \sum_{j=1}^{N} \frac{\partial F_i}{\partial x_j}(\vec{x}_k)\,\delta x_j,$$

or, in matrix form,

$$\vec{F}(\vec{x}_k + \delta\vec{x}) \approx \vec{F}(\vec{x}_k) + J(\vec{x}_k)\,\delta\vec{x}, \qquad J_{ij} = \frac{\partial F_i}{\partial x_j},$$

with the **Jacobian matrix** J in the place of the derivative. Setting the right hand side to zero gives a linear system for the update:

$$J(\vec{x}_k)\,\delta\vec{x} = -\vec{F}(\vec{x}_k), \qquad \vec{x}_{k+1} = \vec{x}_k + \delta\vec{x}. \tag{3.17}$$

Example. The circle $x^2 + y^2 = 4$ and the hyperbola $xy = 1$ intersect in four points. With

$$\vec{F}(x, y) = \begin{pmatrix} x^2 + y^2 - 4 \\ xy - 1 \end{pmatrix}, \qquad J(x, y) = \begin{pmatrix} 2x & 2y \\ y & x \end{pmatrix}$$

and the starting point $(2, 0.5)$, Newton's method finds the intersection near it. The exact value is $x = \sqrt{2 + \sqrt{3}} \approx 1.9319$, $y = 1/x \approx 0.5176$.

Step by step

```
x, y = 2.0, 0.5                     # starting point
for k in range(1, 6):
    F = np.array([x**2 + y**2 - 4,  # the two equations
                  x * y - 1])
    J = np.array([[2 * x, 2 * y],   # the Jacobian matrix, row by row
                  [y,     x    ]])
    delta = np.linalg.solve(J, -F)  # solve J delta = -F
    x = x + delta[0]
```

```
    y = y + delta[1]
    error = abs(x - np.sqrt(2 + np.sqrt(3)))   # distance from the exact x
    print(f"iteration {k}: x = {x:.12f}, y = {y:.12f}, error in x = {error:.1e}")
```

```
iteration 1: x = 1.933333333333, y = 0.516666666667, error in x = 1.5e-03
iteration 2: x = 1.931852741096, y = 0.517637054822, error in x = 1.1e-06
iteration 3: x = 1.931851652579, y = 0.517638090204, error in x = 8.0e-13
iteration 4: x = 1.931851652578, y = 0.517638090205, error in x = 2.2e-16
iteration 5: x = 1.931851652578, y = 0.517638090205, error in x = 0.0e+00
```

Compact

```
from scipy.optimize import root

F = lambda v: [v[0]**2 + v[1]**2 - 4, v[0] * v[1] - 1]
J = lambda v: [[2 * v[0], 2 * v[1]], [v[1], v[0]]]
print(root(F, [2.0, 0.5], jac=J).x)
```

```
[1.93185165 0.51763809]
```

Again the number of correct digits doubles in every iteration, until the round-off level is reached.

A nonlinear problem is thus replaced by a sequence of linear ones. This is how nonlinear PDEs such as the Richards equation are solved (Chapter 17), and it is one reason why the solution of linear systems, the subject of the next unit, is so central.

3.5 Summary

- Finite difference formulas and their truncation errors follow from the Taylor series by combining series so that unwanted terms cancel. The forward and backward differences are of first order, the central differences for the first and second derivative of second order. For any stencil, the weights follow from a linear system.
- The order of a method is measured as the slope of the error in a log–log plot, or from the ratio of the errors for two step sizes. This is the basic verification test.
- Round-off limits the accuracy of a finite difference. The optimal step balances truncation and round-off error and is far larger than the machine epsilon: about $\sqrt{\varepsilon}$ for the forward and $\varepsilon^{1/3}$ for the central difference.

- Linear interpolation and the midpoint and trapezoidal rules are of second order. Polynomials of high degree on equidistant points oscillate (Runge). Combining two rules whose errors have opposite signs gives a rule of higher order, such as Simpson's rule.
- Bisection converges linearly and always. Newton's method converges quadratically near a simple root and generalises to systems, where each iteration solves a linear system with the Jacobian.

3.6 Further reading

LeVeque (2007), chapter 1, for finite difference approximations; Fornberg (1988) for stencils on arbitrary grids.

4 The numerical toolbox II: matrices and waves

Learning goals

After this unit you can

- set up the matrix of a finite difference discretization and describe its structure,
- derive the eigenvalues of the model matrix and its condition number,
- explain what the condition number says about the solution of a linear system,
- decompose a signal on a grid into waves with the discrete Fourier transform,
- explain the shortest resolvable wave, aliasing and the effective resolution of a grid.

Two more tools are needed before we turn to differential equations. Matrices describe what a discretization does to all grid points at once. Waves describe what it does to each spatial scale. The two views are closely related: the eigenvectors of the most important matrix of this lecture are waves.

4.1 From a differential equation to a matrix

4.1.1 A model problem

Consider the one-dimensional Poisson equation with homogeneous Dirichlet conditions,

$$-\frac{\mathrm{d}^2 u}{\mathrm{d}x^2} = f(x), \qquad u(0) = u(L) = 0. \tag{4.1}$$

We divide $[0, L]$ into $N + 1$ intervals of width $h = L/(N + 1)$ with grid points $x_j = jh$. The values $u_0 = u_{N+1} = 0$ are known; the unknowns are $u_1, \ldots, u_N$. At each interior point we replace the second derivative by the central difference Equation 3.6:

$$\frac{-u_{j-1} + 2u_j - u_{j+1}}{h^2} = f_j, \qquad j = 1, \ldots, N.$$

These are N linear equations for N unknowns. We write them out for $N = 4$, using $u_0 = u_5 = 0$ in the first and the last equation:

$$
\begin{aligned}
2u_1 - u_2 &= h^2 f_1 \\
-u_1 + 2u_2 - u_3 &= h^2 f_2 \\
-u_2 + 2u_3 - u_4 &= h^2 f_3 \\
-u_3 + 2u_4 &= h^2 f_4
\end{aligned}
\quad\Longleftrightarrow\quad
\begin{pmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{pmatrix}
\begin{pmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{pmatrix}
= h^2 \begin{pmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{pmatrix}.
$$

In general, discretizing a linear differential equation leads to a linear system

$$A\vec{x} = \vec{b}$$

with as many unknowns as grid points. Here $A = T/h^2$ with the **model matrix**

$$
T = \begin{pmatrix}
2 & -1 & & & \\
-1 & 2 & -1 & & \\
 & \ddots & \ddots & \ddots & \\
 & & -1 & 2 & -1 \\
 & & & -1 & 2
\end{pmatrix}. \tag{4.2}
$$

Row j of the matrix is the stencil of the difference formula, centred on the diagonal. T accompanies us through the units on elliptic and parabolic problems. It has three properties that are typical for matrices from discretizations.

4.1.2 Sparsity

Each row has at most three non-zero entries, because each grid point is coupled only to its neighbours. Of the N^2 entries only $3N - 2$ are non-zero. Such a matrix is **sparse**. Sparse matrices are stored and processed without their zeros, for example with `scipy.sparse`. For a model with 10^8 grid points this is the difference between possible and impossible: the full matrix would have 10^{16} entries.

4.1.3 Symmetry and positive definiteness

T is symmetric, $T = T^T$. It is also **positive definite**: $\vec{x}^T T \vec{x} > 0$ for all $\vec{x} \neq 0$. To see this we write out the quadratic form for $N = 3$,

$$\vec{x}^T T \vec{x} = 2x_1^2 + 2x_2^2 + 2x_3^2 - 2x_1x_2 - 2x_2x_3 = x_1^2 + (x_1 - x_2)^2 + (x_2 - x_3)^2 + x_3^2,$$

a sum of squares that vanishes only if all x_j are zero. The same rearrangement works for every N. Matrices from diffusion problems have this property, and the most efficient solvers rely on it (Chapter 11).

4.1.4 Eigenvalues and eigenvectors

An **eigenvector** $\vec{v}$ of a matrix A is a vector that A only stretches,

$$A\vec{v} = \lambda\vec{v},$$

with the **eigenvalue** λ. For a symmetric matrix all eigenvalues are real and the eigenvectors are orthogonal; for a positive definite matrix all eigenvalues are positive.

The eigenvalues of T can be derived by hand. Since T represents a second derivative, and the second derivative of a sine is a sine, we try a sine wave sampled on the grid,

$$v_j = \sin(j\theta), \qquad j = 1, \dots, N,$$

with a parameter θ to be determined. Row j of $T\vec{v}$ is

$$(T\vec{v})_j = -\sin\big((j-1)\theta\big) + 2\sin(j\theta) - \sin\big((j+1)\theta\big).$$

With the identity $\sin(a-b) + \sin(a+b) = 2\sin a\cos b$ for $a = j\theta$, $b = \theta$ this becomes

$$(T\vec{v})_j = 2\sin(j\theta) - 2\sin(j\theta)\cos\theta = (2 - 2\cos\theta)\, v_j.$$

So $\vec{v}$ is an eigenvector with eigenvalue $2 - 2\cos\theta$, provided the first and last row are also satisfied. These rows lack one neighbour, which is correct only if the missing values vanish: $v_0 = \sin 0 = 0$ holds automatically, and $v_{N+1} = \sin\big((N+1)\theta\big) = 0$ requires $(N+1)\theta = k\pi$. This selects N values of θ:

$$\lambda_k = 2 - 2\cos\frac{k\pi}{N+1} = 4\sin^2\frac{k\pi}{2(N+1)}, \qquad (\vec{v}_k)_j = \sin\frac{jk\pi}{N+1}, \qquad k = 1, \dots, N. \tag{4.3}$$

The eigenvectors are sine waves with k half-waves across the domain (Figure 4.1). The smallest eigenvalue belongs to the longest wave, the largest to the shortest.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.sparse import diags

N = 20
T = diags([-1.0, 2.0, -1.0], [-1, 0, 1], shape=(N, N))
eigenvalues, eigenvectors = np.linalg.eigh(T.toarray())
```

```
fig, ax = plt.subplots(figsize=(6.5, 3))
j = np.arange(1, N + 1)
for index in [0, 1, N - 1]:
    v = eigenvectors[:, index]
    ax.plot(j, v * np.sign(v[0]), "o-", ms=3, label=rf"$k = {index + 1}$, $\lambda =
↪  {eigenvalues[index]:.3f}$")
ax.set_xlabel("grid point $j$")
ax.set_xticks([1, 5, 10, 15, 20])
ax.legend(fontsize=8)
plt.show()
```

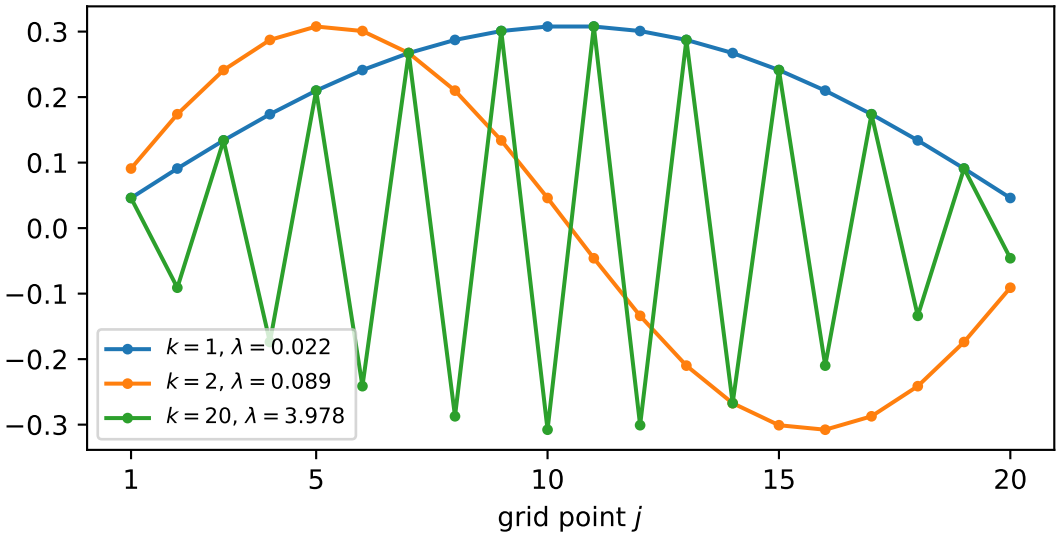


Figure 4.1: Eigenvectors of the model matrix for $N = 20$: the two longest waves and the shortest wave.

4.2 Norms and the condition number

4.2.1 Norms

The size of a vector is measured by a norm, for example the Euclidean norm and the maximum norm,

$$\|\vec{x}\|_2 = \sqrt{\sum_j x_j^2}, \qquad \|\vec{x}\|_\infty = \max_j |x_j|.$$

Norms are what we mean by "the error" of a solution on a grid. The norm of a matrix is the largest factor by which it can stretch a vector,

$$\|A\| = \max_{\vec{x}\neq 0} \frac{\|A\vec{x}\|}{\|\vec{x}\|}, \qquad \text{so that} \qquad \|A\vec{x}\| \le \|A\|\,\|\vec{x}\|. \tag{4.4}$$

For a symmetric matrix and the Euclidean norm, $\|A\|_2$ is the largest eigenvalue in magnitude.

4.2.2 The condition number

How sensitive is the solution of $A\vec{x} = \vec{b}$ to errors in the right hand side? Let $\vec{b}$ be perturbed by $\delta\vec{b}$, and let $\delta\vec{x}$ be the resulting change of the solution. Subtracting $A\vec{x} = \vec{b}$ from $A(\vec{x} + \delta\vec{x}) = \vec{b} + \delta\vec{b}$ gives $A\,\delta\vec{x} = \delta\vec{b}$, hence

$$\delta\vec{x} = A^{-1}\delta\vec{b} \quad \Longrightarrow \quad \|\delta\vec{x}\| \le \|A^{-1}\|\,\|\delta\vec{b}\|,$$

using Equation 4.4. The same inequality applied to $\vec{b} = A\vec{x}$ gives $\|\vec{b}\| \le \|A\|\,\|\vec{x}\|$, or $1/\|\vec{x}\| \le \|A\|/\|\vec{b}\|$. Multiplying the two inequalities,

$$\frac{\|\delta\vec{x}\|}{\|\vec{x}\|} \le \underbrace{\|A\|\,\|A^{-1}\|}_{\kappa(A)} \frac{\|\delta\vec{b}\|}{\|\vec{b}\|}. \tag{4.5}$$

The factor $\kappa(A) = \|A\|\,\|A^{-1}\|$ is the **condition number** of the matrix. It bounds the amplification of relative errors, and is therefore the condition number of Chapter 1 for the problem "solve a linear system". For a symmetric positive definite matrix, $\|A\|_2 = \lambda_{\max}$ and $\|A^{-1}\|_2 = 1/\lambda_{\min}$, so that

$$\kappa(A) = \frac{\lambda_{\max}}{\lambda_{\min}}.$$

A rule of thumb: with $\kappa = 10^p$ about p of the 16 digits are lost.

4.2.3 The condition number of the model matrix

From Equation 4.3 we can compute $\kappa(T)$ for large N. The largest eigenvalue, $k = N$, has $\cos\left(N\pi/(N+1)\right) \approx -1$, so $\lambda_{\max} \approx 4$. For the smallest, $k = 1$, the argument $\theta = \pi/(N+1)$ is small and $\cos\theta \approx 1 - \theta^2/2$:

$$\lambda_{\min} = 2 - 2\cos\frac{\pi}{N+1} \approx \frac{\pi^2}{(N+1)^2} \quad \Longrightarrow \quad \kappa(T) \approx \frac{4(N+1)^2}{\pi^2}. \tag{4.6}$$

```
for N in [10, 100, 1000]:
    T = diags([-1.0, 2.0, -1.0], [-1, 0, 1], shape=(N, N))
    eigenvalues = np.linalg.eigvalsh(T.toarray())
    k = np.arange(1, N + 1)
    exact = 2 - 2 * np.cos(k * np.pi / (N + 1))
    print(f"N = {N:4d}: max. deviation from the formula {abs(eigenvalues -
    ↪  exact).max():.1e},"
          f" condition number {eigenvalues[-1] / eigenvalues[0]:9.1f},"
          f" estimate {4 * (N + 1)**2 / np.pi**2:9.1f},"
          f" non-zeros {T.nnz} of {N * N}")
```

```
N =   10: max. deviation from the formula 4.4e-16, condition number      48.4, estimate
N =  100: max. deviation from the formula 2.7e-15, condition number    4133.6, estimate    4
N = 1000: max. deviation from the formula 3.6e-15, condition number  406095.0, estimate  406
```

The condition number grows as N^2, that is as $1/h^2$. Refining the grid makes the discretization more accurate and the linear system harder to solve. This conflict is the reason why solvers for linear systems take up two units of this lecture (Chapter 10, Chapter 11).

4.3 Fourier analysis

4.3.1 Waves on a grid

A periodic function on a domain of length L can be written as a sum of waves,

$$u(x) = \sum_m \hat{u}_m\, e^{ik_m x}, \qquad k_m = \frac{2\pi m}{L}, \tag{4.7}$$

with wavenumbers k_m, wavelengths $L/|m|$ and complex amplitudes $\hat{u}_m$. The complex exponential combines cosine and sine, $e^{ikx} = \cos kx + i \sin kx$ (Euler's formula), and is used because it is much easier to differentiate and to multiply than the trigonometric functions.

On a grid with N points $x_j = j\,\Delta x$, $j = 0, \ldots, N-1$, and $L = N\Delta x$, only N values u_j are known, so only N amplitudes can be determined. At the grid points $e^{ik_m x_j} = e^{2\pi i\, mj/N}$, and the series becomes

$$u_j = \sum_{m=0}^{N-1} \hat{u}_m\, e^{2\pi i\, mj/N}. \tag{4.8}$$

4.3.2 The discrete Fourier transform

To solve Equation 4.8 for the amplitudes we use the fact that waves on a grid are orthogonal:

$$\sum_{j=0}^{N-1} e^{2\pi i\,(m-n)j/N} = \begin{cases} N & \text{if } m = n, \\ 0 & \text{otherwise.} \end{cases} \tag{4.9}$$

For $m = n$ every term is one. Otherwise the sum is a geometric series $\sum_{j=0}^{N-1} q^j = (1-q^N)/(1-q)$ with $q = e^{2\pi i(m-n)/N} \neq 1$ and $q^N = e^{2\pi i(m-n)} = 1$, so it vanishes.

We multiply Equation 4.8 by $e^{-2\pi i\, nj/N}$ and sum over j. By Equation 4.9 only the term $m = n$ survives on the right hand side, and it equals $N\hat{u}_n$. This gives the **discrete Fourier transform** (DFT):

$$\hat{u}_n = \frac{1}{N} \sum_{j=0}^{N-1} u_j\, e^{-2\pi i\, nj/N}. \tag{4.10}$$

Computed directly, the N sums cost N^2 operations. The **fast Fourier transform** (FFT) obtains the same result with $O(N \log N)$ operations. In NumPy, `np.fft.fft` computes the sum in Equation 4.10 without the factor $1/N$, and `np.fft.rfft` does the same for real data, returning only the amplitudes for $0 \le n \le N/2$. For real data the amplitude of the wave with n periods in the domain is $2|\hat{u}_n|$.

```
N, L = 128, 1.0
x = np.arange(N) * L / N
rng = np.random.default_rng(1)
signal = np.sin(2 * np.pi * 3 * x) + 0.5 * np.cos(2 * np.pi * 10 * x) + 0.2 *
↪  rng.standard_normal(N)

amplitude = 2 * abs(np.fft.rfft(signal)) / N
m = np.fft.rfftfreq(N, d=1 / N)       # number of waves in the domain

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(9, 3))
ax1.plot(x, signal)
```

```
ax1.set_xlabel("$x$")
ax1.set_ylabel("$u$")
ax2.stem(m, amplitude, basefmt=" ", markerfmt=".")
ax2.set_xlim(0, 30)
ax2.set_xlabel("number of waves in the domain, $m$")
ax2.set_ylabel("amplitude")
plt.tight_layout()
plt.show()
```

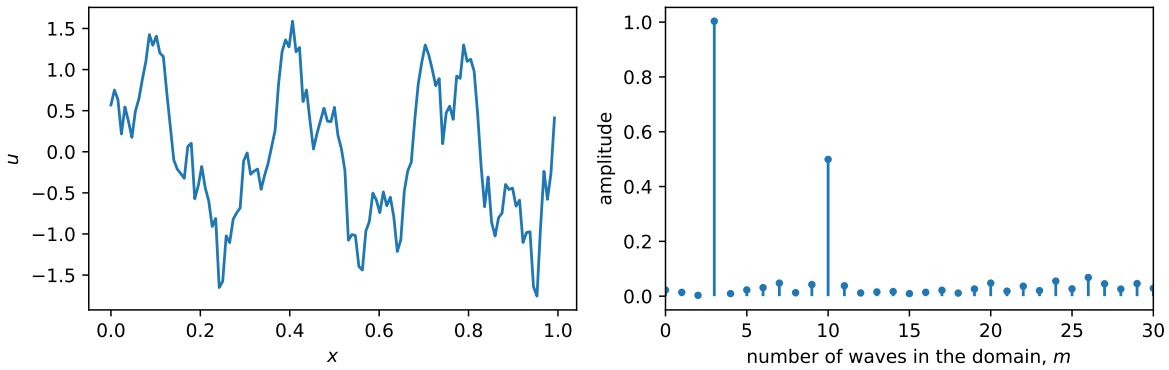


Figure 4.2: A signal composed of two waves and noise (left) and its amplitude spectrum (right).

4.3.3 The shortest wave and aliasing

The shortest wave that a grid can represent has one crest and one trough on two neighbouring points. Its wavelength is $2\Delta x$ and its wavenumber is the **Nyquist wavenumber** $k_{\max} = \pi/\Delta x$.

What happens to a wave with a larger wavenumber? Consider the wavenumbers k and $k + 2\pi/\Delta x$. At the grid points $x_j = j\Delta x$

$$e^{i(k+2\pi/\Delta x)\,x_j} = e^{ikx_j}\,e^{2\pi i\,j} = e^{ikx_j},$$

because $e^{2\pi ij} = 1$ for every integer j. The two waves have the same values at all grid points. On the grid they cannot be distinguished. A wave that is shorter than $2\Delta x$ therefore does not disappear when it is sampled: it is mistaken for a longer wave with a wavenumber in the range $|k| \le k_{\max}$. This is **aliasing**.

In Figure 4.3 a wave with wavelength $\frac{4}{3}\Delta x$, that is $k = \frac{3}{2}\pi/\Delta x$, appears as the wave with $k - 2\pi/\Delta x = -\frac{1}{2}\pi/\Delta x$, which has the wavelength $4\Delta x$.

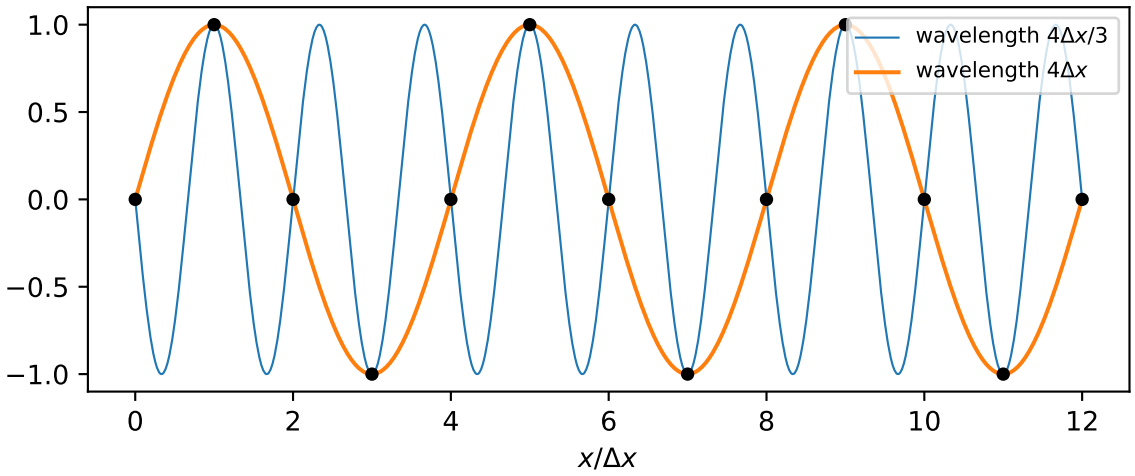


Figure 4.3: Aliasing. At the grid points (dots) a wave with wavelength $4\Delta x/3$ cannot be distinguished from a wave with wavelength $4\Delta x$.

Aliasing matters in nonlinear models. The product of two waves is

$$e^{ik_1 x}\, e^{ik_2 x} = e^{i(k_1+k_2)x},$$

a wave with the wavenumber $k_1 + k_2$. A nonlinear term such as $u\,\partial u/\partial x$ therefore creates shorter waves from longer ones. If $k_1 + k_2$ exceeds $k_{\max}$, the grid misrepresents the product as a longer wave, and energy appears at scales where it does not belong. This can make a model unstable (Chapter 17).

4.3.4 Why waves are the natural test for a numerical method

For a linear equation with constant coefficients each wave evolves independently of the others. It therefore suffices to understand what a numerical method does to a single wave e^{ikx}, and that is easy to compute.

The exact derivative of a wave is

$$\frac{\mathrm{d}}{\mathrm{d}x} e^{ikx} = ik\, e^{ikx}\ :$$

the wave is multiplied by ik. For the central difference Equation 3.5 we factor out e^{ikx} and use $e^{i\alpha} - e^{-i\alpha} = 2i\sin\alpha$:

$$\frac{e^{ik(x+\Delta x)} - e^{ik(x-\Delta x)}}{2\Delta x} = \frac{e^{ik\Delta x} - e^{-ik\Delta x}}{2\Delta x}\, e^{ikx} = i\,\frac{\sin(k\Delta x)}{\Delta x}\, e^{ikx}.$$

The wave is again multiplied by i times a real number. The numerical derivative is that of a wave with the **modified wavenumber**

$$k_{\text{eff}} = \frac{\sin(k\Delta x)}{\Delta x}. \tag{4.11}$$

For $k\Delta x \ll 1$ the expansion $\sin(k\Delta x) \approx k\Delta x - (k\Delta x)^3/6$ gives $k_{\text{eff}} \approx k\,[1 - (k\Delta x)^2/6]$: the relative error is $(k\Delta x)^2/6$. This is the truncation error of Equation 3.5, now expressed for each wave separately.

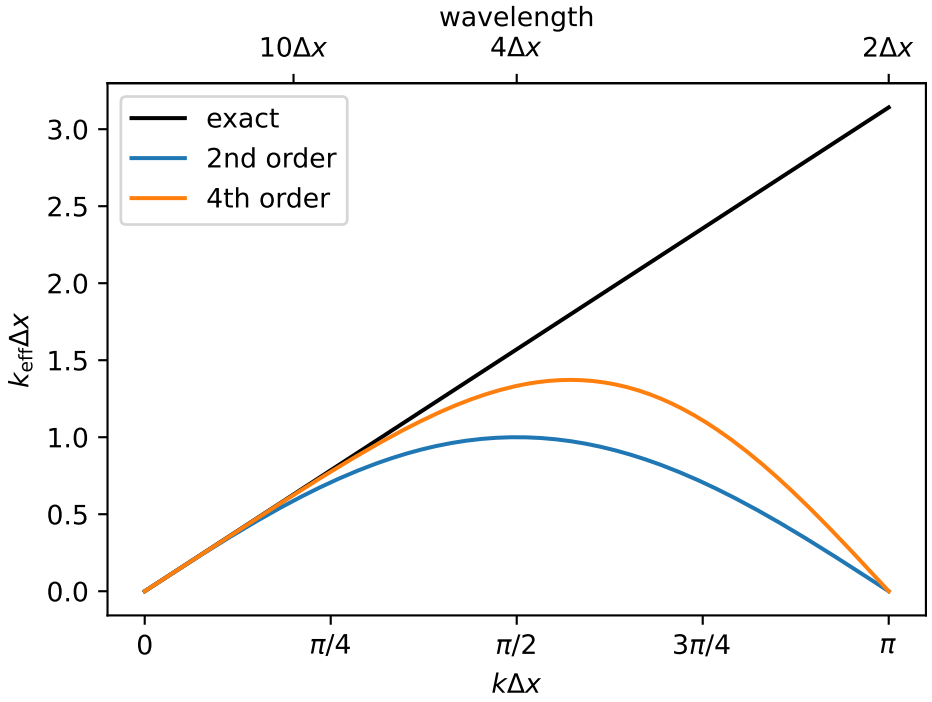


Figure 4.4: Modified wavenumber of the central differences of second and fourth order. The exact derivative corresponds to the diagonal.

Figure 4.4 shows what the statement "second order" hides. For long waves the approximation is excellent. For the $4\Delta x$ wave, $k\Delta x = \pi/2$, the derivative is too small by the factor $\sin(\pi/2)/(\pi/2) = 0.64$. For the $2\Delta x$ wave it is zero: the central difference does not see the shortest wave at all. A wave needs about ten grid points per wavelength to be treated accurately by a scheme of second order; then $k\Delta x = \pi/5$ and the error is 6 %.

This has a consequence that every user of model output should know. The smallest features that a model represents correctly are several times larger than its grid spacing. The **effective resolution** of a weather model with a grid spacing of 2 km is closer to 10 km.

4.3.5 The second difference and the model matrix

The same calculation for the second difference Equation 3.6, now with $e^{i\alpha} + e^{-i\alpha} = 2\cos\alpha$, gives

$$\frac{e^{ik(x+\Delta x)} - 2e^{ikx} + e^{ik(x-\Delta x)}}{\Delta x^2} = -\frac{2 - 2\cos(k\Delta x)}{\Delta x^2}\, e^{ikx} = -\frac{4}{\Delta x^2}\sin^2\frac{k\Delta x}{2}\, e^{ikx}.$$

Compare this with the eigenvalues Equation 4.3 of the model matrix: they are the same expression, with $k\Delta x = k\pi/(N+1)$. The wave analysis and the eigenvalue analysis are two views of one fact. Waves are the eigenvectors of difference operators with constant coefficients, and the factor by which a wave is multiplied is the eigenvalue.

Applied to a complete scheme in space and time, this analysis shows whether a wave is damped, amplified or moved at the wrong speed. This is the von Neumann stability analysis (Chapter 12, Chapter 13).

4.4 Summary

- Discretizing a linear differential equation gives a linear system. Each row of the matrix is the stencil of the difference formula.
- Matrices from discretizations are sparse. Those from diffusion problems are symmetric and positive definite.
- The eigenvectors of the model matrix are sine waves, with eigenvalues $2-2\cos(k\pi/(N+1))$.
- The condition number bounds the amplification of relative errors in the solution of a linear system. For the model matrix it grows as $1/h^2$.
- A grid represents waves down to a wavelength of $2\Delta x$, but treats them accurately only above about $10\Delta x$. Shorter waves are aliased.
- A difference formula multiplies each wave by a factor. Comparing it with the exact factor shows the error scale by scale.

4.5 Further reading

Durran (2010), chapter 3, for the wave view of finite differences; Shewchuk (1994) for eigenvalues, quadratic forms and the condition number.

Part II

Time integration of ODEs

5 One-step methods

Learning goals

After this unit you can

- read the qualitative behaviour of an ODE from its direction field,
- derive the forward and backward Euler methods in three ways and implement them,
- derive the local truncation error of a method and explain how it determines the global error,
- derive the order conditions of two-stage Runge–Kutta methods and use the classical method of fourth order,
- compare methods by the accuracy they reach for a given amount of work,
- explain how adaptive methods choose the time step.

Every prognostic model steps forward in time. After the spatial derivatives of a PDE have been discretized, what remains is a large system of ordinary differential equations in time. The methods of this unit are therefore the time stepping schemes of weather, ocean and groundwater models, studied here on problems small enough to be understood completely.

5.1 The initial value problem

We consider the initial value problem

$$\frac{\mathrm{d}u}{\mathrm{d}t} = f(t, u), \qquad u(t_0) = u_0. \tag{5.1}$$

The function f is the **tendency**: it gives the rate of change of the state for any time and any state. In much of the literature on ODEs the unknown is called y and the step h; we keep u and Δt.

The problem is well-posed (Chapter 2) if f is continuous and does not change too rapidly with u. Precisely, f must satisfy a **Lipschitz condition**: there is a constant L such that

$$|f(t, a) - f(t, b)| \le L\,|a - b| \tag{5.2}$$

for all states a, b in the region of interest. If f is differentiable, L is the largest value of $|\partial f/\partial u|$. The constant L has the unit of one over time: $1/L$ is the shortest time scale on which the tendency responds to a change of the state. It reappears in the error analysis below.

5.1.1 Direction fields

Before solving Equation 5.1 it is useful to understand how its solutions behave. At every point of the (t, u) plane the equation prescribes the slope of the solution passing through it, namely $f(t, u)$. Drawing a short line segment with this slope at many points gives the **direction field**. Solutions are the curves that follow the segments.

As an example consider

$$\frac{\mathrm{d}u}{\mathrm{d}t} = \frac{3 - u}{2}. \tag{5.3}$$

```
import numpy as np
import matplotlib.pyplot as plt

f = lambda t, u: (3 - u) / 2

t, u = np.meshgrid(np.linspace(0, 8, 21), np.linspace(-1, 6, 19))
slope = f(t, u)
norm = np.sqrt(1 + slope**2)

fig, ax = plt.subplots(figsize=(6.5, 3.8))
ax.quiver(t, u, 1 / norm, slope / norm, angles="xy", headwidth=1, headlength=0,
          pivot="mid", color="0.5", scale=35)
t_fine = np.linspace(0, 8, 200)
for u0 in [-1, 1.5, 6]:
    ax.plot(t_fine, 3 + (u0 - 3) * np.exp(-t_fine / 2))
ax.set_xlabel("$t$")
ax.set_ylabel("$u$")
plt.show()
```

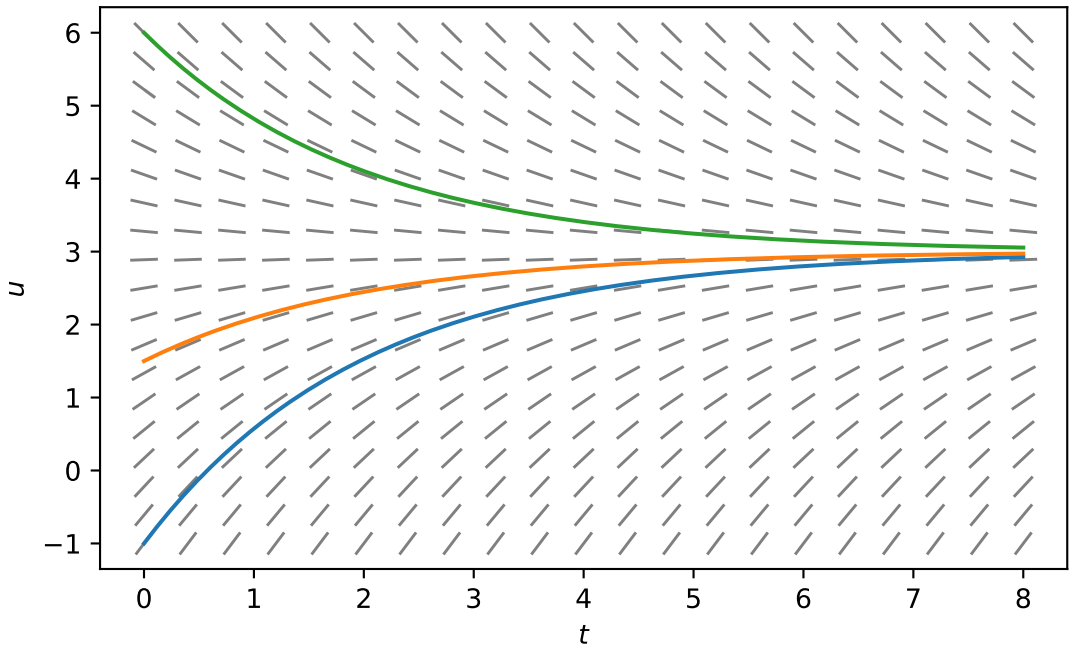


Figure 5.1: Direction field of $\mathrm{d}u/\mathrm{d}t = (3-u)/2$ with three exact solutions. All solutions approach the equilibrium $u = 3$.

Figure 5.1 shows, without any calculation, that $u = 3$ is an equilibrium, since the slope is zero there, and that it is stable: above it the slope is negative, below it positive.

5.1.2 An exact solution

Equation 5.3 is **separable**: all terms with u can be collected on one side and all terms with t on the other,

$$\frac{\mathrm{d}u}{3-u} = \frac{\mathrm{d}t}{2}.$$

Integrating both sides,

$$-\ln|3-u| = \frac{t}{2} + C \quad \Longrightarrow \quad 3-u = \tilde{C}\, e^{-t/2},$$

where $\tilde{C} = \pm e^{-C}$. The initial condition $u(0) = u_0$ gives $\tilde{C} = 3 - u_0$, and therefore

$$u(t) = 3 + (u_0 - 3)\, e^{-t/2}. \tag{5.4}$$

The distance from the equilibrium decays exponentially with the time scale 2. With $u - 3$ as the unknown this is the decay equation Equation 1.4 with $k = \frac{1}{2}$, the equation of radioactive decay with decay constant k.

Exact solutions exist only for special tendencies. For the rest we need numerical methods. Exact solutions remain indispensable, because they are what numerical methods are verified against.

5.2 The forward Euler method

We look for approximations u_n of the solution at the times $t_n = t_0 + n\,\Delta t$. The simplest method can be derived in three ways. Each of them leads to its own family of better methods.

5.2.1 First view: following the tangent

At t_0 we know the solution, u_0, and its slope, $f(t_0, u_0)$. The tangent to the solution at this point is

$$u = u_0 + f(t_0, u_0)\,(t - t_0).$$

If t_1 is close to t_0, the tangent stays close to the solution, and we take its value at t_1 as the approximation: $u_1 = u_0 + \Delta t\, f(t_0, u_0)$. At t_1 we do not know the solution, but we have u_1, and we repeat the step with the slope $f(t_1, u_1)$. In general

$$u_{n+1} = u_n + \Delta t\, f(t_n, u_n). \tag{5.5}$$

This is the **forward Euler method**. In the direction field it moves along one line segment per step (Figure 5.2).

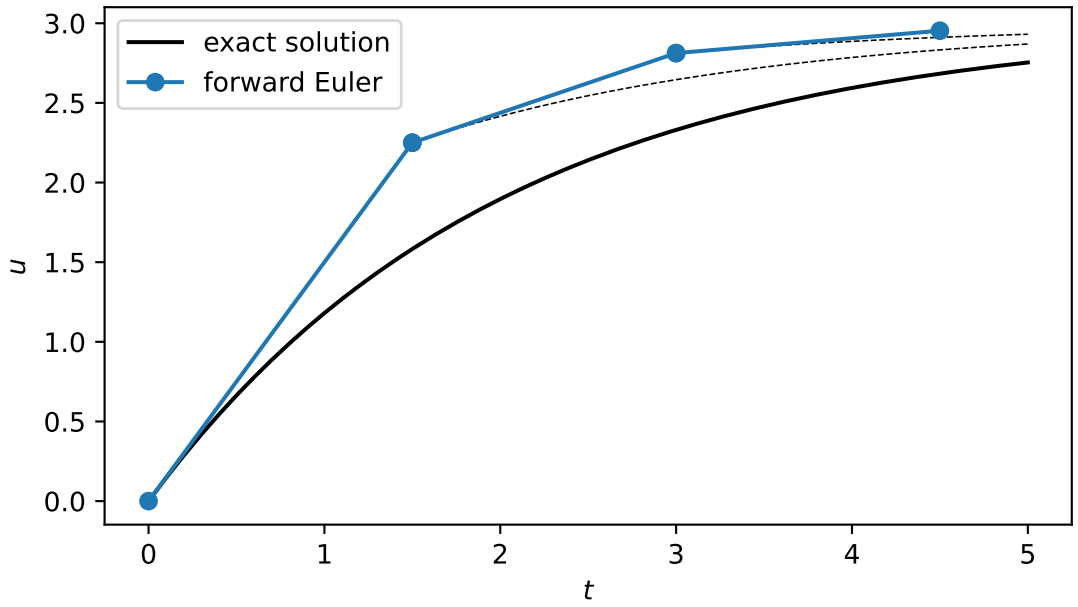


Figure 5.2: Three steps of the forward Euler method for $\mathrm{d}u/\mathrm{d}t = (3-u)/2$ with $\Delta t = 1.5$. Each step follows the tangent of the solution through the current point (dashed), not the solution that started at u_0.

5.2.2 Second view: replacing the derivative

We replace the derivative in Equation 5.1 at time t_n by the forward difference Equation 3.3,

$$\frac{u_{n+1} - u_n}{\Delta t} = f(t_n, u_n),$$

and solve for u_{n+1}. This gives Equation 5.5 again, and explains the name. It also tells us the accuracy to expect: the forward difference is of first order.

5.2.3 Third view: integrating the tendency

Integrating Equation 5.1 from t_n to t_{n+1} gives, without any approximation,

$$u(t_{n+1}) = u(t_n) + \int_{t_n}^{t_{n+1}} f\big(t, u(t)\big)\,\mathrm{d}t. \tag{5.6}$$

The integral cannot be evaluated, because the integrand contains the unknown solution. The forward Euler method approximates it by a rectangle whose height is the integrand at the left

end, $\Delta t\, f(t_n, u_n)$. Better time stepping schemes are better quadrature rules for this integral (Chapter 3).

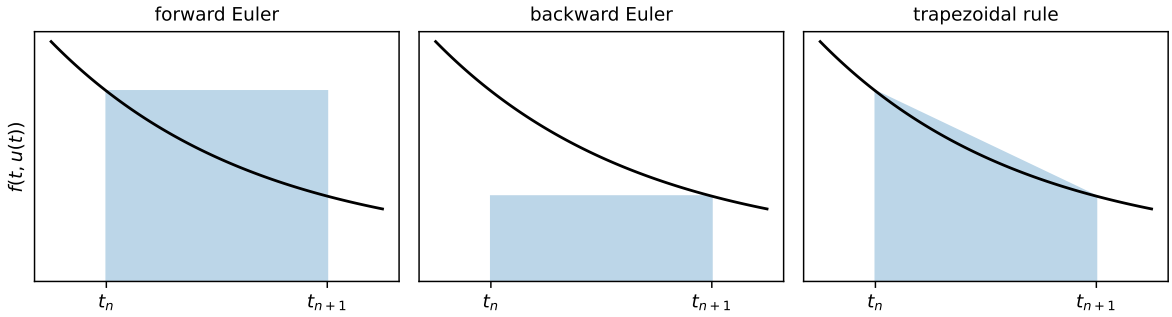


Figure 5.3: The tendency integrated over one time step. Forward Euler uses the value at the start of the step (left), backward Euler the value at the end (middle), the trapezoidal rule the mean of both (right).

5.2.4 Implementation

```
def euler_forward(f, u0, t_end, dt):
    n_steps = int(round(t_end / dt))
    t = dt * np.arange(n_steps + 1)
    u = np.empty(n_steps + 1)
    u[0] = u0
    for n in range(n_steps):
        u[n + 1] = u[n] + dt * f(t[n], u[n])
    return t, u

f = lambda t, u: (3 - u) / 2
for dt in [1.0, 0.5, 0.25]:
    t, u = euler_forward(f, 0.0, 4.0, dt)
    exact = 3 - 3 * np.exp(-2)
    print(f"dt = {dt:4.2f}: u(4) = {u[-1]:.5f}, error = {abs(u[-1] - exact):.5f}")
```

```
dt = 1.00: u(4) = 2.81250, error = 0.21851
dt = 0.50: u(4) = 2.69966, error = 0.10567
dt = 0.25: u(4) = 2.64580, error = 0.05180
```

Halving the time step roughly halves the error, as expected for a method of first order.

5.3 The backward Euler method

In the second view we can equally well use the backward difference Equation 3.4 at the new time t_{n+1},

$$\frac{u_{n+1} - u_n}{\Delta t} = f(t_{n+1}, u_{n+1}),$$

which in the third view is the rectangle with the height of the integrand at the right end (Figure 5.3). This is the **backward Euler method**

$$u_{n+1} = u_n + \Delta t\, f(t_{n+1}, u_{n+1}). \tag{5.7}$$

The unknown u_{n+1} now appears on both sides, inside the tendency. The formula does not give u_{n+1} explicitly; it is an equation that defines u_{n+1} **implicitly** and has to be solved in every time step. Methods such as Equation 5.5, which compute the new value directly from known values, are called **explicit**.

Linear tendency. For the decay equation, $f = -ku$, the equation is linear in u_{n+1} and can be solved by hand:

$$u_{n+1} = u_n - k\,\Delta t\, u_{n+1} \quad \Longrightarrow \quad u_{n+1} = \frac{u_n}{1 + k\,\Delta t}. \tag{5.8}$$

Compare this with the forward Euler result $u_{n+1} = (1 - k\Delta t)u_n$ from Equation 1.6. The factor $1/(1 + k\Delta t)$ lies between 0 and 1 for every time step: the backward Euler solution decays monotonically however large Δt is, where the forward Euler solution oscillated and grew for $k\Delta t > 2$. This is the reason to accept the extra work of an implicit method, and the subject of Chapter 6.

Nonlinear tendency. In general we write Equation 5.7 as a root finding problem for the unknown $U = u_{n+1}$,

$$G(U) = U - u_n - \Delta t\, f(t_{n+1}, U) = 0,$$

and solve it with Newton's method Equation 3.15. The derivative is $G'(U) = 1 - \Delta t\, \partial f/\partial u$, so that one Newton iteration reads

$$U^{(m+1)} = U^{(m)} - \frac{U^{(m)} - u_n - \Delta t\, f\big(t_{n+1}, U^{(m)}\big)}{1 - \Delta t\, \dfrac{\partial f}{\partial u}\big(t_{n+1}, U^{(m)}\big)},$$

started from $U^{(0)} = u_n$. Because u_n is already close to u_{n+1}, two or three iterations usually suffice. For a system of N equations the division becomes the solution of a linear system with the matrix $I - \Delta t\, J$, where J is the Jacobian of the tendency (Equation 3.17). An implicit time step of a PDE model therefore costs at least one solution of a large linear system.

```
def euler_backward(f, dfdu, u0, t_end, dt, newton_iterations=4):
    n_steps = int(round(t_end / dt))
    t = dt * np.arange(n_steps + 1)
    u = np.empty(n_steps + 1)
    u[0] = u0
    for n in range(n_steps):
        U = u[n]
        for _ in range(newton_iterations):
            G = U - u[n] - dt * f(t[n + 1], U)
            U = U - G / (1 - dt * dfdu(t[n + 1], U))
        u[n + 1] = U
    return t, u

# nonlinear test problem du/dt = -u^2, u(0) = 1, exact solution 1 / (1 + t)
f_nl = lambda t, u: -u**2
dfdu_nl = lambda t, u: -2 * u
for dt in [0.5, 0.25, 0.125]:
    t, u = euler_backward(f_nl, dfdu_nl, 1.0, 2.0, dt)
    print(f"dt = {dt:5.3f}: error at t = 2 is {abs(u[-1] - 1 / 3):.5f}")
```

```
dt = 0.500: error at t = 2 is 0.05425
dt = 0.250: error at t = 2 is 0.02867
dt = 0.125: error at t = 2 is 0.01477
```

The backward Euler method is also of first order.

5.4 Errors

5.4.1 Local and global error

Two errors must be distinguished.

The **global error** is what we care about: the difference between the exact and the numerical solution at time t_n,

$$E_n = u(t_n) - u_n.$$

The **local truncation error** τ_n is the error made in a single step that starts from the exact solution. It measures how well the exact solution satisfies the numerical formula. In Figure 5.2 it is the distance, after one step, between an Euler segment and the dashed solution through its starting point.

In addition there is the round-off error of each step. For the time steps used in practice it is much smaller than the truncation error, and we neglect it here.

5.4.2 Local truncation error of the forward Euler method

We expand the exact solution in a Taylor series around t_n,

$$u(t_{n+1}) = u(t_n) + \Delta t\, u'(t_n) + \frac{\Delta t^2}{2} u''(\xi_n),$$

where the last term is the remainder, with ξ_n somewhere between t_n and t_{n+1}. Since the exact solution satisfies the differential equation, $u'(t_n) = f\big(t_n, u(t_n)\big)$, and

$$u(t_{n+1}) = \underbrace{u(t_n) + \Delta t\, f\big(t_n, u(t_n)\big)}_{\text{one Euler step from the exact solution}} + \underbrace{\frac{\Delta t^2}{2} u''(\xi_n)}_{\tau_n}. \tag{5.9}$$

The local truncation error of the forward Euler method is proportional to Δt^2 and to the curvature of the solution. If $|u''| \le M$ on the interval of interest, then $|\tau_n| \le M\Delta t^2/2$.

The second derivative can be expressed by the tendency. Differentiating $u' = f(t,u)$ with the chain rule,

$$u'' = \frac{\partial f}{\partial t} + \frac{\partial f}{\partial u}\, u' = \frac{\partial f}{\partial t} + \frac{\partial f}{\partial u}\, f. \tag{5.10}$$

5.4.3 From the local to the global error

To reach a fixed time T we need $n = T/\Delta t$ steps. If the local errors simply added up, the global error would be about $n \cdot M\Delta t^2/2 = TM\Delta t/2$: one power of Δt is lost, and the method is of first order. This argument is incomplete, because an error made in one step is carried along and changed by all later steps. The complete argument takes three steps.

Step 1: a recursion for the error. We subtract the Euler formula Equation 5.5 from Equation 5.9:

$$\underbrace{u(t_{n+1}) - u_{n+1}}_{E_{n+1}} = \underbrace{u(t_n) - u_n}_{E_n} + \Delta t \left[f(t_n, u(t_n)) - f(t_n, u_n) \right] + \tau_n.$$

The bracket is the difference of the tendency at the exact and at the numerical state. By the Lipschitz condition Equation 5.2 it is at most $L\,|E_n|$ in magnitude. Hence

$$|E_{n+1}| \le (1 + L\Delta t)\,|E_n| + \frac{M\Delta t^2}{2}. \tag{5.11}$$

Each step multiplies the existing error by at most $1 + L\Delta t$ and adds a new local error.

Step 2: solving the recursion. We start without error, $E_0 = 0$, and apply Equation 5.11 repeatedly, writing $q = 1 + L\Delta t$ and $\tau = M\Delta t^2/2$:

$$|E_1| \le \tau, \qquad |E_2| \le q\tau + \tau, \qquad |E_3| \le q^2\tau + q\tau + \tau, \qquad \dots$$

After n steps the bound is a geometric series,

$$|E_n| \le \tau \sum_{j=0}^{n-1} q^j = \tau\,\frac{q^n - 1}{q - 1} = \frac{M\Delta t^2}{2} \cdot \frac{(1 + L\Delta t)^n - 1}{L\Delta t}.$$

Step 3: simplifying. Since $1 + x \le e^x$ for all x, we have $(1 + L\Delta t)^n \le e^{nL\Delta t} = e^{Lt_n}$ with $t_n = n\Delta t$ measured from $t_0 = 0$. One factor Δt cancels:

$$|E_n| \le \frac{M}{2L}\left(e^{Lt_n} - 1\right)\Delta t. \tag{5.12}$$

This is the result. At any fixed time the global error of the forward Euler method is bounded by a constant times Δt: the method **converges**, with **order one**. The rule that came out of the incomplete argument holds in general:

> A local truncation error of order Δt^{p+1} gives a global error of order Δt^p. The order of a method is p.

The constant in Equation 5.12 deserves a comment. It grows exponentially with Lt_n. For a problem whose solutions diverge from each other, this is realistic: errors grow as fast as neighbouring solutions separate, which is the predictability problem of Chapter 8. For a problem whose solutions converge, such as Equation 5.3, the bound is far too pessimistic, because the Lipschitz condition ignores the sign of $\partial f/\partial u$. Errors made early are then damped instead of amplified.

5.5 Methods of higher order

5.5.1 Heun's method

In the third view a better method needs a better quadrature rule for Equation 5.6. The trapezoidal rule Equation 3.14 uses the mean of the integrand at both ends of the step:

$$u_{n+1} = u_n + \frac{\Delta t}{2}\left[f(t_n, u_n) + f(t_{n+1}, u_{n+1})\right]. \tag{5.13}$$

This is the implicit **trapezoidal method**. To avoid the implicit equation, we replace the unknown u_{n+1} inside the tendency by a forward Euler estimate, the **predictor**, and then apply the trapezoidal formula as **corrector**:

$$\begin{aligned} k_1 &= f(t_n, u_n), \\ k_2 &= f(t_n + \Delta t,\ u_n + \Delta t\, k_1), \\ u_{n+1} &= u_n + \frac{\Delta t}{2}(k_1 + k_2). \end{aligned} \tag{5.14}$$

This is **Heun's method**, also called the improved Euler method. It is explicit and needs two evaluations of the tendency per step, two **stages**.

5.5.2 The order of two-stage methods

Is Heun's method really of second order, although the predictor is only of first order? We check this for a whole family of methods at once. Consider

$$k_1 = f(u_n), \qquad k_2 = f(u_n + c\,\Delta t\, k_1), \qquad u_{n+1} = u_n + \Delta t\,(b_1 k_1 + b_2 k_2), \tag{5.15}$$

with parameters b_1, b_2, c. To keep the algebra short we take a tendency that depends only on the state, $f = f(u)$, and write f and f' for $f(u_n)$ and its derivative with respect to u. The result holds in general.

The exact solution. Starting from u_n, the Taylor series is $u(t_n + \Delta t) = u_n + \Delta t\, u' + \frac{1}{2}\Delta t^2 u'' + O(\Delta t^3)$. With $u' = f$ and, from Equation 5.10, $u'' = f'f$:

$$u(t_n + \Delta t) = u_n + \Delta t\, f + \frac{\Delta t^2}{2} f' f + O(\Delta t^3).$$

The numerical solution. We expand the second stage around u_n:

$$k_2 = f(u_n + c\,\Delta t\, f) = f + c\,\Delta t\, f' f + O(\Delta t^2).$$

Inserting into Equation 5.15,

$$u_{n+1} = u_n + \Delta t\,(b_1 + b_2)\, f + \Delta t^2\, b_2 c\, f' f + O(\Delta t^3).$$

Comparison. The two expansions agree up to the terms with Δt^2, leaving a local truncation error of $O(\Delta t^3)$ and hence a method of second order, if

$$b_1 + b_2 = 1 \qquad \text{and} \qquad b_2\, c = \frac{1}{2}. \tag{5.16}$$

These are the **order conditions**. They are two equations for three parameters, so there is a family of second-order methods with two stages.

Method	c	b_1	b_2	Quadrature rule
Heun	1	$\frac{1}{2}$	$\frac{1}{2}$	trapezoidal
Midpoint	$\frac{1}{2}$	0	1	midpoint

The **midpoint method** takes half an Euler step and uses the tendency there for the full step: $u_{n+1} = u_n + \Delta t\, f(t_n + \frac{1}{2}\Delta t,\ u_n + \frac{1}{2}\Delta t\, k_1)$. Forward Euler is the member of the family Equation 5.15 with $b_2 = 0$; it violates the second condition and is of first order.

5.5.3 Runge–Kutta methods

Methods of the form Equation 5.15, with any number of stages, are **Runge–Kutta methods**. The most widely used one has four stages:

$$\begin{aligned} k_1 &= f(t_n, u_n), \\ k_2 &= f(t_n + \tfrac{1}{2}\Delta t,\ u_n + \tfrac{1}{2}\Delta t\, k_1), \\ k_3 &= f(t_n + \tfrac{1}{2}\Delta t,\ u_n + \tfrac{1}{2}\Delta t\, k_2), \\ k_4 &= f(t_n + \Delta t,\ u_n + \Delta t\, k_3), \\ u_{n+1} &= u_n + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4). \end{aligned} \tag{5.17}$$

This **classical Runge–Kutta method** (RK4) is of fourth order. Its order conditions are derived as above, by matching the Taylor series up to Δt^4; there are eight of them. If f depends only on t, the stages are the integrand at the left end, twice at the midpoint and at the right end, with the weights $\frac{1}{6}, \frac{4}{6}, \frac{1}{6}$: Equation 5.17 then is Simpson's rule.

Up to order four, a method of order p needs p stages. Beyond that more stages than the order are needed, which is one reason why RK4 is so popular.

5.5.4 Verification

We verify the orders on the nonlinear test problem $\mathrm{d}u/\mathrm{d}t = -u^2$, $u(0) = 1$, with the exact solution $u = 1/(1+t)$. Each method becomes a function that makes one step: it receives the tendency `f`, the time `t`, the state `u` and the time step `dt` and returns the state at `t + dt`. A second function, `integrate`, repeats such steps up to the end time. The method is handed to it as an argument, so the same loop serves all three methods.

```
def euler_forward_step(f, t, u, dt):
    return u + dt * f(t, u)

def heun_step(f, t, u, dt):
    k1 = f(t, u)                    # slope at the start of the step
    k2 = f(t + dt, u + dt * k1)     # slope at the end, after an Euler step
    return u + dt * (k1 + k2) / 2   # step with the mean of the two slopes

def rk4_step(f, t, u, dt):
    k1 = f(t, u)
    k2 = f(t + dt / 2, u + dt / 2 * k1)
    k3 = f(t + dt / 2, u + dt / 2 * k2)
    k4 = f(t + dt, u + dt * k3)
    return u + dt / 6 * (k1 + 2 * k2 + 2 * k3 + k4)

def integrate(step, f, u0, t_end, dt):
    # integrate from t = 0 to t_end with the one-step method "step"
    n_steps = round(t_end / dt)     # number of steps, computed once
    t_values = [0.0]                # lists of the times and states
    u_values = [u0]
    u = u0
    for n in range(n_steps):
        u = step(f, n * dt, u, dt)  # one step from t = n dt to (n + 1) dt
        t_values.append((n + 1) * dt)
        u_values.append(u)
    return np.array(t_values), np.array(u_values)   # lists -> arrays
```

As in Exercise 1, the time is computed from the step counter `n` instead of adding `dt` repeatedly. The functions work unchanged if the state `u` is a NumPy array instead of a number, because `+` and `*` act on each element of an array.

For the convergence test we integrate to $t = 2$, where the exact solution is $u = 1/3$, with $N = 4, 8, \ldots, 128$ steps.

```
def f_nl(t, u):
    return -u**2

n_list = [4, 8, 16, 32, 64, 128]   # numbers of steps
dt_list = []
for n in n_list:
    dt_list.append(2.0 / n)

def errors_at_t2(step):
    # errors at t = 2 of the method "step" for all time steps in dt_list
    errors = []
    for dt in dt_list:
        t, u = integrate(step, f_nl, 1.0, 2.0, dt)
        errors.append(abs(u[-1] - 1 / 3))   # u[-1] is the last value, at t = 2
    return errors

error_euler = errors_at_t2(euler_forward_step)
error_heun = errors_at_t2(heun_step)
error_rk4 = errors_at_t2(rk4_step)

print("    dt   order Euler   order Heun   order RK4")
for i in range(1, len(dt_list)):    # dt is halved: order = log2(error ratio)
    p_euler = np.log2(error_euler[i - 1] / error_euler[i])
    p_heun = np.log2(error_heun[i - 1] / error_heun[i])
    p_rk4 = np.log2(error_rk4[i - 1] / error_rk4[i])
    print(f"{dt_list[i]:6.4f}  {p_euler:10.2f}  {p_heun:11.2f}  {p_rk4:10.2f}")

fig, ax = plt.subplots(figsize=(6, 3.8))
ax.loglog(dt_list, error_euler, "o-", label="forward Euler")
ax.loglog(dt_list, error_heun, "o-", label="Heun")
ax.loglog(dt_list, error_rk4, "o-", label="RK4")
ax.set_xlabel(r"$\Delta t$")
ax.set_ylabel("error at $t = 2$")
ax.legend()
plt.show()
```

```
    dt   order Euler   order Heun   order RK4
0.2500         1.19         2.06        2.00
0.1250         1.06         2.09        3.82
0.0625         1.03         2.06        3.97
0.0312         1.01         2.03        3.99
0.0156         1.01         2.01        4.00
```

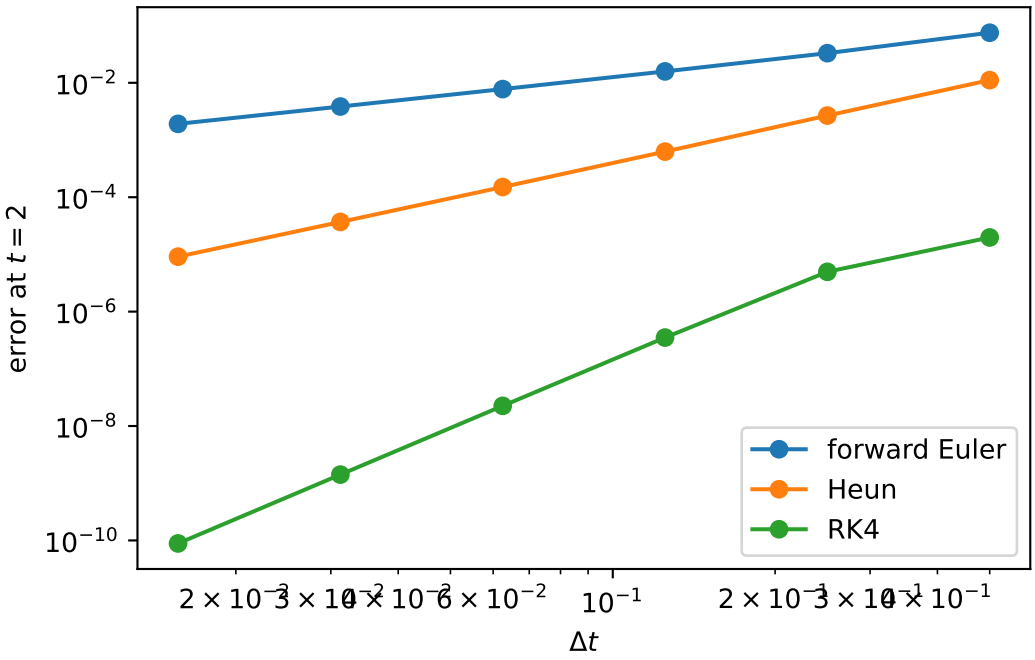


Figure 5.4: Global error at $t = 2$ as a function of the time step for $\mathrm{d}u/\mathrm{d}t = -u^2$. The slopes are the orders of the methods.

5.5.5 Accuracy for a given amount of work

A step of RK4 costs four evaluations of the tendency, a step of forward Euler one. In a weather model one evaluation of the tendency is the entire computation of all spatial derivatives and all physical processes, so the fair comparison is the error reached for a given number of evaluations. With N steps, forward Euler needs N evaluations, Heun $2N$ and RK4 $4N$.

```
evaluations_euler = []
evaluations_heun = []
evaluations_rk4 = []
for n in n_list:
    evaluations_euler.append(1 * n)
    evaluations_heun.append(2 * n)
    evaluations_rk4.append(4 * n)

fig, ax = plt.subplots(figsize=(6, 3.8))
ax.loglog(evaluations_euler, error_euler, "o-", label="forward Euler")
ax.loglog(evaluations_heun, error_heun, "o-", label="Heun")
ax.loglog(evaluations_rk4, error_rk4, "o-", label="RK4")
ax.set_xlabel("evaluations of the tendency")
```

```
ax.set_ylabel("error at $t = 2$")
ax.legend()
plt.show()
```

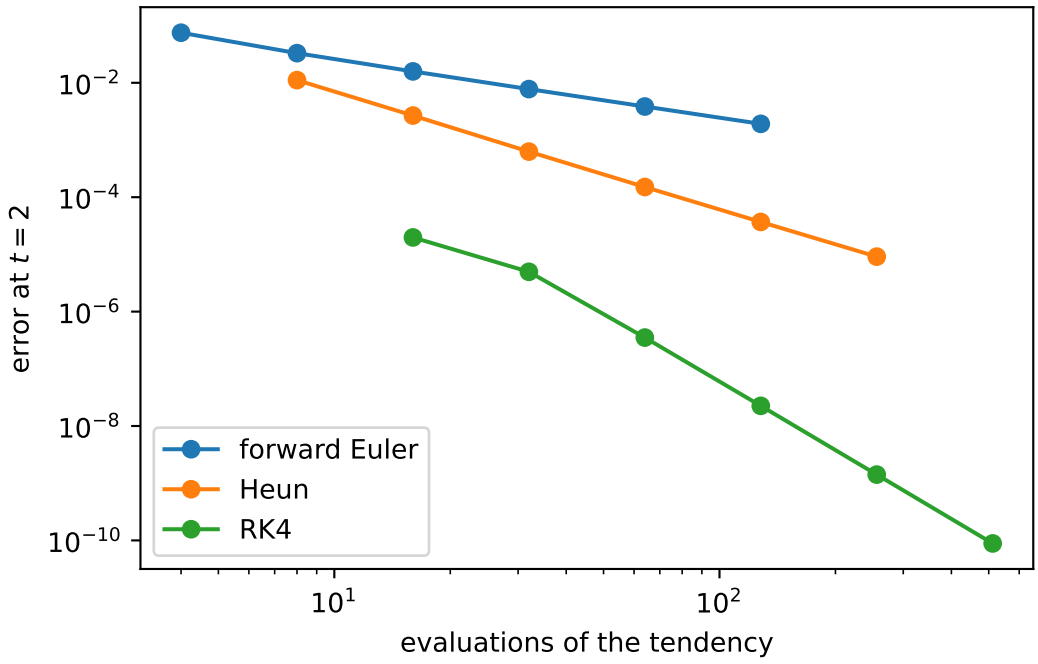


Figure 5.5: Error at $t = 2$ as a function of the number of evaluations of the tendency.

In Figure 5.5 the method of higher order wins as soon as more than a rough approximation is required: with about 100 evaluations RK4 is many orders of magnitude more accurate than forward Euler. Two qualifications apply in practice. The advantage exists only if the solution is smooth. And in models of the atmosphere the time step is often limited by stability rather than accuracy (Chapter 6); then the largest stable time step per evaluation decides.

5.6 Systems of equations

Nothing in the derivations above required u to be a single number. For a system of N equations the state is a vector $\vec{u}$ and the tendency a vector function,

$$\frac{\mathrm{d}\vec{u}}{\mathrm{d}t} = \vec{f}(t, \vec{u}),$$

and all formulas of this unit apply unchanged with vectors in the place of numbers. Equations of higher order are first rewritten as first-order systems (Chapter 2). The functions `rk4_step`, `euler_forward_step` and `integrate` defined above accept a NumPy array as the state; `integrate` then returns a two-dimensional array with one row per time and one column per variable.

As an example we integrate the predator–prey equations of Chapter 2,

$$\frac{\mathrm{d}x}{\mathrm{d}t} = ax - \alpha xy, \qquad \frac{\mathrm{d}y}{\mathrm{d}t} = -cy + \gamma xy.$$

```
a, alpha, c, gamma = 1.0, 0.5, 0.75, 0.25

def tendency(t, u):
    x, y = u
    return np.array([a * x - alpha * x * y, -c * y + gamma * x * y])

u0 = np.array([2.0, 1.0])
t, u_rk4 = integrate(rk4_step, tendency, u0, 40.0, 0.05)
t, u_euler = integrate(euler_forward_step, tendency, u0, 40.0, 0.05)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(9, 3.2), gridspec_kw={"width_ratios":
↪  [2, 1]})
ax1.plot(t, u_rk4[:, 0], label="prey")
ax1.plot(t, u_rk4[:, 1], label="predators")
ax1.plot(t, u_euler[:, 0], "C0--", lw=0.8)
ax1.plot(t, u_euler[:, 1], "C1--", lw=0.8)
ax1.set_xlabel("$t$")
ax1.legend(fontsize=8)
ax2.plot(u_rk4[:, 0], u_rk4[:, 1], "k", label="RK4")
ax2.plot(u_euler[:, 0], u_euler[:, 1], "k--", lw=0.8, label="forward Euler")
ax2.set_xlabel("prey")
ax2.set_ylabel("predators")
ax2.legend(fontsize=8)
plt.tight_layout()
plt.show()
```

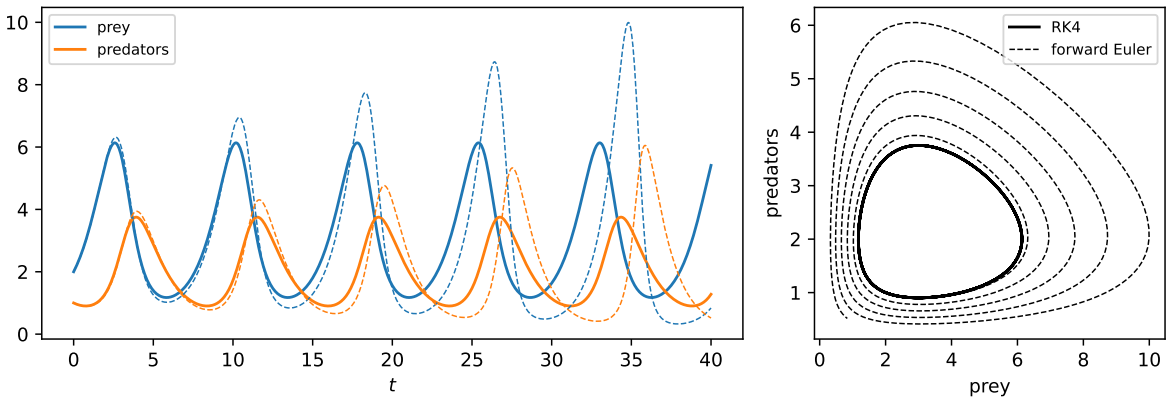


Figure 5.6: Predator–prey equations integrated with RK4 (lines) and forward Euler (dashed) with the same time step $\Delta t = 0.05$. The exact solution is periodic.

The exact solution is a closed curve in the plane of the two populations. RK4 reproduces it. The forward Euler solution spirals outward: the amplitude of the oscillation grows in every cycle. The error does not show up as random noise but as a systematic, qualitatively wrong behaviour. Why forward Euler amplifies oscillations is answered in Chapter 7.

5.7 Adaptive time steps

So far the time step was constant. The local truncation error, however, depends on the solution: for forward Euler it is proportional to u''. A constant step is too small where the solution is smooth and too large where it changes quickly. Adaptive methods choose the step such that the local error stays below a tolerance.

This requires an estimate of the local error without knowing the exact solution. The idea is to compute each step twice, with two methods of different order, p and $p+1$. The difference of the two results,

$$\mathrm{err} = \left| u_{n+1}^{(p+1)} - u_{n+1}^{(p)} \right|,$$

estimates the local error of the less accurate one, which is proportional to Δt^{p+1}. If err exceeds the tolerance tol, the step is rejected and repeated with a smaller Δt. In either case the next step size follows from the requirement that the error, scaled with Δt^{p+1}, should equal the tolerance:

$$\mathrm{tol} = \mathrm{err} \left(\frac{\Delta t_{\mathrm{new}}}{\Delta t} \right)^{p+1} \quad \Longrightarrow \quad \Delta t_{\mathrm{new}} = 0.9 \, \Delta t \left(\frac{\mathrm{tol}}{\mathrm{err}} \right)^{1/(p+1)}, \tag{5.18}$$

with a safety factor of 0.9. **Embedded Runge–Kutta pairs** make this cheap: the two methods share the same stages and differ only in the weights b_i, so the error estimate needs no additional evaluations of the tendency. The default method of `scipy.integrate.solve_ivp`, called RK45, is such a pair of orders four and five.

```
from scipy.integrate import solve_ivp

solution = solve_ivp(tendency, (0, 40), u0, method="RK45", rtol=1e-6, atol=1e-9)

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7, 4.2), sharex=True)
ax1.plot(solution.t, solution.y[0], ".-", ms=4, label="prey")
ax1.plot(solution.t, solution.y[1], ".-", ms=4, label="predators")
ax1.legend(fontsize=8)
ax2.semilogy(solution.t[1:], np.diff(solution.t), ".-", ms=4)
ax2.set_xlabel("$t$")
ax2.set_ylabel(r"$\Delta t$")
plt.tight_layout()
plt.show()
print(f"{solution.t.size - 1} steps, {solution.nfev} evaluations of the tendency")
```

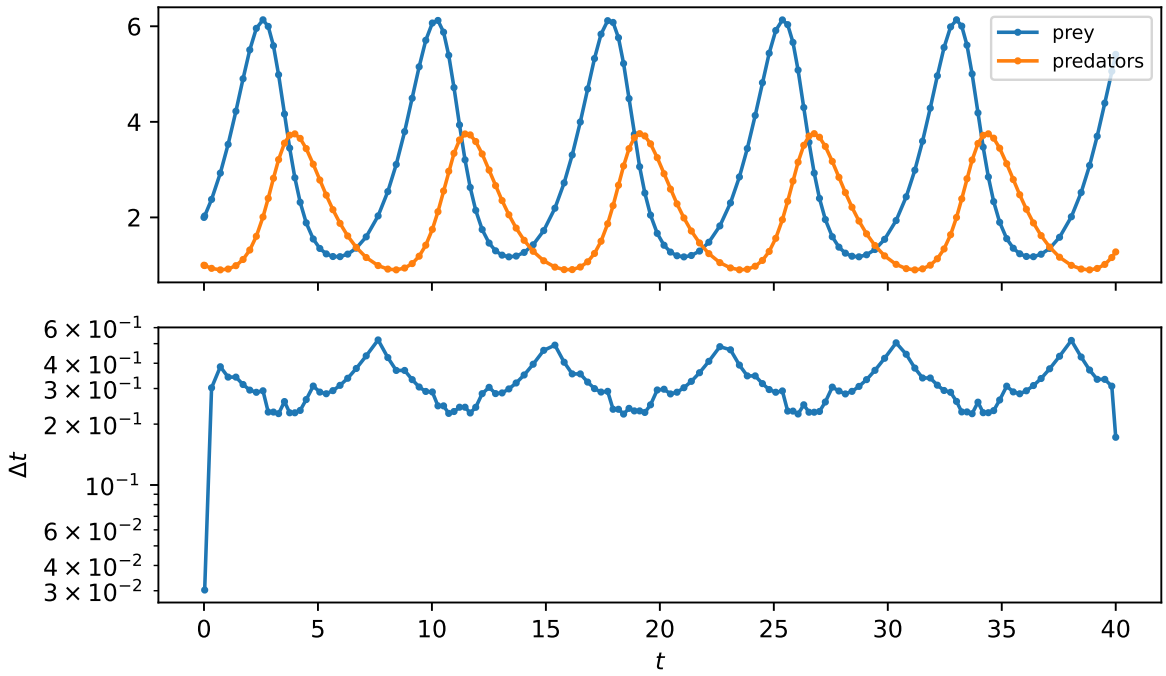


Figure 5.7: Predator–prey equations solved with the adaptive method RK45. Dots mark the time steps chosen by the method (top). The step is small where the solution changes rapidly (bottom).

```
131 steps, 950 evaluations of the tendency
```

Adaptive methods are the standard for ODE problems of moderate size, and for these `solve_ivp` should be preferred to a method written by hand. Large models of the atmosphere and ocean, in contrast, run with a fixed time step. Their step is dictated by stability, the cost of a rejected step is high, and a fixed step keeps the model reproducible and simple to couple with other components.

5.8 Summary

- The forward Euler method follows the tangent, replaces the derivative by a forward difference, or integrates the tendency with a rectangle. It is explicit and of first order.
- The backward Euler method evaluates the tendency at the new time. It is implicit: every step requires the solution of an equation, for nonlinear problems with Newton's method.
- The local truncation error is the error of one step from the exact solution. A local error of order Δt^{p+1} gives a global error of order Δt^p.
- Runge–Kutta methods reach higher order with several evaluations of the tendency per step. Their coefficients follow from matching the Taylor series of the exact solution. Heun's method and the midpoint method are of second order, the classical Runge–Kutta method of fourth order.
- Methods are compared by the accuracy reached for a given number of evaluations of the tendency.
- Adaptive methods estimate the local error from two methods of different order and adjust the time step.

5.9 Further reading

Boyce and DiPrima (2009), chapters 2 and 8, for direction fields, the Euler method and its errors; LeVeque (2007), chapter 5, for one-step methods and their analysis; Durran (2010), chapter 2, for time stepping in geophysical models.

References

Boyce, William E., and Richard C. DiPrima. 2009. *Elementary Differential Equations and Boundary Value Problems*. 9th ed. Wiley.

Durran, Dale R. 2010. *Numerical Methods for Fluid Dynamics: With Applications to Geophysics*. 2nd ed. Springer.

Fornberg, Bengt. 1988. "Generation of Finite Difference Formulas on Arbitrarily Spaced Grids." *Mathematics of Computation* 51 (184): 699–706.

Goldberg, David. 1991. "What Every Computer Scientist Should Know about Floating-Point Arithmetic." *ACM Computing Surveys* 23 (1): 5–48.

LeVeque, Randall J. 2007. *Finite Difference Methods for Ordinary and Partial Differential Equations*. SIAM.

Oberkampf, William L., and Christopher J. Roy. 2010. *Verification and Validation in Scientific Computing*. Cambridge University Press.

Shewchuk, Jonathan R. 1994. *An Introduction to the Conjugate Gradient Method Without the Agonizing Pain*. Carnegie Mellon University.

Wick, Thomas. 2020. *Numerical Methods for Partial Differential Equations*. Leibniz Universität Hannover.